

兼容 Linux 应用的 Rust 微内核的 设计与实现

(申请清华大学工学硕士学位论文)

培 养 单 位： 计算机科学与技术系

学 科： 计算机科学与技术

研 究 生： 朱 懿

指 导 教 师： 陈 渝 副教授

二〇二六年四月

Design and Implementation of a Rust-based Microkernel with Linux Application Compatibility

Thesis submitted to

Tsinghua University

in partial fulfillment of the requirement

for the degree of

Master of Science

in

Computer Science and Technology

by

Yi Zhu

Thesis Supervisor: Associate Professor Chen Yu

April, 2026

学位论文指导小组、公开评阅人和答辩委员会名单

指导小组名单

李 XX	教授	清华大学
王 XX	副教授	清华大学
张 XX	助理教授	清华大学

公开评阅人名单

刘 XX	教授	清华大学
陈 XX	副教授	XXXX 大学
杨 XX	研究员	中国 XXXX 科学院 XXXXXXXX 研究所

答辩委员会名单

主席	赵 XX	教授	清华大学
委员	刘 XX	教授	清华大学
	杨 XX	研究员	中国 XXXX 科学院 XXXXXXX 研究所
	黄 XX	教授	XXXX 大学
	周 XX	副教授	XXXX 大学
秘书	吴 XX	助理研究员	清华大学

关于学位论文使用授权的说明

本人完全了解清华大学有关保留、使用学位论文的规定，即：

清华大学拥有在著作权法规定范围内学位论文的使用权，其中包括：（1）已获学位的研究生必须按学校规定提交学位论文，学校可以采用影印、缩印或其他复制手段保存研究生上交的学位论文；（2）为教学和科研目的，学校可以将公开的学位论文作为资料在图书馆、资料室等场所供校内师生阅读，或在校园网上供校内师生浏览部分内容；（3）按照上级教育主管部门督导、抽查等要求，报送相应的学位论文。

本人保证遵守上述规定。

作者签名： _____

导师签名： _____

日 期： _____

日 期： _____

摘 要

微内核架构虽能提升系统安全性与可靠性，但其发展面临两大核心挑战：其一，现有成熟微内核（如 seL4）的单体式架构导致系统高度耦合，难以演进与维护；其二，用户态系统服务的内存安全无法得到与内核同等级的保障。为应对这些挑战，本文旨在构建一个既具备内存安全特性，又拥有灵活组件化架构的实用微内核系统。

本研究采用 Rust 语言，对经过形式化验证的 seL4 微内核进行了系统性的组件化重构，实现了与之高度二进制兼容的 reL4 微内核。重构过程成功将内核解耦为界限清晰的独立组件（如 CSpace、VSpace、IPC 等），有效提升了代码的可维护性与可复用性。进而，在 reL4 内核的基础上，我们设计了用户态兼容层 reL4-Linux-kit，通过将系统调用转换为 IPC 的机制，使未经修改的 Linux 二进制程序得以直接运行，较好地增强了系统的实用性与生态兼容性。

实验评估表明，reL4 内核通过了大部分的 seL4 官方测试用例，证明了其良好的二进制兼容性；reL4-Linux-kit 能够有效支持多种 Linux 测试集。性能评测显示，当前原型系统与成熟系统相比存在性能差距，但本文也分析并定位了因兼容性约束等原因而暂未优化的关键路径，为后续优化提供了方向。本研究不仅实现了功能目标，更为构建高安全、易演进的操作系統提供了经过实践验证的组件化架构范式与有价值的工程见解。

关键词：微内核；Rust 语言；内存安全；组件化；二进制兼容

Abstract

Although the microkernel architecture can enhance system security and reliability, its development faces two core challenges: Firstly, the monolithic architecture of existing mature microkernels (such as seL4) leads to high system coupling, making it difficult to evolve and maintain. Secondly, the memory safety of user-mode system services cannot be guaranteed at the same level as that of the kernel. To address these challenges, this paper aims to construct a practical microkernel system that not only possesses memory safety features but also features a flexible component-based architecture.

In this study, we employed the Rust language to conduct a thorough component-based refactoring of the formally verified seL4 microkernel, achieving the reL4 microkernel with strict binary compatibility with seL4. The refactoring process successfully decoupled the kernel into well-defined independent components (such as CSpace, VSpace, IPC, etc.), significantly improving code maintainability and reusability. Furthermore, based on the reL4 kernel, we designed a user-mode compatibility layer, reL4-Linux-kit, which enables unmodified Linux binary programs to run directly through a system call-to-IPC mechanism, substantially enhancing the system's practicality and ecosystem compatibility.

Experimental evaluations demonstrate that the reL4 kernel passes most of the official seL4 test cases, proving its excellent binary compatibility. The reL4-Linux-kit effectively supports various Linux test suites. Performance evaluations reveal a performance gap between the current prototype system and mature systems; however, this paper also provides an in-depth analysis to identify key paths that have not been optimized due to compatibility constraints, offering directions for future optimization. This study not only achieves its functional goals but also provides an experimentally verified component-based architectural paradigm and valuable engineering insights for constructing highly secure and easily evolvable operating systems.

Keywords: Microkernel; Rust Language; Memory Safety; Componentization; Binary Compatibility

目 录

摘 要.....	I
Abstract.....	II
目 录.....	III
插图清单.....	VII
附表清单.....	VIII
第 1 章 引言	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 微内核研究现状.....	2
1.2.2 seL4 混合关键系统研究综述	5
1.2.3 微内核兼容宏内核方案研究现状.....	8
1.2.4 Rust 语言特性及其在系统编程中的发展现状	8
1.2.5 使用 Rust 构建组件化内核概述.....	10
1.2.6 国内外研究现状总结.....	11
1.3 微内核架构操作系统的设计原则	12
1.3.1 最小化内核原则.....	12
1.3.2 策略与机制分离原则.....	12
1.3.3 实施细粒度的控制.....	13
1.3.4 系统服务解耦原则.....	13
1.4 关键问题与挑战	14
1.5 研究目标和主要贡献	15
1.6 本文组织结构	17
第 2 章 seL4: 微内核原则的实践	18
2.1 seL4 微内核总体架构	18
2.2 seL4 capability 机制和内核对象管理.....	19
2.3 seL4 的内存管理	23
2.3.1 虚拟内存的管理.....	23
2.3.2 内核所需要的内存分配.....	24
2.4 seL4 线程间通信 (IPC) 机制.....	25
2.4.1 快速 IPC	25

2.4.2 Endpoint 和 Notification 消息传递	26
2.5 seL4 调度与执行模型	27
2.6 seL4 的中断和异常处理	28
2.7 seL4 的混合关键系统 (MCS) 支持	30
2.7.1 seL4 内核中的混合关键系统算法	30
2.7.2 seL4 中的混合关键系统支持实现	32
2.8 seL4 的 SMP 支持	33
2.9 seL4 安全验证与形式化证明	34
2.10 本章小结	35
第 3 章 基于 Rust 语言的组件化 reL4 微内核操作系统	36
3.1 reL4 的挑战和目标	36
3.1.1 挑战与动机	36
3.1.2 设计目标与原则	36
3.1.3 reL4 内核实现与解决的挑战	37
3.2 reL4 对 capability 机制的实现	39
3.3 reL4 的内存管理与虚拟内存子系统	41
3.3.1 架构无关的 VSpace 组件设计	42
3.3.2 策略驱动的 kernel 层内存管理实现	42
3.3.3 组件化协作的实践价值	43
3.4 reL4 线程间通信 (IPC) 机制实现	43
3.4.1 基础定义与架构抽象	44
3.4.2 核心机制与统一传输层 (IPC 组件)	44
3.4.3 策略执行与兼容层 (Kernel 组件)	45
3.4.4 reL4 的 IPC 组件化设计的价值	45
3.5 reL4 的任务管理与调度	46
3.6 reL4 在特定架构下的异常处理与系统优化	47
3.6.1 ARM FPU 支持	47
3.6.2 ARM 的 SMC call 的支持	48
3.6.3 ARM 的用户态时钟读取支持	49
3.7 reL4 的混合关键系统 (MCS) 支持	49
3.7.1 新的内核对象与内存管理	49
3.7.2 时钟系统、消息系统与调度器的改造	50

3.8 reL4 的多处理器 (SMP) 并行支持.....	51
3.8.1 每核 (percpu) 变量的抽象与管理.....	52
3.8.2 大内核锁的实现.....	52
3.8.3 核间通信与细粒度调度.....	53
3.9 reL4 构建系统与工具链支持.....	53
3.9.1 组件化 Rust 项目的大小仓库开发范式.....	53
3.9.2 reL4 的 xtask 构建脚本支持.....	55
3.9.3 reL4 的 pbf 文件格式解析支持.....	55
3.9.4 reL4 的集中配置脚本支持.....	56
3.10 本章小结.....	56
第 4 章 reL4-Linux-kit 用户态兼容层.....	58
4.1 reL4-Linux-kit 设计目标、原则和总体架构.....	58
4.1.1 挑战与动机.....	58
4.1.2 设计目标与原则.....	59
4.2 root-task 与用户态进程管理.....	60
4.3 Linux 用户态程序的二进制兼容机制.....	61
4.3.1 app 主动调用 syscall 的处理.....	61
4.3.2 app 在运行过程中触发错误行为.....	62
4.3.3 app 运行过程中外部设备发送消息的行为.....	63
4.3.4 reL4-Linux-kit 和其他 Linux 兼容方案的区分.....	63
4.4 函数调用和 IPC 通信的透明转换机制.....	64
4.4.1 Rust 语言条件下函数调用和 IPC 调用两套代码并存的方案.....	66
4.4.2 灵活可配置的组件间函数调用或 IPC 调用选择方案.....	67
4.4.3 函数调用和 IPC 调用的消息传递一致性解决方案.....	69
4.4.4 reL4 下的 IPC 调用适配方案.....	71
4.5 用户态核心服务组件.....	71
4.5.1 kernel thread.....	71
4.5.2 其他系统服务组件.....	72
4.6 reL4-Linux-kit 的运行样例分析：以文件系统服务为例.....	73
4.7 本章小结.....	74
第 5 章 实现与评估.....	76
5.1 系统实现情况.....	76

5.2 功能实现评估	77
5.2.1 reL4 对 seL4 二进制接口的功能实现评估的测试环境	77
5.2.2 reL4 对 seL4 二进制接口的功能实现评估	78
5.2.3 reL4-Linux-kit 的运行环境	79
5.2.4 reL4-Linux-kit 对 Linux syscall 的实现评估	80
5.3 reL4 性能评估	82
5.3.1 reL4 内核性能评估的测试环境	82
5.3.2 One Way IPC 测试	82
5.3.3 中断相关测试	84
5.3.4 信号相关测试	85
5.3.5 同步相关测试	85
5.3.6 映射相关测试	87
5.4 reL4-Linux-kit 综合性能评估的测试环境	89
5.5 reL4-Linux-kit 综合性能评估	89
5.6 本章小结	91
第 6 章 总结与展望	93
6.1 全文总结	93
6.2 未来工作展望	94
参考文献	95
附录 A 其他附表	99
致 谢	107
声 明	108
个人简历、在学期间完成的相关学术成果	109
指导教师评语	110
答辩委员会决议书	111

插图清单

图 1.1	本文主要研究内容所处的软件层面	15
图 2.1	seL4 内核总体架构设计	18
图 2.2	CSpace 查找样例	22
图 2.3	seL4 中断处理流程图	29
图 3.1	reL4 内核总体架构设计	37
图 3.2	CSpace 组件架构	41
图 3.3	VSpace 组件架构	42
图 3.4	IPC 组件架构	44
图 3.5	reL4 构建流程	54
图 4.1	reL4-Linux-kit 架构图	60
图 4.2	reL4-Linux-kit 下 syscall 转为 IPC 方案	62
图 4.3	reL4-Linux-kit 下 page fault 转为 IPC 方案	63
图 4.4	reL4-Linux-kit 下 timer 信号转为 IPC 方案	63
图 4.5	reL4-Linux-kit 的 IPC 和函数调用双向透明转换机制	65
图 4.6	reL4-Linux-kit 的组件组合方案	65
图 4.7	reL4-Linux-kit 示例	73
图 5.1	同一地址空间下的 one way ipc 性能对比	83
图 5.2	不同地址空间下长度为 0B 的 one way ipc 性能对比	83
图 5.3	不同地址空间下长度为 10B 的 one way ipc 性能对比	84
图 5.4	irq 性能对比	84
图 5.5	signal 性能对比（向高优先级进程发送）	86
图 5.6	signal 性能对比（向低优先级进程发送）	86
图 5.7	不同线程数量下等待时间对比	87
图 5.8	内存映射的性能对比	88

附表清单

表 2.1	seL4 的主要 capability 类型及简要说明	20
表 2.2	seL4 的以消息传递为主的 syscall 设计	26
表 4.1	TOML 中可配置信息列表	67
表 5.1	reL4 未测试的测例分类及其数量	78
表 5.2	已实现但未被 Linux 测例使用的系统调用	80
表 5.3	Linux 测例需要但未实现的系统调用	81
表 5.4	Linux 测例在不同操作系统配置下的性能测试结果（单位：秒）	90

第 1 章 引言

1.1 研究背景与意义

操作系统的架构演进始终围绕提升安全性、可靠性与可维护性等核心目标展开。宏内核（Monolithic Kernel）虽在市场中长期占据主导地位，但其高度耦合的设计导致内核结构臃肿、漏洞难以彻底根除，且单一组件的故障极易扩散至整个系统。相比之下，微内核（Microkernel）架构遵循“机制与策略分离”的原则，将文件系统、设备驱动等绝大多数系统服务移出内核，以用户态进程形式运行，从而在理论上实现了更小的可信计算基（TCB）、更强的隔离性及更优的可维护性。

尽管如此，微内核在其演进过程中仍面临两个关键挑战：

其一，内核自身的耦合性问题。尽管微内核在理念上追求最小化，但现有典型实现（如 seL4^[1]）仍将进程调度、内存管理、进程间通信（IPC）等核心机制紧密封集成于单一内核映像中。这种内部耦合性不仅限制了内核功能的灵活演进与定制化扩展，也阻碍了其核心组件在不同项目中的复用。一个直观的例证是，seL4 内核最初完成形式化证明的核心代码仅有约 8700 行 C 语言代码，其代码的精炼性是其能够被彻底验证的关键。然而，随着对更多硬件架构和系统特性的支持，其代码量急剧膨胀——在项目源码的 kernel 目录下，C 代码已超过 57000 行。相较于初始版本，代码规模增长数倍，且引入了大量具有复杂依赖关系的条件编译特性，这为内核的进一步迭代与维护带来了不小挑战。

其二，用户态服务的兼容性和安全性问题。一个完整的操作系统生态远不止于内核本身。即使内核本身经过形式化验证，若用户态系统服务（如设备驱动、网络协议栈等）仍使用 C/C++ 等内存不安全的语言实现并不加以严格验证，整个系统的安全防线仍存在关键薄弱环节。攻击者可利用用户态服务中的内存漏洞绕过内核的安全机制，实施内核防护无法有效阻止的攻击。而对整个系统进行形式化验证，目前仍面临高昂的成本与较大的技术复杂性。

本文重点研究组件化微内核系统这一可能的微内核系统演进方向。组件化微内核系统包括了微内核操作系统的内核，以及在微内核基础上实现的必要的用户态系统服务，这些内核和服务，都用组件化的思想进行构建，以追求更好的组件可复用性和可扩展性。

本文主要内容将分别围绕内核态和用户态的组件化微内核系统实践展开。本章的剩余部分，首先将回顾理解组件化微内核实践所必需的背景知识。这包括：微内核的发展历程、seL4 微内核的关键技术与基本原则，以及 Rust 语言在编写内核

方面的现状等内容，然后给出在构建组件化微内核系统时面临的一些困难与挑战，最后介绍了本文的主要工作与贡献。

1.2 国内外研究现状

1.2.1 微内核研究现状

微内核从发展到现在，也经历多个阶段，本节按时间顺序，对微内核的发展，进行了简要的梳理，并列举了其中的经典代表及其具体特点。

1.2.1.1 第一代微内核

第一代微内核主要诞生于学术界和工业界探索新的操作系统架构的背景下，其中最经典的代表包括 Mach^[2]、Minix^[3]等探索性的微内核。他们的主要努力方向为致力于将传统内核的文件系统和设备驱动等复杂功能迁移到内核外，以提升系统的模块化程度，可靠程度以及安全性。

这些微内核验证了微内核架构的可行性，证明了在一个只有进程管理，内存管理以及 IPC 的基本操作系统内核之上也能构建一个实用的操作系统，同时对后续的微内核设计提出了许多的概念性上的引领，如 Mach 基于 Port 和 Right 的通信模型和迁移线程的 C/S 运行模型等等，奠定了现代微内核操作系统的基石，也对当下主流操作系统的实现产生了重要影响，例如 macOS 的 XNU 内核^[4]就是基于 Mach 微内核进一步开发的混合架构的操作系统。

Mach 微内核由卡内基梅隆大学开发，其基于端口和消息的 IPC 方法，试图使用微内核架构兼容当时流行的 Unix 操作系统。Mach 微内核操作系统所提出的基于端口和消息的 IPC 的方法，不仅为内核中的消息分发提供了基础，其对消息接收方的位置透明性封装，即无需知道消息接收者到底是本机上的一个进程还是远程网络上的一台机器这一特性，也为分布式计算提供了便捷的接口。同时，其为了减少数据拷贝而提出的 copy-on-write 等机制，也成为后续操作系统的重要特性。当然，提到 Mach 微内核，更为人熟知的就是在操作系统发展早期，其和 Linux 操作系统的经典论战^[5]，这在根本上奠定了遵循实用主义的 Linux 成为后来内核发展的主流。但 Mach 也并不能视为完全失败，其在 XNU 内核上的实践，依然证明其 IPC 方案具有独到的设计。

作为同样的第一代微内核，Minix 则是一个主要面向教学用的操作系统，其核心设计目的是通过策略与机制分离提供高可靠性，同时，也有其作为教学操作系统的鲜明特点，为了教学的清晰性，会存在牺牲其性能等方面的考量。即便相比于 Mach，MINIX 更侧重于教学目的而非高性能，但依然影响深远，其极简化的设

计和对高可靠性的追求，对后续的微内核设计产生了深刻的影响，Minix 开发者编写的《操作系统设计与实现》^[6]作为一本操作系统教学的经典教材，也影响了众多操作系统开发者。MINIX 的一个变种至今仍被用于 Intel 的管理引擎（ME）中^[7]，体现了其长久的生命力。

第一代微内核更多是一个探索性的工作，其影响深远，但由于缺少对微内核在实践中可能遇到的困难的了解，导致了显著的 IPC 性能瓶颈等问题。而后续的微内核，则基于这些探索，吸收了其中的精髓，并努力克服了其中的问题，更进一步地探索了微内核的实现形式。

1.2.1.2 第二代微内核

以 Mach 为代表的第一代微内核方案由于其存在的 IPC 性能问题，最终没有成为主流操作系统的技术路径选择。这引发了后来者对微内核操作系统的深刻反思，并诞生了第二代的微内核 L3^[8]以及 L4^[9]等内核，除此之外，这个阶段以黑莓的 QNX^[10]为代表的其他微内核操作系统，则对微内核的实时性能等方向进行了探索。

以 L4 微内核为代表的第二代微内核在反思第一代微内核的问题的基础上，开发了高性能的 IPC 方案，方案包括但不限于使用寄存器和 L1 缓存来进行消息传递，为了能够充分利用缓存，甚至完全使用汇编语言来编写最初版本的操作系统。这种深度的优化目的带来了相比于第一代微内核而言的充分的性能提升，其 IPC 效率相比于 Mach 提升在 20 倍以上，有效打破了微内核低效的刻板印象。

在 L4 之后，也产生了 Fiasco^{[11][12]}，OKL4^[13]等诸多借鉴了 L4 思路的微内核，构成了 L4 微内核家族^[14]，有关 L4 微内核演化和 Fiasco 等内核技术细节的完整论文列表，也可参阅德累斯顿理工大学操作系统教研组维护的权威书目^[15]。

但是这种性能提升是有其代价的。L4 内核只是一个通信的中间件，而不对其上转发的消息做任何检查。这也带来了一些安全上的问题。例如，假设系统中存在一个恶意的软件，不停地对目标服务进程发送消息实施 DDoS 攻击，即使无法通过目标服务进程中的安全性检验，但依然会不停地占用目标服务进程的信道，从而造成目标服务进程无法给其他进程提供服务等问题。换言之，L4 微内核自身的安全性较弱，仍然一定程度上依赖于用户态的服务和应用本身的安全性。另外，L4 内核家族本身虽然都是由 L4 微内核衍生出来的，却很少考虑同一操作系统家族之内的系统兼容性，也进一步加剧了生态的碎片化。

而黑莓的 QNX 微内核在微内核架构的发展上，则走向了另一个方向，它以可靠性、安全性和实时性等方面著称，在车规级虚拟化技术领域，基于 QNX Neutrino 微内核构建的 QNX Hypervisor 2.0 于 2019 年获得德国莱茵 TÜV（TÜV Rheinland）

颁发的 ISO 26262 ASIL D 认证，是全球第一个通过了最高等级车规安全认证（ISO 26262 ASIL-D）的商用虚拟机管理程序^[16]。QNX 本身也具有 POSIX 的兼容性，因此相比于 L4 内核家族，具有生态更加丰富等优点，其目前在车载操作系统，医疗器械操作系统以及机器人操作系统等需要实时领域内核支持的操作系统应用场景下，具有广泛的适用性和优势。当然 QNX 本身实际上是一个商业软件，其具体开发使用也需要授权，这也是该微内核和其他微内核项目的一个重要不同之处。

总体而言，第二代微内核的开发，在吸取了第一代操作系统在实践上的经验教训之后，进一步地进行了开发，无论是 L4 系列家族对微内核新的形态的探索，还是保持对 POSIX 等通用接口兼容性，并提升其安全性的 QNX，都对微内核发展进行了有益的探索。

1.2.1.3 第三代微内核

第三代微内核主要是以 L4 家族的后继者 seL4^[1]为代表的。seL4 微内核的最大特点即为通过全面的形式化验证实现了前所未有的可靠性，安全性和隔离性，同时作为 L4 微内核的继任者，又具有可观的高性能。seL4 的实现细节将在本文的第2章详述，故不在此赘述。

近年来，也有类似于吸收了 seL4 微内核相关思想的其他微内核实现，例如，zircon 微内核是谷歌为了实现 Fuchsia OS 开发^[17]的微内核，其吸收了基于句柄的访问控制思想，实现了细粒度的安全控制，但 zircon 微内核提供了超过 150 个系统调用，也在内核中提供了更多的其他功能，以提供更为实用的操作系统内核功能，减少在不同进程地址空间来回切换的开销。其体现的是工程驱动的实用主义设计理念。

1.2.1.4 微内核最新的一些研究成果

对微内核的最新探索聚焦于微内核本身的多个方面。

首先，聚焦于传统微内核的 IPC 短板，新的各类基于硬件的 IPC 加速方案被提出。IPADS 实验室提出的 XPC 方案^[18]，添加了专用的 XPC 引擎单元并设计了 XCALL 和 XRET 指令进行消息的加速传递，整个消息发送均放在用户态，具有性能优势，但也有增加了新硬件设计，需要修改硬件和设计专用的内核等问题。而 skybridge^[19]的方案则使用现有的 Intel 处理器上 vmfunc 指令具有执行速度优势这一点，使用基于虚拟化的隔离，并通过 vmfunc 指令进行在不同虚拟机环境下的系统组件之间的通信，该方案在无需修改硬件的基础上，相比于 seL4 微内核的通信方案有性能提升。

其次，既然微内核的实现思路是使用基于 MMU 的地址空间隔离，来进行微

内核的组件间隔离，那么随着近些年的新兴硬件隔离方案的产生，使用硬件进行组件隔离的方案来进行微内核的组件间隔离也成为一种研究方向。

在 x86 架构下的 Intel 机器上，提供了 MPK 等硬件特性，可以在无需切换地址空间的前提下，允许将不同的物理页面标记其属于不同的保护域，在不同保护域之间的切换相比于地址空间的切换，开销极低，但也有保护域数量有限，最多只能使用 16 个的限制，除了地址空间的隔离，x86 架构下往往结合 Intel CTE 技术来实现控制流的隔离，从而实现更为全面的组件隔离。

ARM 则提供了 MTE 和 PA 两种技术，MTE 为每个内存赋予一个内存标签，并在访问时，验证指针和内存标签的匹配性，从而进行内存的保护，而 PA 则是利用指针中未被使用的高比特位存放加密的指针认证码，从而防范控制流的劫持。

除此之外，CHERI (Capability Hardware Enhanced RISC Instructions)^[20]则像 seL4 一样，基于能力机制为指针提供了更全面的检查，在最新的 Morello 硬件实现中，CHERI 项目也提供基于 ARM 的实现方案。

另外，在基于硬件隔离的方案之外，使用语言特性等软件方案进行隔离的方案，也是微内核隔离方面的另一个重要工作方向。以 redleaf^[21]为代表的工作，利用了 Rust 语言的生命周期等机制，对处于同一个地址空间的内核组件进行了编译期的检查，结合堆隔离的设计，保证了多个处于同一个地址空间的内核组件能够相互隔离。

华为的鸿蒙微内核^[22]则对微内核在商业化落地的场景做了更多的探索。鸿蒙更进一步地探索了在大规模用户态进程下的真实应用场景下，如何灵活地解决微内核的性能问题，如何在一个新的微内核的基础上兼容现有的 Linux 和安卓生态等问题。

1.2.2 seL4 混合关键系统研究综述

混合关键系统 (MCS) 调度理论是实时系统领域的重要分支，其为在单一硬件平台上整合不同安全关键级别的任务提供了理论基础，自 2007 年 Vestal 等人^[23]提出来之后，便成为实时系统研究中新的研究方向之一。seL4 微内核作为高安全保障的系统基石，其引入 MCS 支持并非偶然，而是为了将形式化验证的安全性与实时调度能力相结合，以满足航空、汽车等安全关键领域对操作系统的新需求。要理解 seL4 中 MCS 设计的深意，需首先对其理论发展脉络有一个清晰的认知。

混合关键系统的主要思想如下：在一个实时系统中包含不同的任务，他们存在着不同的关键级别，即，不同的任务错过截止日期后所带来的后果是不完全一样的，如在实时车机上，就存在许多关键性安全功能（如动力系统控制，ABS 系统等）和非关键性功能（如车载导航系统，音乐等），整个系统的目标应当为，在

保证高关键性任务不错过截止时间的前提下，尽最大可能的安排低关键性的任务。

在单核的混合关键系统研究上，最早是 Vestal 提出了相应的 MCS 相关的模型，并认为在其所设定的条件下，Audsley 的 priority assignment 算法^[24]相比于 RM (rate monotonic) 和 deadline monotonic 均更优，该算法在响应时间分析的基础上，以 $O(n^2)$ 的时间复杂度将不同任务放置在多个优先级上。

随后 Baruah 等人^[25]给出了单核情况下 MCS 的 AMC 通用分析框架，该框架在前述 MCS 模型的基础上，不仅设定了每个任务的关键级别，也设定了系统能够在多个不同的关键级别上，最开始系统在低关键级别按照优先级调度所有任务，当某个任务发生超时，将系统转换到高关键级别，并在高关键级别下丢弃所有低关键级别任务的运行。这成为了后续混合关键系统研究的通用框架。

在此基础上，EDF-VD 算法^{[26][27]}引入了虚拟截止时间，具体来说，在离线预处理阶段判断是否可调度并生成相应的运行参数 k 和每个任务的虚拟截止时间，在运行调度期间，如果系统任务的关键级别小于等于 k ，那么就用该任务的虚拟截止时间运行，否则使用原始截止时间。该方案呈现出来的方案和 AMC 的框架具有类似的思想。

在多核混合关键系统研究上，其调度算法主要分为三类：全局调度 (global Scheduling)，分区调度 (Partitioned Scheduling) 以及半分区调度 (semi-partition Scheduling)，对于分区调度，实质上 and 单核的情况下并无太多不同，因此不做介绍。最早提出对多核处理的是 Mollison^[28]，它提出的 (MC^2) 方案，使用了五级的关键性级别，在高关键级别使用分区的单核算法，而低优先级则使用全局的分配方案。

值得一提的是，在该论文的后续工作^[29]中，对 (MC^2) 方案情况下末级缓存和 DRAM 的隔离性进行了一些分析，并通过页面着色等方案来试图完成隔离。

例如，Baruah 等人^[30]开始将 EDF-VD 方法应用于多核的全局调度方案研究，其基本框架仍然基于 EDF-VD，允许任务在多核上进行任意迁移，并和传统的非 MCS 的 fpEDF 全局调度算法进行对比。Gratia 等人^{[31][32]}则使用了一个 RUN 调度器和他的改进版本 GMC-RUN 调度器的方案来进行全局调度。

对于半分区调度的方案，Al-bayati 等人^[33]提出了双分区的混合关键性算法 DPM，该算法在高关键性模式和低关键性模式下都进行分区，但允许低关键性任务在发生关键性级别切换时，进行有限的迁移以提高资源利用率，除此之外，该论文还在双分区的情况下，对 Worst-Fit, Best-Fit, First-Fit 三种划分方案共九种组合进行了评估。相比之下，Xu 等人^[34]对于分区迁移的方案则不同，在某个核心上发生关键性级别切换之后，所有的低关键性任务都会迁移到没有发生关键性级别切

换的其他核心上（如果存在这样的核心）。Naghavi^[35]则将半分区的迁移方案和可靠性管理等结合起来。

另一种半分区的划分方法则是按照多核所在的 cluster 进行划分的。例如 Ali 等人^[36]给出了一个在低关键性级别使用更小的 cluster 而高关键性级别使用大 cluster 进行调度的方案。

在多核情况下，核间的资源竞争会对单个核心上的任务产生未知的影响。前面已经提到过 Chisholm 等人^[29]针对末级缓存以及 DRAM 对多核 MCS 的影响的相关工作，Kim 等人^[37]也对 DRAM 控制器相关的隔离提出了一些建议。

Kritikakou 等人^[38]针对多核 MCS 系统中，由于总线竞争导致的某个核心上高关键性任务被另一个核心上低关键性任务所延迟的情况进行了分析，并提出了对应的解决方案^[39]，该方案对高关键性任务插入若干观察点，以观察该高关键性任务是否能按时完成，并通过分布式的控制方案，控制其他核心的低关键性任务。除此之外，Yun 等人^{[40][41]}也对此进行了一些分析，Freitag 等人^[42]将这这种由于多核互相干扰导致的执行速度变慢，抽象为虚拟时间的减速，并基于此进行了分析。

当然，除了混合关键系统的算法以外，针对混合关键系统模型本身的各个要素也有各类研究。

在关键性级别的数量上，最初的 AMC 框架只有双关键性级别的分析，而 Fleming^[43]、Ekberg 等人^{[44][45]}增加了对离散多关键性级别的分析，尤其是针对航空领域 DO-178B 等标准所具有的五级关键性级别。除此之外，pWCET^[46]提出了 WCET 的概率分布，而 Ranjbar 等人^[47]更进一步的将在不同关键级别上的执行时间改为概率分布的执行时间，并以此为基础使用切比雪夫大数定律做出相应的分析，有效提升了 MCS 系统的吞吐率。

在 AMC 框架下，最初的原文在将系统转换到高优先级之后，不再返回低优先级运行，而 Ekberg 等人^[44]则分析了高优先级切换回低优先级的情况，并进一步地使用有向无环图的方式来描述相应的模式切换情况并进行了分析^[48]。

FMC^[46]则针对 EDF-VD 框架下的关键性切换进行了优化。在 EDF-VD 模型中，一旦存在高关键性任务超时，则整个系统就被切换到高关键性级别上，但是 FMC 则提出，只让超时的任务运行在高关键性级别上，这允许高关键性任务的模式切换相互独立，大幅提升了系统资源利用率，对低关键级别的服务质量也有明显提高。后续的工作^{[49][50]}都继续基于此继续改进多核情况下的，进一步在保证高关键性任务的前提下，提升低关键级别的服务质量。

除上述的因素之外，一些额外的考虑，比如处理器存在动态频率问题^[51]，系统功耗考量^[52]等也被添加到 MCS 系统中。

综上所述, MCS 调度理论历经十余年发展, 已从单核调度模型演进为涵盖多核协同、资源隔离、模型扩展的复杂体系。而 seL4 微内核的创新^[53]在于, 它并未直接采用上述某一种复杂的调度算法, 而是选择了经典的 Sporadic Server 算法^{[54][55]}作为其内核机制, 巧妙地将“时间”作为一种可通过能力模型进行管理和隔离的资源(其具体实现见第2.7节)。这种设计充分体现了微内核的“机制与策略分离”原则: 内核提供简单、可验证的预算管理机制, 而将复杂的调度策略决策权留给用户态。这为本文第3章中在 reL4 内核实现 MCS 特性奠定了理论基础, 并启示了我们如何在一个组件化的架构中优雅地集成此类特性。

1.2.3 微内核兼容宏内核方案研究现状

L4Linux^[56]是微内核与宏内核架构融合的一次重要实践, 其核心思想是将一个完整的 Linux 内核作为 L4 微内核的一个用户进程运行, 用户态的 Linux 应用的系统调用变为一个 IPC, 转发给同样在用户态的 Linux 内核进程, 而 Linux 内核的硬件资源管理, 进程管理则主要依赖于 L4Linux 来进行。

L4Linux 项目继承于 MkLinux 项目^[57], 该项目意图在 Mach 微内核的基础上运行 Linux, 但是 MkLinux 项目受限于 Mach 微内核高昂的 IPC 开销, 而 L4 微内核在 IPC 上的出色性能, 为 L4Linux 这一个项目的实现提供了基础。

该项目的实现仍然也有一些问题, 首先, L4Linux 项目本身并不能避免 Linux 内核本身的 Bug, 其次, 其本身仍然是微内核的架构, 也意味着其在 IPC 上仍然存在劣势, 最后, L4Linux 需要一定程度上改动 Linux 以适配 L4 内核, L4 微内核本身代码量不多, 相对稳定, 但是 Linux 是一个在不断迭代的内核, 因此, 需要不断地进行适配工作且工作量很大, 而没有一劳永逸的适配方案。即便如此, L4Linux 项目仍在不断地维护适配中。尽管存在诸多缺点, L4Linux 项目的成功, 仍然为微内核与宏内核架构融合提供了开创性的示例。

1.2.4 Rust 语言特性及其在系统编程中的发展现状

Rust 语言由 Mozilla 研究院于 2010 年首次发布, 其设计目标是在不牺牲性能的前提下, 提供编译期的内存安全和并发安全。近年来, Rust 因其卓越的特性, 正被逐步引入到 Linux、Windows 等主流操作系统内核中, 标志着系统编程领域正在发生重要变革。

Rust 的安全保障并非通过运行时垃圾回收 (GC), 而是通过一套在编译期进行验证的严格规则系统。

- **所有权系统与借用检查器:** Rust 语言设计了一套所有权系统与借用检查器, 每个值都有一个称为其所有者的变量。值在任何时候有且只有一个所有者。

当所有者离开作用域，这个值将被丢弃（内存被释放）。除此之外，通过创建引用，Rust 允许值被借用，而不是获取其所有权。借用分为：不可变借用（`&T`）：允许只读访问，同一时间允许多个不可变借用。可变借用（`&mut T`）：允许读写访问，同一时间只能有一个可变借用，且与不可变借用不能共存。这套机制让 Rust 在编译期就杜绝了 `Use-after-free`、`Double-free` 等内存管理中的问题。

- **生命周期**：作为所有权系统的补充，Rust 的生命周期机制确保了每个引用在其有效范围内被使用。
- **并发安全**：Rust 语言还为并发提供了一些类型上的支持，如 `Send Sync` 等 `trait`，提供了 Rust 语言的多线程安全性。而一个类型要能安全地用于多线程上下文，它必须满足 `Send` 和/或 `Sync trait` 的约束。

Rust 语言的提出，一方面为现有的内核项目提供了内核开发语言的新选项，另一方面也为开发新的操作系统内核提供了全新的语言选项。

以 Linux 为代表的传统内核，主流上以 C 语言进行开发。C 语言在内核开发上固然具有明显的开发优势，即对硬件行为的极致控制，但也意味着所有的硬件行为都需要程序员的管理，在内存管理、无序并发等容易引发 bug 的地方，需要颇具经验的程序员经过仔细的斟酌和衡量甚至复杂的调试，即便如此，仍然难以从根本上杜绝各类 C 语言不加检查所带来的隐蔽 bug。

从 Linux6.1 开始，内核社区正式接纳了 Rust 作为内核开发的第二门语言^[58]，和 C 语言通过外部函数接口（FFI）的方式共同协作进行内核的开发。目前，Rust for Linux 项目主要聚焦于逻辑相对独立，同时代码质量参差不齐，各类内存和并发 Bug 众多的驱动程序，展现出了良好的效果。

基于 Rust 语言从头开发的新操作系统内核也层出不穷。一些用于教学目的的探索性操作系统内核，如 rCore 等操作系统^[59]已经在清华大学等高校的操作系统教学中，得到了有效应用。而面向通用性的操作系统，如 Redox 项目^[60]，则展示了使用 Rust 开发整个系统软件栈的可行性与实用性。

在使用 Rust 语言编写微内核操作系统方面，李龙昊^[61]做了最早期的探索，在 RISC-V 架构下，使用 Rust 语言实现了 seL4 的部分代码，通过将 Rust 代码作为库代码和 C 内核链接的方式来实现，并使其编写的 reL4 内核通过了部分 seL4test 的测试。廖东海^[62]^[63]等人，则在李龙昊的基础上，进一步地完善了 reL4 内核的工作，在 RISC-V 架构下，利用 RISC-V 的 N 扩展，添加了用户态中断的异步支持，试图以此提升 reL4 内核的 IPC 性能。李龙昊、廖东海等人的系列工作，为本研究探索组件化微内核系统提供了前期基础和有益参考。

Rust 语言的一些高级特性，也为操作系统等系统编程领域，提供了新的可能。例如，Rust 的代码配置方案基于 Cargo.toml 文件，其能够清晰地给出系统编程中不同组件间的依赖关系。基于这一特性，ArceOS^[64]等 Rust 实现的操作系统也践行了组件化的系统开发方式，其按照底层复用、上层定制等原则，实现了一系列可复用的、具体操作系统无关的 crate 组件和一系列具体操作系统功能相关的 modules 组件。基于这些不同类型的组件，可以构造出一系列组件可复用，功能各异，甚至形态不同的定制化操作系统集合。

总之，Rust 作为一门新兴的系统开发语言，已经在各类系统开发中，相比于传统 C 语言开发的内核，展现了安全性优势。

1.2.5 使用 Rust 构建组件化内核概述

组件化内核，顾名思义，就是使用组件化的方式来构造内核。它将操作系统的核心——内核，视为一项复杂的软件工程并使用组件化的思想进行编写。在这一过程中，打破了传统内核开发的模式，让各个组件独立开来编写，使得程序员能够如同编写普通程序般方便地编写内核组件，提升了开发的灵活性与效率。

更为重要的是，一个经过精心定义与验证的组件，如同构建软件大厦的预制砖块，能够在多个项目或系统中灵活复用，较为有效地避免了重复劳动与资源浪费，实现了核心组件在不同项目中的高度复用。这种可重用性，不仅提升了开发效率，也有利于保证代码质量的一致性与稳定性。

在后续的维护阶段，组件化也具有优势。通过将系统划分为清晰界定的组件，开发者能够更加精准地定位问题所在，从而实施更加高效、精准的修复与升级策略。此外，由于组件间的松耦合设计，对某一组件的单独升级或改造几乎不会影响到系统的其他部分，这有效地降低了维护的复杂性与成本，确保了系统的持续稳定运行。

总之，组件化开发方式以其独特的优势，在大型软件工程的构建中发挥着重要的作用。现有的主流操作系统也一定程度上使用了组件化的思想，如 Linux 内核中，内核模块这一功能的设计就可以被视作一种粗粒度的组件化形式。

组件化内核的思想，主要遵循着底层复用，上层定制的原则来实现。从下到上，内核中的组件被分为了三层：最底层是与具体操作系统无关的组件，中间是与具体操作系统相关，但是能在重构内核形态等操作时，毫无影响的接入新形态的操作系统的组件，最上面是与操作系统具体形态、具体功能强烈相关的组件。

组件之间可能存在一定的依赖关系，一个解耦合良好的组件化操作系统，通常遵循以下的依赖处理关系：1、组件间具有明确的依赖关系。2、组件间不能存在反向依赖关系。3、组件间不能存在循环依赖关系。

另外，一个好的组件化操作系统，每个具有明确边界的组件，都应当能够以具有同样边界但是内部实现不同的同类组件进行替换，从而提供不同的性能特点，使得操作系统可以真正地像“搭积木”一样搭建起来。

而使用 Rust 语言在组件化操作系统的实现与支持上，也具有独特优势。

Rust 语言具备良好的层次化模块系统，Rust 的 `mod` 关键字用于定义模块，同时支持模块嵌套等操作，便于形成清晰的代码层级结构。Rust 语言可以通过 `pub` 关键字来管理模块中相关数据和方法的可见性。通过在 `Cargo.toml` 文件中设置 `workspace` 等内容，可以方便的引入外部模块。另外，模块系统和文件系统具有一致性，为项目组织管理带来了便利。

Rust 语言也具有多种适合于组件化的语言特性。比如所有权机制，不仅有利于在语言层面实现各个组件之间清晰的边界，为不同的组件所拥有的堆对象提供清晰的所有权机制，更是内核中安全管理内存和资源的编译时保障；再比如 `trait` 机制，为不同具体实现但提供相同对外接口的不同组件，提供了灵活的语言机制，也提供了实现组件间约定的有效方法。这些语言层面的设计，对实现项目的组件化提供了支持。

Rust 社区也已经提供了各类经过良好封装和迭代的 `crate` 包，并通过统一的 Cargo 包管理工具，以及 `Cargo.lock` 来锁定版本等各类包管理方法进行各种 `crate` 包的管理。目前官方的 `crate.io` 社区非常活跃，已有超过 19 万 `crate` 包。依托于方便的层次化模块系统，和 Cargo 工具，可以便捷地使用成熟的已有组件而无需重复造轮子。这种站在前人肩膀上的开发过程，对于敏捷开发各种项目，尤其是内核这样的大型项目，具有重要意义。

Rust 的特性（feature）则为了 `crate` 包的灵活组合提供了可能。以 ArceOS 的分配器 `allocator` 为例，对外统一提供了特定的内存分配释放接口，但是在其内部，可以通过特性机制，来选择基于 `TLSF`、`slab`、`buddy` 等不同内部实现算法的内存分配器。此外，Rust 的 `feature` 还能在具有依赖关系的组件间进行传递。

1.2.6 国内外研究现状总结

纵览微内核的发展历程，从 Mach 的理念验证到 L4 的性能突破，再到 seL4 将形式化验证与高性能结合，体现了对内核最小化、机制与策略分离等核心原则的坚持，这些原则会在第 1.3 节中详述。然而，既有工作多聚焦于内核自身的正确性、性能或验证，其工程实现范式——尤其是如何构建一个内部解耦、易于复用且能兼容主流生态的微内核系统——尚未得到充分探索。

Rust 语言与组件化开发范式为这一探索提供了新的可能，但如何将其系统地应用于微内核架构，仍面临一系列待解决的关键问题。李龙昊^[61]等人的工作为

我们在该方向的探索奠定了基础，但是 reL4 内核的组件化工作，以及在用户态的系统兼容，才是我们工作的重点，这些问题将在 1.4 小节说明，在 1.5 小节阐述本文的解决思路。

1.3 微内核架构操作系统的设计原则

基于现有的微内核操作系统的实践，本节归纳了一个微内核架构操作系统应有的设计原则。

1.3.1 最小化内核原则

这是微内核的基本原则，也是它之所以被称为“微”内核的原因，即在内核中只保留最必要的内容。

最小化内核原则带来了诸多安全方面的优势，内核中的代码数量少更有利于进行形式化验证等安全性验证，即使不通过形式化验证等安全性验证，也可以减少内核中出现 bug 的概率，有利于保证系统的安全。

最小化内核原则也能一定程度上提升性能。在微内核架构中，IPC 性能是其重要的性能瓶颈之一，而内核中过长的系统调用链会带来 IPC 性能的进一步下降。因此，缩短内核中的调用链长度，对于性能提升具有重要的影响。这也是以 Mach 为代表的微内核所带来的教训之一，在以 L4 为代表的后续微内核设计中，都尽量遵循该原则以提升性能。

1.3.2 策略与机制分离原则

在最小化内核原则的基础上，选择什么内容放在内核中，什么内容放在用户态，成为一个关键性的问题，这就需要遵循策略与机制的分离原则。

以中断为例，中断触发的时候切换到内核态，并保存上下文，这是硬件的机制，但是在中断触发之后做什么操作，则属于中断处理的策略。对于前者，由于不得不读写内核态才能写入的寄存器，因此必须放在内核态执行，而后者则既可以像宏内核一样，放在内核中进行直接处理，也可以选择通过通信的方式，发送消息到用户态进行处理，即微内核的处理方式。

因此，微内核实现最小化内核原则的方式，就是将只有必须放到内核态执行的操作放在内核态；而其他并非必须的操作，尽可能放在用户态。为了让用户态策略能够调用内核态实现的机制，就必须通过系统调用接口向用户态暴露相应的机制调用入口；同时也需要改写操作方式，将在宏内核中实现的策略与机制混合代码，拆分为内核态机制与用户态策略，并设计合适的衔接机制。

1.3.3 实施细粒度的控制

基于微内核架构的最小化内核原则及策略和机制分离的原则，内核中存在的数据结构及其操作，均是最小化的数据结构和基本元操作。这决定了微内核应当遵循实施细粒度控制的原则。

这种细粒度的控制，一方面表现为，对内核中存在的数据结构的细粒度操作。通过典型的如 `capability` 等机制，对每个内核对象进行管理。每个内核对象的操作必须显式地声明并通过权限的验证才可以进一步地执行。

另一方面也表现为，对用户态的每个进程（线程）都会进行各类资源和权限的控制，例如 CPU 时间/内存空间的限制或者是否能够访问某个 `syscall`，访问磁盘设备等资源。

相比于宏内核而言，更细粒度的系统控制是微内核安全性保障的一个重要体现。

1.3.4 系统服务解耦原则

微内核尽可能地将内核服务放置于用户态，其中一个重要的出发点就是希望通过地址空间的隔离，将敏感的内核态操作机制和与内核机制无关的策略相区分。但是一个完整系统的实现，除了内核态的机制外，同样依赖于各类系统服务，它们在微内核下被置于用户态中。如果用户态的所有服务都放在同一个用户态进程中，虽然实现了策略和机制的分离，但是由于缺少组件间的隔离，一个系统服务组件的崩溃会引起崩溃的扩散。因此，在微内核中，也往往需要遵循系统服务解耦原则。

系统服务解耦原则要求每个系统服务（如设备驱动、文件系统）作为独立用户态进程运行，通过明确的接口（如消息传递）与内核或其他服务交互。

系统服务解耦原则使得微内核能够通过地址空间的隔离，让系统服务之间彼此隔离并只能通过进程间通信通过特定的接口进行交互。这一方面使得系统服务组件的崩溃不会扩散，另一方面在某个系统服务崩溃发生之后，一个设计良好的微内核系统不需要重启整个系统，只需要重启该服务即可。

而良好的系统服务解耦，给出了清晰的系统服务组件之间的接口，这也使得系统可以动态地加载或者卸载某个系统服务组件，或者更换同一个系统服务组件接口的不同实现。提高系统的灵活性。

这种组件化的思想在软件工程的设计中也并不少见，例如在宏内核中也有模块化的特性，但是通过地址空间隔离进行组件间的隔离是微内核所特有的。

1.4 关键问题与挑战

本文希望基于李龙昊^[61]和廖东海^{[62][63]}等人的工作,进一步探讨组件化微内核系统这一可能的微内核系统演进方向的具体实践。在推进这一实践的过程中,需要梳理并应对以下关键问题和面临的挑战。

问题一:微内核操作系统的内核实现中的耦合性问题、可扩展问题。在宏内核下,内核组件间的耦合性问题,导致了内核结构臃肿、单一故障容易扩散到整个内核等问题,尽管如此,宏内核的开发者由于功能的庞大臃肿和软件工程的开发范式往往也能认识到这些问题,对代码结构进行优化,并将内核规划为高内聚低耦合的模块以提升宏内核的开发效率。相应的,在微内核中,尤其是经过形式化验证的 seL4 微内核中,内核故障问题不再成为问题,但是由于微内核的设计哲学本身强调了内核代码的精简,不认为微内核的规模到了需要进一步模块化的地步,这导致了随着内核代码量的增加,内核结构臃肿问题,逐步成为开发者需要面对的问题之一。

以李龙昊^[61]、廖东海^{[62][63]}等人实现的 reL4 内核为例,尽管内核代码被拆分到了多个文件中,但是这些文件之间缺乏清晰的界限,并且存在复杂的相互调用关系,在逐步扩展 reL4 微内核功能的过程中,结构不够清晰的问题已开始对开发效率产生负面影响。

问题二:微内核操作系统中的功能可复用问题。在微内核下,存在一些微内核特有的系统设计,例如 seL4 的 capability 功能或者 IPC 部分的设计,也存在一些不同操作系统下通用的功能,例如基本的页表操作管理等。微内核下的特有的系统功能是否可以便捷地被其他操作系统所复用?操作系统所通用的页表操作等功能,是否可以便捷地被一个微内核操作系统所复用?更进一步的,一些宏内核下的成熟的功能模块,是否可以作为微内核操作系统中的用户态服务进行便捷的复用?这构成了微内核操作系统中的功能可复用问题。这种软件的可复用性具有软件工程上的重要性,有利于避免过多的重复开发问题,提升软件开发效率。

然而现有的微内核并不注重这种软件工程上的功能可复用性的要求。这导致了其他的内核项目难以复用微内核的特有设计,以实现一些特定的功能,很大程度上使微内核开发中的可重用工作,演变为重复性的人力浪费。

问题三:微内核操作系统的生态兼容性问题。微内核将传统宏内核的大部分功能都放到用户态中,而微内核的 syscall 和宏内核的 syscall 存在兼容性问题,除此之外,即使通过一些技巧性的方法,解决了 syscall 的问题,一个宏内核操作系统的生态也不仅仅是内核部分,也包括用户态的各类文件和库等,现有的大量库都基于宏内核的 syscall 和编程模型,难以使用微内核编程模型进行重构。最终,从

上到下的软件生态的显著差异，导致了微内核难以复用宏内核的应用生态。

从 reL4 微内核的实现来看，李龙昊^[61]和廖东海^{[62][63]}等人的工作尚未涉及这一问题，其工作主要集中于内核对 seL4 内核层面的部分功能复刻与局部创新。

1.5 研究目标和主要贡献

本文针对上一节提到的组件化微内核系统的实现所提出的几个关键问题，在相应的软件层面开展研究，图 1.1 展示了本文的研究内容所处的软件层次。在详细说明现有的微内核操作系统的基础上，将微内核进行组件化拆分，实现可复用的内核组件库，针对微内核和宏内核生态无法兼容的问题，构建一定的微内核下的用户态宏内核服务，以支持宏内核的运行。

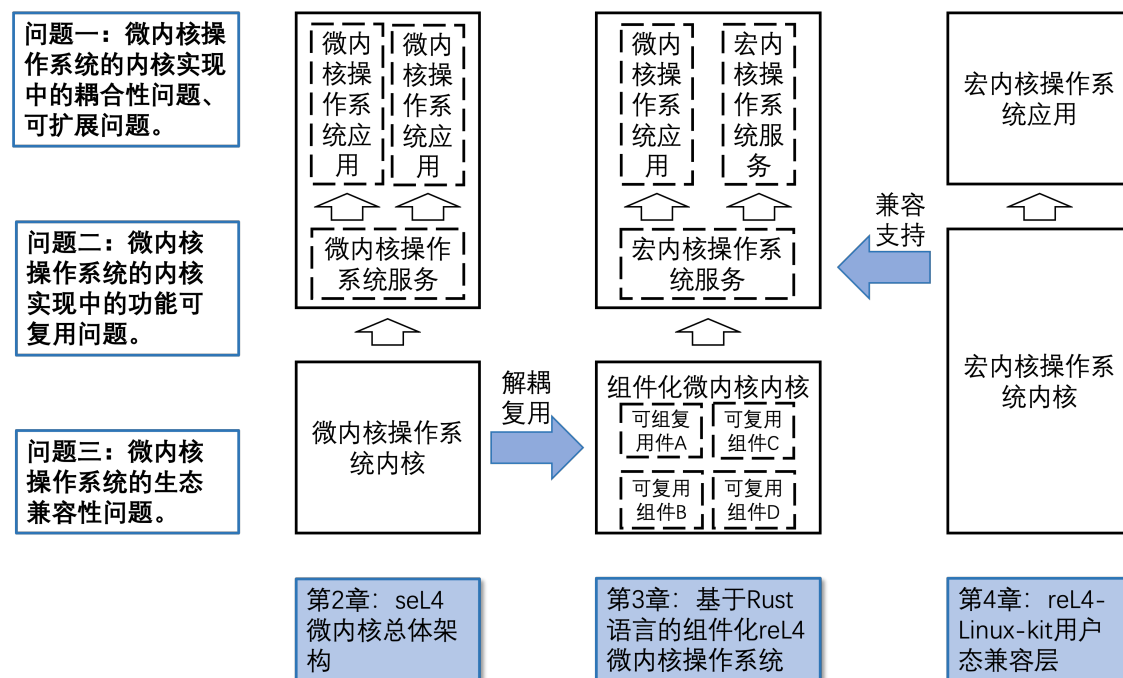


图 1.1 本文主要研究内容所处的软件层面

为实现组件化微内核系统的探索，并应对前文提出的关键挑战，本研究设定以下几个具体研究目标：

- 针对内核耦合性与可扩展性问题，设计并实现一个组件化的 reL4 微内核：该内核需借鉴 seL4 的设计精髓，但运用 Rust 语言的所有权、生命周期及类型安全等核心特性，从头构建一个功能完备、内存安全的微内核 reL4，将调度、内存管理、IPC 等核心功能解耦为独立组件，以提升内核自身的可维护性、可定制性与长期演进能力。该内核需通过 seL4 的官方测试套件，以验证其与 seL4 在核心功能与接口上的正确性与兼容性。
- 针对功能可复用性问题，构建可复用的内核组件库：组件库旨在将内核通用

功能（如页表管理）与微内核特有机机制（如能力模型）等功能封装为可独立复用的模块，降低后续开发成本。

- 针对生态兼容性问题，在 reL4 微内核之上，构建一个提供部分 Linux 系统调用兼容性的用户态服务原型（reL4-Linux-kit）：基于 reL4 微内核提供的线程、地址空间管理与线程间通信（IPC）等原语，开发一系列基本的系统服务组件（如文件系统服务）。在此基础上，设计并实现一个用户态兼容层，该兼容层需为应用程序提供一组稳定的、与 Linux 内核部分系统调用相兼容的接口。
- 功能正确性与性能评估：首先，利用 Linux 下的测试工具对实现的系统调用兼容性进行验证，确保其行为符合预期。其次，对 reL4 微内核及其服务进行详细的性能剖析，与传统 C 语言实现的 seL4 系统进行对比分析，量化 Rust 语言引入的安全保障在性能上的开销与收益，并深入分析其性能差距背后的原因。

针对前文提出的关键挑战，本研究的主要贡献体现在内核实现、用户态系统构建与评估几个层面，具体如下：

- 设计并实现了 reL4，一个与 seL4 二进制接口兼容的内存安全微内核。reL4 是本研究的核心贡献之一，其设计与实现成功验证了以下关键点：
 - 本研究实践并验证了将 Rust 语言用于系统级安全微内核底层开发的可行范式。通过将 seL4 经形式化验证的设计理念与 Rust 编译时保障的内存安全特性相结合，为构建高可靠、高安全性的微内核操作系统提供了一条新的、可能降低形式化验证成本的技术路径。
 - Rust 编程语言能够在不依赖垃圾回收机制的情况下，高效地实现微内核的核心抽象（如 capability 及其管理空间、端点对象、线程控制块等）。
 - 通过合理的 unsafe 代码封装和安全抽象设计，可以在提供与 C 语言相同硬件操作能力的同时，将绝大部分内核代码置于 Rust 编译器的安全检查之下。
 - 实现了与成熟 seL4 的二进制兼容，为复用其丰富的验证与测试工具链奠定了基础。
- 构建了基于 reL4 的用户态 Linux 应用程序兼容性原型 reL4-Linux-kit。本研究超越了单一内核的实现，探索了在内存安全的微内核之上，使用 Rust 构建的系统生态提供主流操作系统 API（Linux Syscall）的可行性。弥补了李龙昊等人的工作中，强调内核功能正确性但是忽略了微内核操作系统作为一个系统所需要提供的用户态支持。

- 提供了详尽的性能对比分析与见解。本研究不仅完成了系统构建，还提供了详细的定量性能评估。通过与传统系统的基准测试对比，本研究工作给出了在微内核基础上，使用 Rust 等高级语言构建宏内核兼容系统的具体数据与详细分析，这些实证结果对未来操作系统研究者与开发者在语言选型与性能优化方面具有参考价值。

1.6 本文组织结构

在第 1 章中，本文首先介绍了整个研究工作的目的、意义、以及相关的研究工作。本文回顾了过去的微内核实践的贡献和不足，以及近年来的微内核方向的研究工作。以此作为引入，介绍了 reL4 内核及其用户态的 reL4-Linux-kit 项目的设计目标。

在第 2 章中，本文介绍了 seL4 内核的具体设计实现，既包括了 seL4 中特有的 capability、MCS、形式化验证等内容，也包括了操作系统通用的内存管理，线程管理，中断管理等内容，是如何在 seL4 微内核中实现的。

在第 3 章中，则阐述了在 seL4 内核的基础上，如何进一步地使用组件化的方案，使用 Rust 语言实现了 reL4 内核的开发。在该章节中，分别介绍了在第 2 中介绍的 seL4 部分，是如何在 reL4 中实现的。除此之外，为了能够在 reL4 中使用 seL4 特有机制，我们也相应实现了 Rust 版本的构建工具等代码，来支持 reL4 的实现。

在第 4 章中，则介绍了我们如何在 reL4 的基础上，实现了 Rust 语言编写的内核服务线程，并兼容部分 Linux 的系统调用接口，实现微内核的安全性与宏内核的通用性兼容性的结合。

在第 5 章中，我们对 reL4 和 reL4-Linux-kit 的实现，进行了功能和性能的评估。

在第 6 章中，我们对整篇论文进行了总结。

第 2 章 seL4：微内核原则的实践

2.1 seL4 微内核总体架构

在本章中，首先介绍 seL4 微内核的总体架构，后续小节将详细分析其特有特性以及通用的内存管理、线程管理、中断管理在 seL4 中的实现。

基于第 1.3 部分所述的微内核的各项原则，seL4 实现了一个微内核必要的各个要素。如图 2.1 所示：

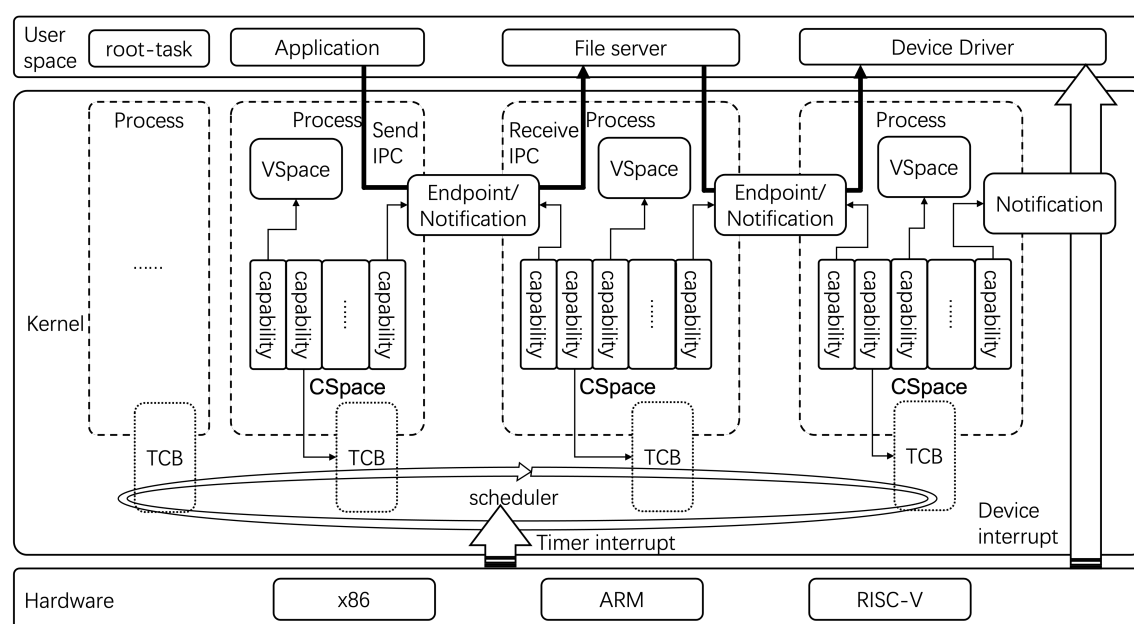


图 2.1 seL4 内核总体架构设计

seL4 内核支持 x86、ARM 和 RISC-V 等多种架构，其生态涵盖了从 QEMU 模拟器到 Avnet MaaXBoard、BeagleBoard (ARM) 以及 Rocketchip、HiFive (RISC-V) 等多种硬件平台，具有丰富的硬件资源支持。

seL4 内核主要基于 **capability** 来管理所有的内核对象，**capability** 是内核权能的一种句柄抽象，既包括了对内核对象的一些操作能力，也包括了一些抽象的，没有具体内核对象的操作相关的能力。所有的 **capability** 经过统一抽象后被存放在一个称为 **Cspace** 的管理空间中，并由其进行细致化的层级管理，关于 **Cspace** 和 **capability** 的具体管理行为在 2.2 节进行详细描述。

这些 **capability** 中，包含了对每个线程所需的线程控制块 **TCB**，虚拟地址空间 **VSpace** 等对象的管理权能，这些线程相关的抽象共同构成了 seL4 内核中的一个线程，内核中的线程调度则依赖于硬件平台提供的时钟中断，使用内置的调度器来进行调度。内核中的消息通知机制，则基于内核中的 **Endpoint** 和 **Notification** 等内

核对象来进行, 既包括硬件平台提供的中断消息通过 Notification 进行传递, 也包括了线程之间通过 Endpoint 和 Notification 进行通信, 而对 Endpoint 和 Notification 也有相应的 capability 来进行管理。总之, 整个内核及其运行都基于 capability 实现了层级化管理, 基于 capability 机制实现的 seL4 内核内存管理, 线程管理和调度, 中断和异常等相关的重点内容将在 2.3 至 2.6 节进行详细的介绍。

除此之外, seL4 微内核还提供了实时系统所需的 MCS (mixed-criticality system) 特性, 以及多核平台下的 SMP 特性, 在 2.7 和 2.8 节分别介绍了相关内容。

整个系统除了上述提到的必要内核机制之外, 均已置于用户态, 并通过各种 syscall 调用提供的机制来实现相应的用户态策略。用户态程序的运行从 root-task 任务开始, 这是用户态的根任务。在系统启动时, root-task 的启动参数会被传递给 seL4 微内核, 并由内核设置好相应的运行环境, 代表整个系统剩余资源管理权限的 capability 集合会被传递给 root-task 并由 root-task 进一步分配。按照微内核的通用设计原则, 整个系统其他的服务线程和应用线程都由 root-task 来启动, 并按照其需求来分配对应的资源。

在传统宏内核架构操作系统中在内核态实现的服务都被放在用户态, 以图 2.1 中所示, 传统宏内核中的文件系统和磁盘管理都在内核态, 而 seL4 微内核或者其他微内核视角下, 都是如 file server 和 device driver 类似的用户态线程。申请文件系统服务的操作被转为一个 seL4 下的 IPC 操作发送给 file server, file server 又可能根据实际需求, 进一步地向磁盘的 device driver 发送消息并由磁盘驱动负责磁盘块的读写, 并将结果返回给 file server, file server 再把结果返回给申请文件系统服务的线程, 构成一个 IPC 链而不是如宏内核那样构造函数调用链。

2.2 seL4 capability 机制和内核对象管理

在一个操作系统内核中, 几乎所有的操作都是在不断地跟各类内核对象进行交互。但是直接使用内核对象的地址访问该内核对象存在安全问题, 因此出现了使用句柄 (handle) 等方式访问内核对象的设计, seL4 也同样采用了这样的设计思路。不同的是, seL4 作为一个微内核, 可以通过设计, 将内核对象数量限定为有限的若干种, 并通过抽象, 将内核对象相关的重要信息封装在一个称为 capability 的, 固定为 128bits 大小的“句柄”中。除此之外, 为了简化设计, 少部分的 capability 所代表的权能, 由于所需的表示信息在 128bits 之内就能全部表示 (也可能无任何表示信息), 因而不需要额外指向对应的内核对象而直接用 capability 本身表示。

seL4 通过 capability 机制对内核对象及各类操作进行统一的抽象, 并通过一个集合了所有的 capability 的 CSpace 对内核中的所有操作权能进行管理。

表 2.1 中列举出了主要的 `capability` 的类型，这些 `capability` 又可以进一步地划分为几类：内核通用的 `capability`，跟内核特性（如 `mcs` 特性）相关的 `capability`，跟硬件架构相关的 `capability`。

表 2.1 seL4 的主要 `capability` 类型及简要说明

capability 分类	capability 类型	类型说明
通用的 capability	<code>null_cap</code>	无类型，用于创建新的 <code>capability</code>
	<code>untyped_cap</code>	用于描述未分配的内存块
	<code>endpoint_cap</code>	用于线程间同步通信
	<code>notification_cap</code>	用于线程间异步通信
	<code>reply_cap</code>	用于消息的回复
	<code>cnode_cap</code>	用于构建 CSpace 的层级结构
	<code>thread_cap</code>	用于管理 TCB
	<code>irq_control_cap</code>	用于控制修改中断的属性
	<code>irq_handler_cap</code>	用于接收中断
	<code>zombie_cap</code>	用于已经废弃但未清理的 <code>capability</code>
MCS 相关的 capability	<code>domain_cap</code>	用于控制不同的 domain
	<code>sched_context_cap</code>	用于描述 <code>mcs</code> 的控制结构 scheduling context
x86 架构相关的 capability	<code>sched_control_cap</code>	用于修改 scheduling context 的属性
	<code>frame_cap</code>	用于 x86 下的物理页面管理
	<code>page_table_cap</code>	用于 x86 下的页表（PT）管理
	<code>page_directory_cap</code>	用于 x86 下的页目录表（PD）管理
	<code>pdpt_cap</code>	用于 x86 下的页目录指针表 （PDPT）管理
	<code>pml4_cap</code>	用于 x86 下的顶级页映射表 （PML4）管理
	<code>asid_control_cap</code>	用于 x86 地址空间标识符（ASID） 的创建与控制
	<code>asid_pool_cap</code>	用于 x86 下将 ASID 与页表结构关联
	<code>io_space_cap</code>	用于 x86 IOMMU 管理
	<code>io_page_table_cap</code>	用于 x86 下为 DMA 设备分配 IOMMU 页表

续表 2.1 seL4 的主要 capability 类型及简要说明

capability 分类	capability 类型	类型说明
	io_port_cap	授予 x86 架构下对特定 IO 端口进行读/写操作的权限
	io_port_control_cap	用于 x86 下创建和管理 io_port_cap 的权限
	vcpu_cap	用于 x86 硬件虚拟化（VT-x）下的虚拟 CPU 管理
	ept_pt_cap	用于 x86 下 EPT（扩展页表）的页表管理
	ept_pd_cap	用于 x86 下 EPT 的页目录管理
	ept_pdpt_cap	用于 x86 下 EPT 的页目录指针表管理
	ept_pml4_cap	用于 x86 下 EPT 的顶级 PML4 表管理
RISC-V 架构相关的 capability	frame_cap	用于 RISC-V 下的物理页面管理
	page_table_cap	用于 RISC-V 下的多级页表管理
	asid_control_cap	用于 RISC-V 下的地址空间标识符的创建
	asid_pool_cap	用于 RISC-V 下的地址空间标识符赋予
ARM 架构相关的 capability	frame_cap	用于 ARM 下的物理页面管理
	page_table_cap	用于 ARM 下的多级页表管理
	vspace_cap	用于 ARM 下的顶级页表管理
	asid_control_cap	用于 ARM 下的地址空间标识符的创建
	asid_pool_cap	用于 ARM 下的地址空间标识符赋予
	smc_cap	用于 ARM 下 SMC call 的权限

对于管理所有 capability 的 CSpace 的结构, 可以类比于页表。在 CSpace 的顶层, 包含了 2 的 radix 次方个 slot 构成的 array 用于存放 capability, 每个 slot 大小 128bits, 和 capability 大小一样。在顶层 CSpace 中的某些 slot, 也可以存放一些 cnode, 即用于表示二级或者更深级数的 CSpace 的 capability。

CSpace 通过一个 cptr 来指定某个 slot, 并找到其中存放的 capability。这个 cptr

类似于页表中的虚拟地址, 用于描述 slot 的地址。但是正如前面所述, 这不是一个真实的内核对象的地址。这个 cptr 会通过 guard bits 和 radix bits 进行层级切分。其中 guard bits 用于判断是否跟当前的 CSpace 相符, radix bits 正如前文所述, 用于指明该层的 slot array 中包含多少个 slots。

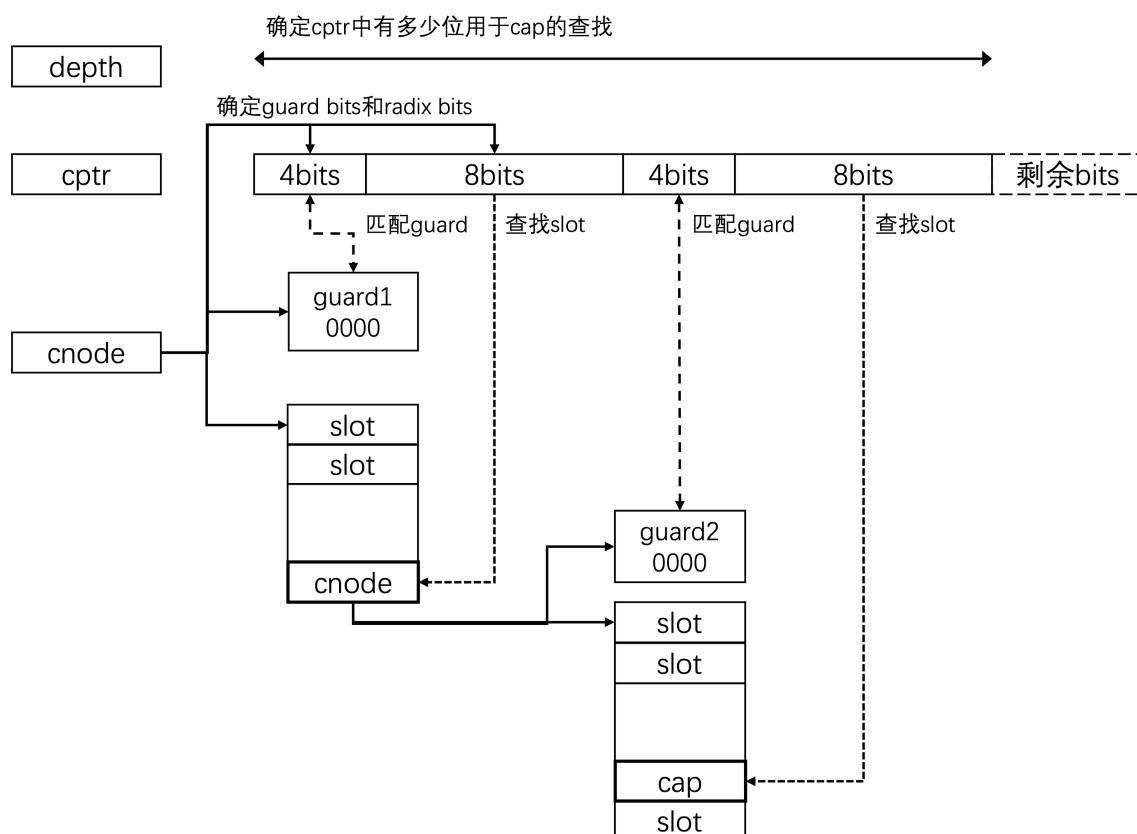


图 2.2 CSpace 查找样例

以图 2.2 中所绘制的 CSpace 为例, 查找对应的 capability 的过程如下:

- 1、图中的 cptr, 第一层的 guard 占用了 4 个 bits, radix 占用了 8 个 bits。那么最高的 12 个 bits 会被用于访问最顶层的 slot array, 该 array 具有 $2^8=256$ 个 slots。
- 2、如果这个 cptr 最高的 4 位为 0, 说明通过了 guard 的检查, 否则报错。
- 3、如果剩余的低 20 位都是 0, 那么在最高 4 位后面跟着的 8 位所指出的那个 slot 处存放着查找到的目标 slot。
- 4、如果该 slot 中存放的不是一个 cnode 的 capability, 或者达到了设定的查找深度, 那么就认为找到了。
- 5、如果并非如此, 即可以向下查找, 那么就查看第二层的 guard bits 和 radix。如果同样是 4 个 bits 的 guard 和 8 个 bits 的 radix, 会继续匹配剩余 20 位中最高 4 位表示的 guard 是否都为 0, 再之后的 8 位用于查找 slot, 最后的 $20-12=8$ 位如果为 0, 且该 slot 不为 cnode, 或者同样到达了查找深度, 就认为找到了, 如果不是就继

续。以此类推。

除了通过 CSpace 来进行 capability 的管理之外,seL4 也维护了 capability 之间的关系,在 capability 的创建,销毁之外,还有 Mint(从某个 capability 继承全部或者部分权限),Copy,Move,Revoke(不仅删除自己,还删除所有派生出去的 capability) 等更为复杂的 capability 操作,这些派生操作额外构成了 capability 之间的派生关系。

除了 root-task 所具备的根 capability 之外,其他 capability 都是由某个 capability 派生出来的,这些 capability 所具备的各类权限只是其父节点的一个真子集。通过反复的派生,最终构成一棵 capability 派生树,整个系统的所有权限都在这棵能力派生树上。如图 2.1 中所示的不同线程的 CSpace 中的所有 capability,其实质上都是由 root-task 的 capability 所派生出来的。

这样一个树状的关系则保证了 seL4 系统的安全性。一个 capability 所具有的权限,只来自于其派生的父 capability,而当内核通过 Revoke 操作撤销某个父 capability 时,还会删除所有其派生出来的 capability,从而保证了没有无来源的 capability 存在,最后,树状的 capability 派生模式,可以保证无环且父节点唯一,为上述的派生和撤销提供了简便的管理模式。

在实际的操作系统实践中,部分的 capability 被所有的线程所需要,比如管理地址空间的 capability,管理线程 TCB 的 capability,因此,seL4 固定了部分必须的 capability 存放的 slot,并约定好相应的操作直接去对应的 slot 处查找。

capability 系统让 seL4 对所有内核对象有了灵活的细粒度的控制,是对微内核细粒度控制原则的充分体现。seL4 的所有内核对象都体现在 CSpace 里的不同 capability 中,并通过对应的 capability 接口,对内核对象具有的权限进行了创建、销毁、派生等操作。

2.3 seL4 的内存管理

2.3.1 虚拟内存的管理

在支持虚拟内存的硬件架构上,操作系统可以开启分页或者分段或者段页式的虚拟内存机制,从而使用超过物理内存限制的地址空间,并通过地址空间来隔离不同进程,提升安全性,微内核的安全性也来自于这种基于地址空间对进程的隔离。

seL4 所基于的硬件架构,均支持多级的分页内存机制,为此,seL4 提供了一套称为 VSpace 的机制用于虚拟内存的管理,这套称为 VSpace 的机制包含了虚拟地址空间管理中的多种不同的 capabilities。

在名称上,seL4 系统中的物理内存页被称为 frame,而虚拟内存所需要的页表

项和页表, 包括 page Directory, page table 等, 都被描述为不同类型的 capability。当然, 由于 seL4 支持多个架构, 因此其页表项 pte 的相应属性都有一些架构相关的内容, 构成 VSpace 的 capability 也有其特点, 参考表 2.1 中的描述, 在 RISC-V 架构下均用 page_table_cap 这一个 capability 来表示多级页表的每一层, 但是在 ARM 架构下, 低层次页表仍然使用 page_table_cap 这一个 capability, 但是最顶层的页表则使用 vspace_cap 这一个 capability 来表示, 而 x86 下则具有 PT/PD/PDPT/PML4 四级页表管理机制。因此, 虽然在 seL4 的抽象下, VSpace 都包含多级页表的 capability, 但是在不同架构下, 也各有细微的差别。

除了页表之外, seL4 还提供了多地址空间的支持, 在 ARM 和 RISC-V 架构下的多地址空间支持基于地址空间标识符 asid, 在 x86 下则被称为 PCID, 该机制主要是为了避免上下文切换时刷新整个 tlb 带来较大的性能开销。在 seL4 中对这部分的管理包括了 asid_pool_cap 和 asid_control_cap 两个 capability, 之所以使用两个 capability 而不是把所有 asid 相关的权限放在一个 capability 中, 是由于 asid_control_cap 被设计为全局唯一的用于 root-task 创建 asid pool 的权限, 而 asid_pool_cap 则是用于具体管理每个 asid pool。

另外, 对于 page fault 的处理, 也是传统内核中必要的一部分内存管理内容。在 seL4 中, 则会传递给线程设定的 fault handler 的 Endpoint 进行处理, 这部分在 2.4 节进行描述。

在 seL4 中对虚拟内存的接口设计, 充分体现了策略与机制分离的原则。在内核中只提供必要的设置页表和页表项等 VSpace 的接口, 而更复杂的操作全都交给了用户态。

2.3.2 内核所需要的内存分配

对于 seL4 而言, 在系统启动之后, 除了预先分配给内核的内存空间, 剩余物理内存空间作为一种系统资源, 都应当被表示为 capability 的集合交给 root-task。微内核在内核中只保留机制, 而让策略放在用户态的设计哲学, 决定了需要用户态的 root-task 来确定内存分配的策略。而内核在运行过程中需要不断地创建新的内核对象, 因此需要提供相应的机制以供用户态的策略可以申请内核对象。这也是策略与机制分离原则的另一个体现。

在 seL4 中, 未被分配出去的内存空间, 也被当成一种特殊的 capability, 也就是 untyped capability。untyped capability 用于表示一段未被分配出去的内存空间, 并可以通过该 capability 唯一的系统调用来将其 retype 为其他的内核对象。

每个 untyped 的内存存在申请的时候并非一次性全部分配给申请的线程, 而是按照申请的内存大小向上对齐到 2 的整数次幂进行分配。此时, 剩余的未分配的

untyped 的内存仍然会被记录，以便下一次进行再分配。

2.4 seL4 线程间通信（IPC）机制

2.4.1 快速 IPC

在以 Mach 为代表的传统的微内核实现中，IPC 的性能是微内核性能的瓶颈。在宏内核下两个不同内核模块之间的 function call 交互，在微内核下会转变为两个用户态进程（线程）之间的通信，这需要进入内核，（可能）切换地址空间并传递消息，再切换回来，其中存在较大的性能开销。因此，以 L4 微内核为代表的一系列微内核都通过寄存器传递参数等方式，用于实现快速 IPC。

具体到 seL4 中，seL4 中的 IPC（Inter-Process Communication）实际应当为 ITC（Inter-Thread Communication），这是因为，在 seL4 的线程管理系统中，缺少这种进程（Process）的概念，而只有线程（Thread）。但是说起 seL4 的线程间通信，之所以仍然沿用 IPC 这个词，则是由于整个操作系统历史上，IPC 都是用来指代这种跨执行流的通信，如果新创造一个 ITC 的词汇，会带来传播和理解上的困难，因此 seL4 官方选择仍然沿用旧的缩写，但是赋予其符合实际的新的含义。

在 seL4 中，使用消息寄存器进行参数的传递。在不同架构下寄存器不同，因此，seL4 也会给定不同的消息寄存器的数量，同时，其他的寄存器也会用作 capability 传递等用途。

在消息传递的过程中，也有可能出现消息数量大于该架构下可用的消息寄存器数量的情况。在这种情况下，才会采用内存拷贝的方式进行消息的传递。

在 seL4 中实现了 fastpath 机制用于快速的消息传递，在 fastpath 中会经过一定的判定，如果满足所有的判定条件，就可以快速执行消息传递，否则进入 slowpath 的处理，而 slowpath 实质上就是传统操作系统的 syscall 的实现。

在 seL4 的 syscall 中，也同样针对 IPC 做了专门设计：向内核发起真正系统调用的实现，统一被放在一个 SYS_CALL 的系统调用号中，并进行进一步的分发和处理；而其他系统调用号（参考表 2.2）则用于实现各类 IPC（尽管在这些通信过程中也在某些时刻需要向内核申请资源）。这样的设计减少了传统内核中 IPC 机制需要经历的 syscall 分发等步骤。在本文的后续章节中，若有提及不在表 2.2 中的内容，均指代向内核发起真正的传统意义上的系统调用。

表 2.2 seL4 的以消息传递为主的 syscall 设计

syscall 分类	syscall 名称	syscall 说明
通用的 syscall	SYS_CALL	系统调用分发入口
	SYS_REPLY_RECV	这是一个同步的组合操作，先进行接收再进行回复
	SYS_SEND	这是一个同步的发送操作，向目标端点发送消息，如果接收方未准备好就阻塞
	SYS_NB_SEND	尝试发送，发送语义和 SYS_SEND 相同，若接收方未就绪会立即返回错误而不阻塞
	SYS_RECV	等待并接收消息，无消息时阻塞
	SYS_REPLY	向之前接收的消息发送方回复结果
	SYS_YIELD	将当前线程放入调度队列尾部，允许同优先级线程运行
	SYS_NB_RECV	尝试接收消息，接收语义与 SYS_RECV 相同，若接收方未就绪立即返回错误而不阻塞
MCS 特性相关的 syscall	SYS_NB_SEND_RECV	先执行非阻塞发送，然后阻塞接收消息
	SYS_NB_SEND_WAIT	先执行非阻塞发送，然后等待通知（Notification）或超时事件
	SYS_WAIT	等待通知信号或超时事件发生
	SYS_NB_WAIT	检查通知信号状态，无事件时立即返回

2.4.2 Endpoint 和 Notification 消息传递

在 seL4 中，IPC 机制并不直接将消息传递给对应的目标线程，而是同样通过 capability 机制进行传递，如图 2.1 中所示，使用了 Endpoint 和 Notification 两个专门为了 IPC 设计的 capability 来进行通信，这是为了避免 DDoS 的攻击而采用的一种安全策略。在 seL4 下的微内核的用户态模型类似于 client/server 的消息通信模型，这导致了，系统服务的线程可能遭受 DDoS 的类似攻击，采用基于 capability 的 Endpoint 机制，若使用阻塞式的发送接口发送消息，在接收方未就绪时可以阻塞发送方，而采用非阻塞式的接口在接收方未就绪时会直接丢弃消息，从而避免 DDoS 类型的发送攻击。

seL4 提供了两种 `capability`，一个是用于同步消息传递的 `Endpoint`，另一个是用于异步消息传递的 `Notification`。对于同步的 `Endpoint capability`，可以传递较为复杂的信息，而 `Notification` 则只能传递简短信息。`Endpoint` 和 `Notification` 的阻塞和非阻塞还会影响内核的调度，这部分在 2.5 节中详述。

另外，由于 `capability` 的派生机制，同一个 `Endpoint capability` 派生或拷贝出的新 `Endpoint` 需要加以区分，seL4 为此提供了 `badge` 机制。`badge` 是一个标识，用于 `server` 区分不同 `client` 的消息，也可作为可信验证方式，确认消息来源是否为可靠的消息发送方。`Notification` 传递的信号被称为 `ntfnMsgIdentifier`，既可以当成信息内容本身传递，也可以当作一种 `badge`，用于标识不同的 `Notification`。

2.5 seL4 调度与执行模型

seL4 中的每个用户态任务对应一个 TCB 内核对象，并由相应的 TCB `capability` 管理。为了进行任务调度，seL4 还维护了一个调度队列的数据结构，用于管理这些 TCB。

seL4 内核中，使用了一个单内核栈的设计。

在 Linux 等传统宏内核中，每个用户态线程除了各自用户空间的用户栈之外，在内核空间也配有一个对应的独立内核栈，当在某个线程的用户态代码运行时，发生了陷入或者中断等事件，就进行栈空间的切换，并将内核栈每次都固定的指向栈底。内核会保证在返回到该用户态之前，内核栈空间中的内容都被清空或者无效，从而进入内核时指针指向栈底是安全的。另外，在两个线程之间切换的时候，实质上是两个内核栈指明的运行过程之间的切换。

seL4 采用单内核栈设计：当陷入或中断发生时，同样切换栈空间，内核栈指针始终指向固定位置。为此，需要在每个进入并离开内核的路径上，都在完成内核处理过程之后，在最上层的函数中，调用 `schedule` 选择新线程并调用 `activateThread` 激活该线程并最终使用 `restore_user_context` 函数返回到对应的用户态。

这种单内核的架构，也意味着不允许在内核态存在栈上的阻塞方法。具体来说，在宏内核中，如果某个线程 A 发起系统调用，例如申请磁盘读写操作，那么在执行到某个步骤的时候需要等待磁盘操作完成，这时候可以手动调用调度器切换到其他线程 B，此时，线程 A 的内核栈中存放着它在内核中的执行步骤。

但是在单内核栈中，若允许上述的步骤，线程 A 的运行栈帧之下一一定存放了线程 B 的栈帧，此时回到线程 B 的用户态并再次进入内核态，就会破坏线程 A 的栈帧内容。

因此，在这样的设计下，多次内核执行过程，需串行交替执行，不允许其被打

断产生阻塞并在栈上保存内容。如有相关的保存内容的需要，需要显式的存放在堆上。

这也决定了，任何一次进入内核中的执行路径都是确定的，其一定存在某一次调用路径的栈空间最大，因此 seL4 只需要一个确定的足够大的栈空间即可，而无需产生任何动态申请内核栈空间的操作。这一点和上述的内核固定内存分配有着密切的联系。

对于 seL4 的调度模型，在每个进入内核的路径中，在必要时将当前线程放置到调度队列中，离开内核时调用 `schedule` 函数选择下一个运行线程。除此之外，为了支持多个优先级，seL4 为每个优先级都设置了一个相应的调度队列，在调度时，先通过一个 `bitmap`，快速的找到存在可调度任务的最高优先级，随后在该优先级所代表的调度队列中，找到对应的 TCB，这种通过 `bitmap` 和对应的队列进行调度的方法和 Linux 的调度方法有相似之处。

如 2.4 节中所述，Endpoint 和 Notification 两种不同的通信方式，对调度也存在一定的影响，以 Endpoint 类型的消息为例，由于可能存在的消息导致的线程阻塞，会影响调度器的行为。

在 Endpoint 的数据结构中，包含了消息的发送和接收队列。在消息发送时，若检查到目标的 Endpoint 接收队列为空，则进行任务切换，并将消息发送线程的状态标记为 `blocked on receive`，同样的，在试图接收时，如果发现目标 Endpoint 的发送队列为空，则同样立即进行任务切换，并标记其状态为 `blocked on send`。线程处于这些状态下时，调度器会不调度相应的线程。

2.6 seL4 的中断和异常处理

中断和异常的硬件机制决定了中断和异常必须进入到内核态进行处理，但 seL4 作为一个微内核，也需要遵循微内核的最小化内核原则和策略与机制分离的原则，中断的机制不得不放在内核中，但是中断的处理策略则需要放在用户态。seL4 的中断处理流程如图 2.3 所示，需要说明的是，在该图中，虽然用户态持有了 Notification 的 `capability`，但是如第 2.2 节中所述，有部分对象由于描述它所占用的比特数少于 128，可以直接在一个 `capability` 的大小中放下，而 Notification 恰巧就是这样一个结构。因此，在操作上，直接使用该 `capability` 进行操作即可，无需再找对应的内核对象。

seL4 将中断和异常的处理和 IPC 的机制结合起来，除了一些内核本身需要用到的中断之外（比如需要时钟中断用于任务调度），额外地使用了 Notification 机制，用户态可以将某个 Notification 的 `capability` 作为参数注册给某个中断，将该中

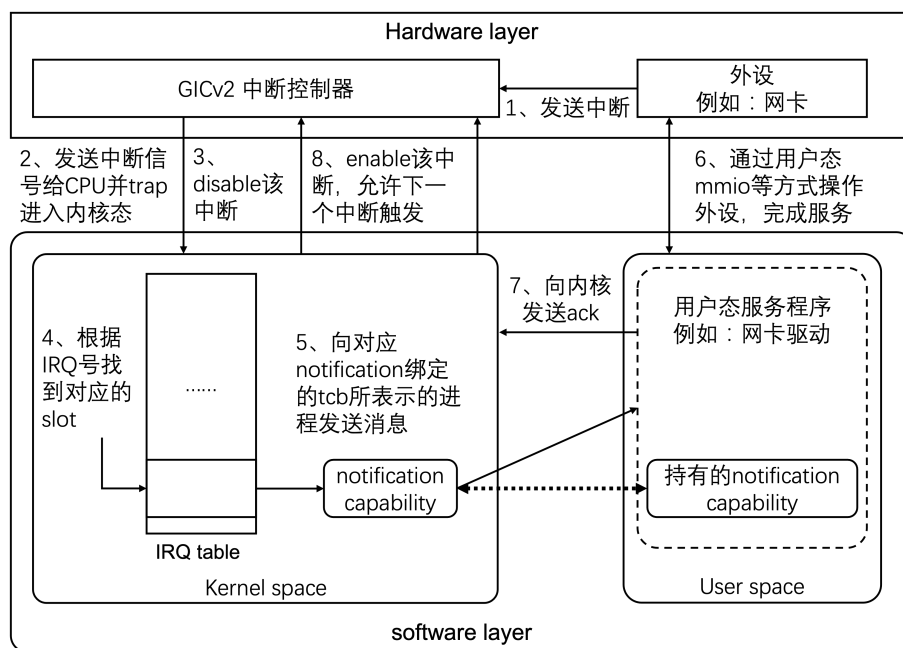


图 2.3 seL4 中断处理流程图

断的消息传递给该 Notification，该 Notification 也可以绑定到某个 TCB 上，从而指定由该 TCB 所表示的线程来决定如何处理该中断。

为了维护这些 Notification capability 和中断的绑定关系，seL4 额外地为所有中断分配了一段存放对应 Notification 的 capability 的空间，和一段存放 irq 的状态的空间。

当然如果某个中断并没有被注册相应的 Notification 用于传递该中断消息，seL4 也会进行相应的错误处理。

同样的，中断作为内核的机制，也应当存在对应的 capability 进行细粒度的权限管理，这就是在表 2.1 中的 `irq_handler_cap` 这一个 capability。如果这个 capability 不能派生并分发给某个 TCB 表示的线程，那么就无法将 Notification 绑定到该中断上，以此来保证中断处理的安全性。

另外，在中断的处理机制上，遵循策略与机制分离等原则，seL4 的实现也有一些细节需要注意。

以 aarch64 架构为例，在 qemu-arm-virt 平台上实现的 seL4 使用了 gicv2 中断控制器。该中断控制器中设计的中断状态包括 `inactive`（未触发），`pending`（触发但是未被 CPU 处理），`active`（触发并被 CPU 处理），`pending and active` 四种。第四种状态的含义为，每个中断的标识位只有 1 位，但是在同一个中断上，最多可以允许同时有两次中断信号被使用，其中一个中断信号正在被 CPU 处理（即 `active`），而此时另一个中断又正好已经到达却尚未被 CPU 处理（即 `pending`）。

最后一种状态相对复杂，seL4 将其进行了简化。每次接到中断之后都会直接

清除该中断的允许位，让他必须在用户态调用了 `seL4_IRQHandler_Ack` 函数向内核发起系统调用之后，才能够重新打开该中断的允许位，这又是一个策略和机制分离的重要体现，将对中断的确认放置到了用户态来进行。这也直接导致了，第四种状态未被使用。

这给用户态系统服务的编写带来了与宏内核不同的特点。仍然以中断需要在用户态进行确认这一点为例，如果用户态需要实现一个高性能网卡的驱动，每次收到一个包就进入内核，这会带来巨大的性能开销。在持续有网络流量到达的情况下，可以将网卡寄存器 `mmio` 映射到用户态，采用类似于 `napi` 的机制，在收到一个网络数据包的到达中断之后，系统网络服务线程持续轮询网卡寄存器中指定的数据包，并在最后调用确认中断的系统调用，再开启网卡的中断。在这种设计下，只有第一个包的中断会触发并进入内核态，从而提升系统服务的性能，这种用户态接口设计，兼顾了高性能的用户态网络驱动服务的需要和微内核的策略与机制分离原则。

2.7 seL4 的混合关键系统（MCS）支持

混合关键系统是实时系统中的一个重要概念。这个概念的最初提出就是为了解决如下问题：在一个实时系统中包含不同的任务，他们存在不同的实时性约束要求和到达周期，在此基础上，更进一步抽象出了关键性级别的要求，即不同任务具有不同的关键性级别，高关键性级别任务错过任务截止时间可能造成严重后果，而低关键性级别可以在一定条件下允许错过截止时间，如何调度这些具有不同关键性级别的任务就成为一个问题。

seL4 本身被设计为一个安全的内核，其中也包括了时域上的安全性。例如，如果某个低关键性级别的任务错误地占用了过多的时间，就会导致高关键性级别的任务在某些情况下错过截止时间，从而带来灾难性的后果。为此，seL4 引入了 MCS 这一系统特性的支持^[53]，其核心创新在于将时间作为一类可通过能力模型管理的内核资源。

2.7.1 seL4 内核中的混合关键系统算法

如第 1.2.2 小节所述，混合关键系统研究经历了从单核心到多核心、涵盖多维度的复杂体系演变过程。现有混合关键系统调度算法的实现往往需要对内核调度器进行侵入式改造，且复杂度较高。

为在 seL4 内核中增加混合关键系统支持，需要考虑微内核的策略与机制分离原则的实现问题。具体而言，seL4 作为微内核，应当只提供实现机制，而不提供

具体策略。同时，为避免不断演进的混合关键系统算法对调度器实施侵入式改造，seL4 在内核中内置的混合关键系统支持采用固定机制加可配置策略的组合方式：在内核态提供固定的基础计时机制，在用户态实现可配置的具体调度算法策略。

基于这样的设计目的和考量，seL4 实现了一个简单的 Sporadic Server 算法，用于提供内核态的基本计时策略，并相应地支持各种用户态的混合关键系统调度算法。

Sporadic Server 算法基于 server 的思想。最早的基于 server 的算法是 Polling Server。针对系统中的非周期性任务，Polling Server 算法提供了一个称为 server 的周期为 T 的周期性任务，server 任务具有预算 C ，每当 server 任务的周期到达，就补充满它的预算，检查是否存在已经到达的非周期性请求，如果有，就使用 server 任务运行非周期性请求，直到消耗完 server 在该周期内的所有预算 C ；如果没有非周期性请求，就在该周期内放弃运行该 server 任务。

Polling Server 算法具有明显的缺点，就是对于非周期性请求的处理存在较高的延迟，因为它必须等待 server 的周期到达，才能真正地处理非周期性请求。这引入了额外的延迟，而实际上，上一个 Polling server 所拥有的预算 C 可能根本没有用完就被直接丢弃，它完全可以保留，从而立即满足非 server 周期内到达的非周期性请求需求。这个思路下产生了 Deferrable Server (DS) 算法。

DS 算法的思路就是只要 server 有预算就可以立即执行非周期性请求，而 server 的周期只有一个作用就是用于周期性地补满请求预算 C 。但是这种立即执行也使得其他周期性实时任务的可调度性分析变得复杂。

为了解决引入的复杂可调度性分析问题，Sporadic Server (SS) 算法被提出。该算法在 DS 算法的基础上进一步修改了请求预算补充的方案，针对每次预算消耗都进行了记录。与定期补满预算 C 不同，尽管该算法仍然有周期 T ，但该周期 T 的含义变为：当次预算消耗会在 T 时刻之后补充回去，且所有的预算补充不能超过上限 C 。为了防止复杂的可调度性分析，SS 算法规定，当一个 server 预算被消耗完之后，在下一次补充到达之前不可被调度。这个设计思路让 SS 算法的可调度性分析等价于一个严格的周期性任务。

当然，原生的 SS 算法存在一些理想的情况，而 seL4 内核所基于的算法^[55]对一些现实情况进行了修正。比如由于计时器差异导致的预算被消耗完的时候，实际上因为计时器差异，剩余预算 C 并不总是正好等于零，如果当前剩余的预算 C 变为了负数，在这种情况下，必须减少之后的预算补充，以避免由于计时器带来的偏移。

而在内核中引入该基于 server 思想的算法，其作用更多的是作为一个严格的

计时控制方案,在内核中给用户态提供了一个通用的用户态可管理的定时机制。真正每个用户态线程可以使用多少时间预算,是由获得时间预算 `capability` 的线程通过相应的 `capability` 机制控制的,微内核提供了一种保证该控制有效的机制,并让用户态决定策略。

2.7.2 seL4 中的混合关键系统支持实现

区别于传统优先级调度,seL4 将关键性和优先级进行了区分,使用 `Sporadic Server` 算法实现了名为 `scheduling context` (以下简称 `sc`) 的一个新的内核对象,其对应的权能称为 `scheduling context capability` (以下简称 `scCap`),用该对象来表示线程的关键性级别。在关键性和优先级进行区分的基础上,也更改了一个线程是否为 `runnable` 的定义,除了原先基于优先级情况下的判定之外,还需要结合 `scCap` 的参数,判定某个线程在满足不同关键性的需求条件下,是否还有足够的时间运行,进而保证时域上的安全性。

除此之外,seL4 微内核也需要灵活决定是否启用 `MCS` 机制,以兼容原生的 seL4 实现。因此,在用户态的线程可以通过是否绑定对应的 `scCap` 来决定是否使用 `MCS` 机制,如果不是,就按照 seL4 原有的固定优先级调度机制进行调度。

seL4 还提供了超时异常 (`timeout fault`) 的通知机制,在某个用户态线程超时,可以选择发送对应的消息给用户态的程序,让用户态来决定当发生超时时采用的处理策略,如提供更多的时间片预算或者终止线程等等。这一设计是由于 seL4 的 `MCS` 特性将时间也当成了一个被管理的资源,而超时作为不被允许的资源错误使用,也需要沿用 seL4 统一的 `fault handler` 机制来处理。

另外,seL4 下,使用 `MCS` 特性之后,Endpoint 等通信机制在线程调度相关部分的设计思路也被明显改变。在抽象概念上,Endpoint 的消息传递转变为执行权限和时间预算资源的转移,而并非简单的消息传递,以更贴近图 2.1 中的 C/S 架构。

考虑这样一个情形,在系统中存在一个 `server` 线程用于处理 `client` 线程发送的若干请求,那么 `server` 可以视作代替 `client` 来执行一些操作,`server` 可以运行的时间以及这些时间所具有的关键性级别实际上取决于 `client`。如果 `client` 本身具有低关键性级别,则 `server` 在处理该 `client` 发起的请求的时候,所具有的调度权限和时间资源不应该超出 `client` 的低关键性级别和 `client` 所具有的时间资源,反之,就应该让 `server` 以高关键性级别运行。因此,相关的通信需要添加这种思路下,时间资源和关键性级别的传递赋予机制。

在 seL4 的设计中,实现了一个 `reply` 内核对象,用于表达这种时间资源的传递。这个 `reply` 内核对象区别于无 `MCS` 特性下的 `SYS_REPLY`,虽然同样命名为 `reply`,但两者完全不同且没有明显的关联,`SYS_REPLY` 只是无时间资源传递的一

种消息机制。在消息发送的时候, 阻塞语义会被表达为自身的 CPU 时间资源被转移到对应的 `reply` 对象中并进行发送, 由于自身没有剩余的时间资源, 从而导致了阻塞, 而接收方获得一定的时间资源, 并可以继续运行以接收消息。调用消息接收接口时, 也是类似的时间资源转移语义。另外, 在 `server` 处理完 `client` 的请求之后, `sc` 中如果还有剩余的请求时间, 则需要通过 `reply` 等系统调用返还给 `client` 线程。

在 `reply` 内核对象的具体实现上, `seL4` 使用了一个 `call stack` 的栈逻辑。之所以使用该逻辑, 这源于更复杂的消息调用场景。在上述的例子中, 假设存在多个 `server` 线程, 他们之间构成一个调用链 (如图 2.1 中 `application` 调用 `file server`, `file server` 调用磁盘 `device driver` 所示), 该调用链的形式本身就是一个栈的形式, 如果使用 FIFO 队列或其他形式, 当最后调用的 `server` 线程试图返还剩余的请求时间时, 会因为队头并非上一层调用的 `server` 线程而无法返还, 而使用栈的实现形式则能完美契合这种调用关系链。

`seL4` 的 MCS 特性对整个内核的实时性管理以及内核通信设计都产生了重要影响。其针对 C/S 架构下服务模型的消息发送语义改造, 体现了微内核系统服务解耦原则的深化; 对时间资源的 `capability` 化则扩展了微内核的安全模型, 是 `seL4` 相比于传统 L4 内核的关键创新。

2.8 seL4 的 SMP 支持

在多核上的支持上, `seL4` 主要解决了以下的问题:

- 1、多核心独享资源的管理。多个核心具有重复的专属于某个核心的资源, 比如中断, 这些资源每个核心都需要存在一份, 同时, 需要明确自己是哪个核心, 并能够按照约定访问独属于自己的资源, 而不能访问属于其他核心的资源。对此, `seL4` 的实现相当简单, 直接把相应的数组扩展了一个维度, 同时设计了一些宏, 用于多个核心的资源访问。

- 2、多核心共享资源的管理。多个核心可能同时访问一个共享资源, 需解决并发访问问题。`seL4` 的实现同样较为简单, 使用了一个大内核锁, 在所有的内核入口处, 试图抢到这把锁, 并在离开内核的时候, 释放这把锁, 从而保证同一时刻只有一个核心在内核中。

- 3、多核之间通信, 以及线程在核间迁移等动态多核心交互的问题。`seL4` 实现了基于不同硬件的 IPI 中断机制的核间通信机制。

在 Linux 内核的早期开发阶段, 曾经也使用过大内核锁, 但是最终出于多核情况下的性能考量, 放弃了这一思路, `seL4` 同样面临此问题。总体来说, 大内核锁简化了内核设计的复杂性, 利于验证, 是体现最小化内核原则的典型设计, 但这可

能成为多核性能的瓶颈。

对于 seL4 内核而言, 如果不考虑性能这一点, 大内核锁让 seL4 更为易于被验证, 这是一个明显优势, 至于 seL4 内核中, 大内核锁对多核性能的影响, seL4 团队曾经做过实验^[65], 其结论为, 在当时 (2015 年), 在一个中等核心数量 (小于等于 8 个核心) 的平台上, 一个大内核锁带来的性能损失是可以接受的。

但随着集成电路工业的持续快速发展, 在 2015 年的合理推断已经不再符合现状。在一些 seL4 所瞄准的嵌入式方向的平台上, 集成大于等于 8 个核心的 CPU 平台在十年后 (2025 年) 已经是常见的情况, 而 seL4 在这些平台上, 由于大内核锁的存在, 可能会导致核间冲突明显增加。其次, 在该论文^[65]出现时, seL4 认为大内核锁对性能的影响不大, 一定程度上也因为大量的细粒度锁本身也有一定的开销, 这一点在 ARM 等弱内存模型的平台上更为突出, 但是, ARM 也在快速发展, 其内存屏障等指令的性能开销可能需要被重新审视。第三, 在该论文^[65]发表之后 seL4 也在不断演进, 更多的特性在 seL4 中被支持, 例如 MCS 特性相关的论文^[53]在 2018 年才发表。既然 seL4 的代码量在不断增加, 那每个核心进入内核并独占它的时间开销也需要重新被衡量。

2.9 seL4 安全验证与形式化证明

seL4 内核的核心特点在于其使用形式化验证的方式, 验证了内核的安全性。其形式化验证方案采用一种分层式的方法, 从系统的模型设计出发, 向下到 C 代码的实现, 编译器从 C 代码到二进制代码的转换, 以及在特定硬件模型下, 如 ARM 等弱内存模型的平台上的硬件行为等, 多层次地验证了该内核的安全性。

seL4 本身也有其验证边界, 并非绝对安全。一方面, 硬件本身可能存在各类安全漏洞, 如 Spectre 漏洞本身是依赖于硬件的分支预测和缓存策略中的漏洞实施侧信道攻击, 这是 seL4 无法避免的。另一方面, 由于其完成了严格证明的微内核本身只有一个简单的内核, 而一个真正可用的微内核操作系统还包括各类放在用户态的系统服务, 针对这部分的攻击, 仍然可能存在安全隐患。但是至少, seL4 的证明创造了一个极小且被验证过的可信计算基 (TCB), 从而使攻击面主要转移到了未被验证的用户态组件上。

与 Rust 编写的内核不同, seL4 本身的安全性, 并不来源于编译器的静态检查, 以指针解引用为例, 它通过形式化证明的方式, 证明内核中并不存在空指针解引用行为。而 Rust 编写的内核, 在 safe 的边界中, 本身并不允许裸指针的强制类型转换和解引用, 这都被认为是 unsafe 的部分, 不过, 作为一个操作系统内核, 使用一定数量的 unsafe 的操作是难以避免的。因此, 使用 Rust 语言对 safe 的部分进行

保证而使用形式化验证等方式进一步保证 `unsafe` 内容的内核, 具有重要研究价值。

2.10 本章小结

本章对 seL4 的实现及其对微内核原则的遵循进行了概述。详细地阐述了 seL4 微内核对微内核的通常性原则的遵循, 同时也通过一些细节性的例子, 阐述 seL4 的独特性设计。

seL4 使用了特殊的 `capability` 机制对内核对象及其权限进行了细致性地管理, 通过仿照页表设计的 `CSPACE` 对 `capability` 进行管理并通过 `guard` 等细节设计, 进一步地强化了其具有的安全属性。

在内存管理方面, seL4 的内存分配依赖于用户态的策略, 内核只提供了物理内存分配和虚拟内存映射的基本操作接口, 并创新性地结合 `capability` 机制进行统一的管理, 为整个系统的内存分配提供了灵活且安全的实现机制。

seL4 作为 L4 内核家族的一员, 同样继承了 L4 内核 IPC 上的 `fastpath` 等设计以提高系统 IPC 的性能, 除此之外为了系统安全性进一步地设计了基于 `Endpoint` 的通信机制, 线程间的通信通过 `Endpoint` 作为中介进行发送。

在系统的任务管理上, seL4 的微内核采用了单内核栈设计, 并通过基于优先级的调度队列进行任务调度。seL4 的快速 IPC 设计也一定程度上影响线程间的调度逻辑, 阻塞式的消息发送接口结合系统调度器的特定设计, 从而减少了 seL4 在被消息阻塞的线程上的 CPU 资源浪费。

除此之外, seL4 也有多种扩展特性支持。为了增强 seL4 的实时性, 增加了 MCS 特性, 以提供了时间上的细粒度管理, 这进一步地影响了 seL4 的通信模型和线程管理模型。为了支持多核的 CPU, seL4 提供了 SMP 特性, 使用基于大内核锁的方案提供了基础的多核支持。

最后, seL4 实现了形式化证明以证明其安全性, 尽管仍然有验证边界等条件, 但作为世界上首个经过形式化证明验证的微内核, 具有重要的里程碑意义。

第3章 基于 Rust 语言的组件化 reL4 微内核操作系统

3.1 reL4 的挑战和目标

3.1.1 挑战与动机

seL4 的内核代码的设计为实现形式化验证提供了便利，但其设计思想决定了，整个内核具有一定的耦合度，这在 seL4 的设计之初，支持的平台和架构数量少，支持的特性不多的情况下，是合理的，但是随着内核功能的扩展、内核边界的扩充、内核支持的平台的增长，这种高度耦合的代码结构逐渐暴露出弊端：它增加了代码维护的复杂度，制约了新功能的开发效率，并阻碍了其核心代码在其他项目中的复用。然而，出于对系统安全性与正确性的极致追求，直接对经过形式化验证的 seL4 内核进行大规模重构并不可行。

上述耦合性问题在李龙昊^[61]、廖东海^{[62][63]}等人的 reL4 内核项目的后续演进中得到了具体印证。由于继承了 seL4 原生的紧耦合架构并注重于功能实现的兼容，当尝试为 reL4 添加 AArch64 架构支持时，我们遇到了明显的代码侵入和集成困难；而对于 MCS、SMP 等更为复杂的新特性，其集成难度则更为突出。这些挑战不仅反映了具体的工程实践难题，更揭示了现有微内核设计哲学的一个局限：在强调“内核最小化”的同时，往往未充分重视内核内部模块间的清晰划分与解耦。这正对应了本文第 1.4 节所提出的问题一（内核耦合性与可扩展性问题）。

在寻求内核模块合理划分（应对问题一）的基础上，模块划分的质量标准随之浮现，这即引出了第 1.4 节所提出的问题二（功能可复用性问题）。我们认为，“可复用性”本身即是评估模块划分优劣的重要准则：一个合理的模块划分，应能产生高内聚、低耦合的组件，这些组件既能被本内核的其他部分有效复用，也能作为独立单元被其他项目集成。符合这一标准的模块，便构成了第 1.2.5 节中讨论的软件组件。当然，完全由通用组件构建一个内核是不现实的；每个内核都包含其特有的、与核心设计紧密绑定的专有组件。因此，reL4 的设计目标之一就是在通用组件与专有组件之间寻求平衡。

3.1.2 设计目标与原则

基于以上分析，我们为组件化 reL4 微内核确立了以下核心设计目标：

1. **完全的 Rust 实现：**reL4 最终应当是一个完整的 Rust 语言编写的内核，而非之前的 reL4 内核方案，即大部分使用了 Rust 语言重构，并使用库的方式和少部分 C 代码链接的方式。

2. **严格的二进制兼容**: reL4 需要兼容 seL4 的二进制接口,但不要求跟 seL4 具有一样的内部实现。这构成了 reL4 的组件化设计的重要挑战,因为它限制了我们进行重构和设计的实现边界。
3. **完全的组件化**: reL4 应当被设计为具有良好组件边界的组件化内核,而并非像 seL4 一样具有较强的耦合性,同时在通用组件和专有组件之间寻求平衡。

3.1.3 reL4 内核实现与解决的挑战

为此,我们基于已有的 seL4 内核设计和现有的 reL4 项目的基础,利用 Rust 语言的内存安全特性与组件化软件工程方法,设计并实现了组件化的 reL4 内核,并在后续的工程实践中,根据实际情况不断调整组件之间的边界,最终形成了如图 3.1 所展示的主要架构图中所示的多个系统组件,每个系统组件的说明如下:

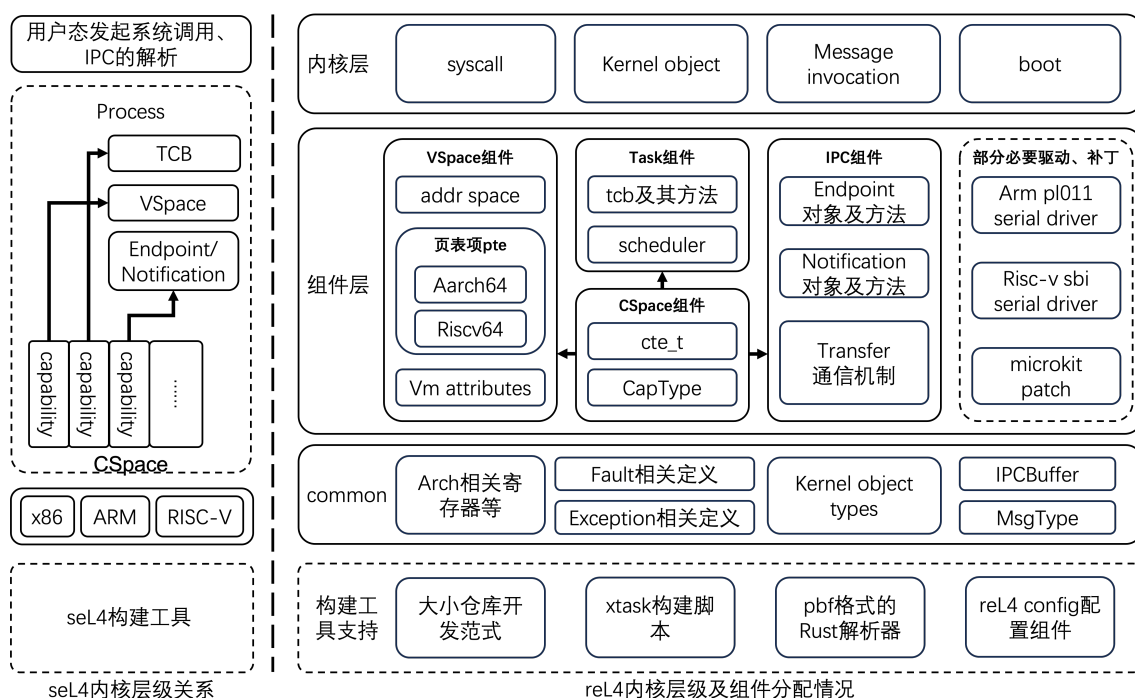


图 3.1 reL4 内核总体架构设计

- **common:** 作为最底层的基础组件支撑,提供各类 reL4 的基础数据结构,宏定义等内容。该组件旨在提供一组高效、低开销的基础抽象和公共数据结构,因此,该组件的相关内容会在第 3.2 节至第 3.6 节中所有使用到的组件中提及。
- **CSpace:** 提供了 L4 系列微内核所特有的 **capability** 的管理机制,维护了包含所有兼容 seL4 内核中 **capability** 的空间,即 **CSpace**,也包含了 **capability** 的插入删除等各类管理方法,该组件的核心挑战在于,如何在 Rust 的安全内存模型中,对 seL4 **capability** 的底层位操作进行安全且高效的封装,该组件会在第 3.2 节中所描述,基本严格对应于 seL4 的 **capability** 组件。

- **VSpace**: 提供了微内核的地址空间相关的操作, 相关内容会在 3.3 中描述。
- **Task**: 提供了微内核的任务管理相关基本操作, 相关内容会在第 3.5 节中描述。
- **IPC**: 提供了微内核中各类消息传递的相关操作, 相关内容会在第 3.4 节中描述。
- **kernel**: **kernel** 组件作为整个项目最顶层的上层组件, 使用了上述组件所提供的功能, 完成了整个内核的功能, 例如, 负责系统的初始化启动流程, 并作为系统调用的总入口点, 向下调度各组件以处理具体请求等。同样的, 如图 3.1 中所示, **kernel** 作为多个组件的顶层组件, 其相关内容也在本章的所有章节有所体现, 部分如第 3.6 节中所示的中断管理等部分, 也主要放在 **kernel** 组件中。
- **部分必要驱动**: 从原则来说, 微内核的驱动都应当放在用户态, 但是从实践来说, reL4 微内核本身也需要一些调试工作, 因此内核态实现串口驱动等工具是必要的, 同时, Rust 社区提供了很好的各类串口驱动的封装, 因此, 为了开发便捷, 引入了这一部分内容。

在完成了 reL4 组件化的基础上, 我们还验证了基于这样的组件化内核架构, 是否能够更方便地扩展内核功能。举个例子, 我们增加了 seL4 的 MCS 特性和 SMP 特性, 这些特性对整个内核的数据结构, 甚至 syscall 的语义都有较大的影响, 横切 (Cross-cutting) 多个组件, 因此, 我们不得不以条件编译 (feature flags) 的集成策略嵌入到内核中去, 而非增加独立组件, 这体现了在保持架构简洁性与支持复杂功能之间的一种实践权衡。相关实现在第 3.7 节和第 3.8 节描述。

在实现上述目标的过程中, 我们也面临并成功解决了以下关键挑战, 这些解决方案也成为了本文的贡献之一, 并在后续章节详细展开:

- **二进制兼容性保障**: 需要实现对 seL4 内核的二进制兼容性, 这需要全面测试验证所有 seL4 的相关特性。
- **安全与性能的权衡**: 实现中需要考虑 Rust 的语言特性, 例如: C 语言中的各类转换在 Rust 中是 `unsafe` 的, 而试图按照 Rust 的规则, 在一个已经经过验证的内核上增加 `unsafe` 块的封装检查, 又会引入额外的性能问题。
- **组件边界的划定**: 组件化构建模式需要合理的考虑各个组件的边界, 并非把一个功能的所有代码都放在一个组件中就是最优的设计, 以第 3.3 节将讨论的内存管理为例, 它实际按照图 3.1 中设计的那样, 被拆分为三个部分: **common** 中的一些基本的宏定义等内容、**VSpace** 组件中的一些基本操作方法、**syscall** 层的 **syscall** 解析和调用逻辑。从而让 **VSpace** 组件的方法可以更好的被复用。

- 专用工具链的开发：使用 Rust 语言重构整个内核还引入了一些额外的必要工作。在 seL4 中，除了内核的 C 代码本身之外，还有多种辅助工具以辅助系统的构建，相应的，reL4 内核也必须使用相应的多种辅助工具，来实现内核的构建，这些在第 3.9 节中介绍。

3.2 reL4 对 capability 机制的实现

如第 2 章所述，seL4 通过一个 128 位的 `capability` 结构体及其复杂的派生树来实现对内核对象的细粒度访问控制。重构此机制是 reL4 项目的核心任务与挑战之一。其挑战性源于一个核心矛盾：必须在保持与 seL4 的二进制接口（ABI）完全兼容（即 `capability` 的位域布局不可更改）的前提下，将其融入 Rust 严格的内存安全与类型安全模型之中。

seL4 为了便于验证，对内核对象部分以及 `capability` 的实现，设计并构造了一种 `bf` 文件，在其中描述了各种不同配置下的不同内核数据结构中，各个字段所占用的位数和偏移，并在编译期合并成为一个最终的 `pbf` 文件，最终通过脚本转为 `structure_gen.h` 头文件，从而被 C 代码所使用。

reL4 无法直接使用这些 C 头文件，因此必须开发相应的 Rust 替代方案，我们的解决方案也经历了一个从手工生成到自动生成的演进过程。

初期，reL4 使用一个 `plus_define_bitfield` 的宏进行处理，需要编程手动指明各类信息，包括每个 `capability` 所占用的 bytes，在 `capability` 中每个字段占用的 bit，对应的偏移，以及通过 bit 数和偏移量得到对应的数值之后，需要左/右移多少位以获得最终的值等内容。在经过这套宏定义转换之后，会生成对应名称的 `capability` 结构体和对应的 `capability` 中各个位域的读写操作。这套机制除了可以用于 `capability` 之外，也可以用于 seL4 其他类似的，在 `pbf` 文件中指明的结构体字段的解析。

但是它存在一个缺点，每增加一个 `capability`，都需要手动指明上述信息，尤其是其中的 bit 偏移等内容容易被写错，这是容易触发 bug 的重要原因。针对这个问题，reL4 为此开发了一个脚本（具体实现细节于 3.9.3 节中所述），更自动化地进行了 `capability` 的从 `pbf` 文件到最终的 reL4 中的 Rust 代码的转换，有效解决了手动维护的痛点，保证了与 seL4 内核二进制定义的一致性。

而 Rust 的安全语义引入的性能和安全的权衡则是另一重大挑战。

以 `capability` 为例，在 C 语言实现的 seL4 中，每个 `capability` 占用 128bit，并通过占用若干个 bits 的 `type` 字段用于指明其类型，在使用时需要根据这个 `type` 字段提供的类型信息来强制转换为相应的类型，其他的位域成员则在确定类型后，通过位操作来增删查改 `capability` 附带的各种信息。

但是, Rust 和 C 语言具有不同的类型系统。在 C 语言下, 通过检查其中的类型位域, 可以轻易地强制转换为对应的具体 `capability`, 而这样的强制类型转换, 在 Rust 下, 必然是一个 `unsafe` 的行为, 因为这打破了其安全的类型系统限制。由于这些 `capability` 本身在大量的系统调用中作为参数和用户态进行交互, 因此在考虑二进制兼容的情况下无法改写其数据结构的底层实现, 这导致了难以避免的对 `capability` 的 `unsafe` 操作行为。

Rust 中的 `unsafe` 并非不可接受, 只要做好封装即可解决。从实践的角度来说, 所有的 `capability` 都只占用 128bits, 并且所有的操作都是对其中的位操作, 这样的简单机制, 让这种强制类型转换足够简单, 易于封装。因此, reL4 封装了在抽象 `capability` 到具体的 `capability` 的 `unsafe` 转换, 允许在检查了对应的 `capability` 的 `type` 字段是否和目标类型相匹配之后, 进行强制的类型转换。

不过, 事情并没有那么简单。在 seL4 的实现中, seL4 内核中在任何需要做这样的强制类型转换的地方都进行了 `capability` 的类型检查, 且经过了形式化验证。而 reL4 在很多行为和设计上参考了 seL4, 这导致了在上述的封装好的强制转换中的 `type` 检查, 在此之前的代码逻辑中已经全覆盖地检查过了, 这带来了一定的实质上的冗余运算。由于 `capability` 的强制类型转换众多, 该重复检查会带来较大的性能损失。

对此, 我们使用 `debug_assert_eq!` 等 Rust 中的宏, 在只有 `debug` 的编译模式下才启用该检查。在 `Debug` 版本中, 该检查会生效, 用于捕获 reL4 自身可能存在的潜在 bug。但是在 `release` 版本中, 则会被编译器移除这一部分代码, 实现零开销的封装。

这虽然一定程度上偏离了 Rust 对 `unsafe` 代码进行严格封装的理想原则, 但它是基于对 seL4 验证过的逻辑的信任, 是在安全性与性能之间做出的一项经过深思熟虑的工程权衡。它体现了 reL4 的项目哲学: 在拥抱 Rust 安全性的同时, 不牺牲微内核的核心性能优势。

reL4 的 CSpace 组件基于上述安全封装的 `capability` 表示, 实现了与管理 `capability` 的增删查改 (`Insert`, `Delete`, `Search`, `Modify`)、派生 (`Mint`, `Copy`) 与撤销 (`Revoke`) 等核心操作。其实现向上层提供了安全、易用的接口, 完全复现了 seL4 `capability` 派生树 (详见第 2.2 节) 的管理语义。如图 3.2 所示:

在 `common` 层对 `bf` 文件进行解析的基础上, CSpace 组件对架构相关/无关的 `capability` 操作进行了封装, 封装成了 `cap` 相关的基础操作。并在此基础上进一步的构成了 `cte` 的操作集合, 提供给内核的其他组件使用, 通过 CSpace 组件的组件化封装, seL4 所特有的 `capability` 功能也可以较好地被其他需要相关功能的操作系

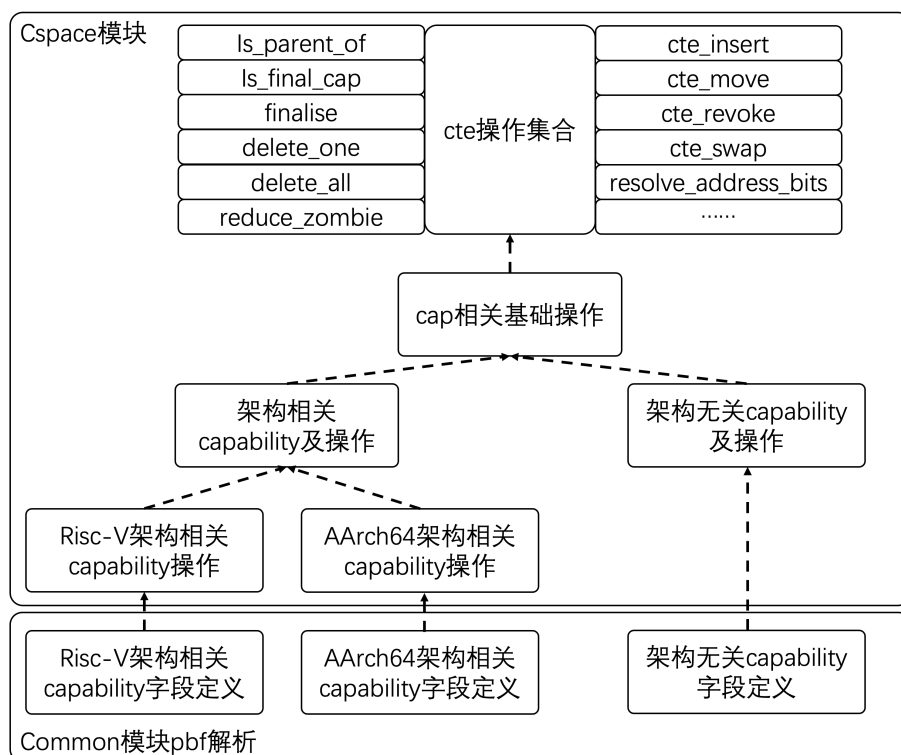


图 3.2 CSpace 组件架构

统所复用。

在上述的架构图中，对 **cap** 和 **cte** 进行了区分，所谓的 **cte** 是存放 **capability** 的插槽，而 **cap** 则是 **cte** 插槽中所存放的 **capability**。他们之间既有区别又有联系，区别在于，对 **capability** 管理来说的增删查改，往往实际上是对 **cte** 这个插槽做的操作，联系在于，对于 **cte** 插槽来说，他的有些操作也离不开里面存放的具体 **cap** 类型，比如对于 **cnode** 类型的 **cap**，可能需要进一步如图 2.2 中所示，查找下一级的 **cte** 插槽。

3.3 reL4 的内存管理与虚拟内存子系统

reL4 的内存管理系统是其组件化设计的典型代表，我们遵循机制和策略分离的原则，将其核心功能剥离至独立的 **VSpace** 组件中，而将内核对相关功能的调用逻辑放在 **syscall** 层中，根据内核解析 **syscall** 的需要来按需调用。这种设计使得 **VSpace** 组件成为一个与操作系统策略无关的、可重用的虚拟内存操作库，如图 3.3 所示。

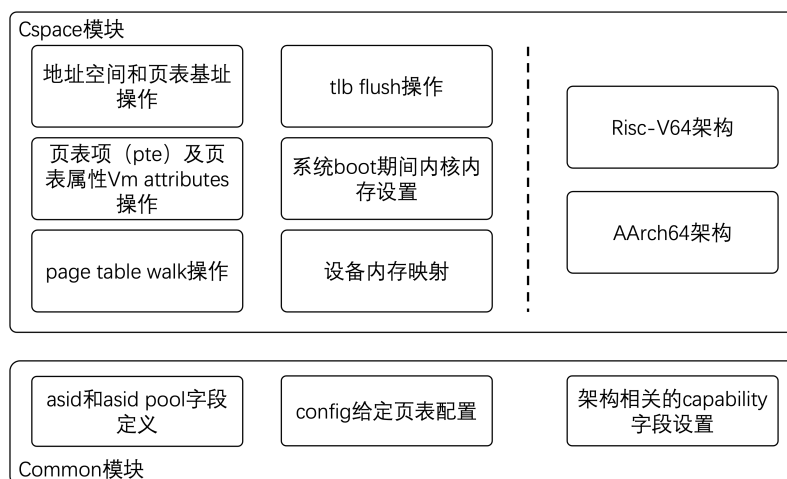


图 3.3 VSpace 组件架构

3.3.1 架构无关的 VSpace 组件设计

VSpace 组件的核心目标是封装与硬件架构相关的虚拟内存操作，向上提供统一的抽象接口，其功能主要包括：

- 页表操作：实现不同架构（目前实现了 aarch64、riscv64）下的页表遍历（Page Table Walk）、映射（Map）、解除映射（Unmap）等底层操作。
- 页表项管理：为不同架构的页表项（Page Table Entry）提供通用的类型定义，和页表项的各类属性的读写接口方法。
- 地址空间标识符（Address Space Identifier, ASID）管理：提供 ASID 的分配、回收、与页表关联等操作。
- TLB（Translation Lookaside Buffer）管理：提供按照 ASID 刷新，按地址刷新 TLB 等多种 TLB 无效化操作。

通过将这些不同硬件架构下，不同操作系统都需要的通用虚拟内存管理操作集成到 VSpace 组件中，reL4 将所有架构相关的内存管理代码进行了隔离，这也使得 VSpace 组件本身成为了一个有价值的独立模块，理论上可以被其他需要类似底层操作的操作系统项目所复用，体现了组件化设计的优势。

当然，如图 3.3 中所示，仍然存在部分模块属于 reL4 内核所特有的内存管理策略，例如系统 boot 期间的内存分配设置，和设备内存映射等功能。这些虽然放在 CSpace 模块中，但是每个内核都需要专门实现相关部分。

3.3.2 策略驱动的 kernel 层内存管理实现

在 kernel 层的内存管理，则根据兼容 seL4 接口的需求，专注于 seL4 相关的内存管理策略的实现，其主要任务包括：

- 页表相关调用的解析：解析来自用户态的内存管理模块调用的相关系统调用，

并根据其语义，进一步地调用下层 VSpace 组件提供的相应机制来执行具体的操作。

- 对 untyped 对象的 retype 操作：虽然同样是系统调用，但是页表相关调用的解析与 MMU 打交道，而这部分则是将 untyped 的内存对象，retype 为某个具体的内核对象。其主要依赖于 common 组件中的 capability 相关数据结构定义。与具体硬件相关的页表映射部分无关，因此其主要实现在 kernel 层。
- 系统启动阶段的初始化：在内核启动时，负责映射内核自身的代码与数据区、映射必要的设备内存（如串口）、初始化 root-task 的地址空间并将其所需的 Capability 填入其 CSpace，以及将剩余的物理内存初始化为 untyped capability 并授予 root-task。

3.3.3 组件化协作的实践价值

reL4 内存管理的实现清晰地展现了组件化内核的协作模式：kernel 层作为策略层，决定“做什么”（What）；而 VSpace 组件作为机制层，负责“怎么做”（How）。这种解耦有效提升了代码的清晰度与可维护性。例如，若要为 reL4 移植一个新的硬件架构，开发者只需专注于在 VSpace 组件中实现该架构的特定操作，而无需改动大量的上层系统调用处理逻辑。反之，若要修改某个系统调用的行为，也通常只需在 kernel 层进行调整。

综上所述，reL4 通过 VSpace 和 kernel 两个组件的协同设计，不仅复现了 seL4 强大的内存管理功能，更通过清晰的界限划分，实现了相对于原版 seL4 单体设计所没有的模块化、可维护性与可复用性。

3.4 reL4 线程间通信（IPC）机制实现

reL4 的 IPC 实现是其组件化设计的又一代表。如第 2.4 节所述，seL4 的 IPC 机制较为复杂，涵盖了快速路径、多种系统调用以及 Endpoint 与 Notification 两种抽象内核对象。为清晰地管理这种复杂性，reL4 将 IPC 功能分解至三个界限分明的组件中：common、IPC 和 kernel，这种分解严格遵循了分层与抽象的原则。

图 3.4 中给出了 common 组件和 IPC 组件中的相关内容，在 common 层提供的相关定义的基础上，IPC 层提供了统一的通用 transfer trait，并在此基础上，进一步地按照 endpoint 和 notification 的区别，向上层的 kernel 层提供了不同的调用接口。

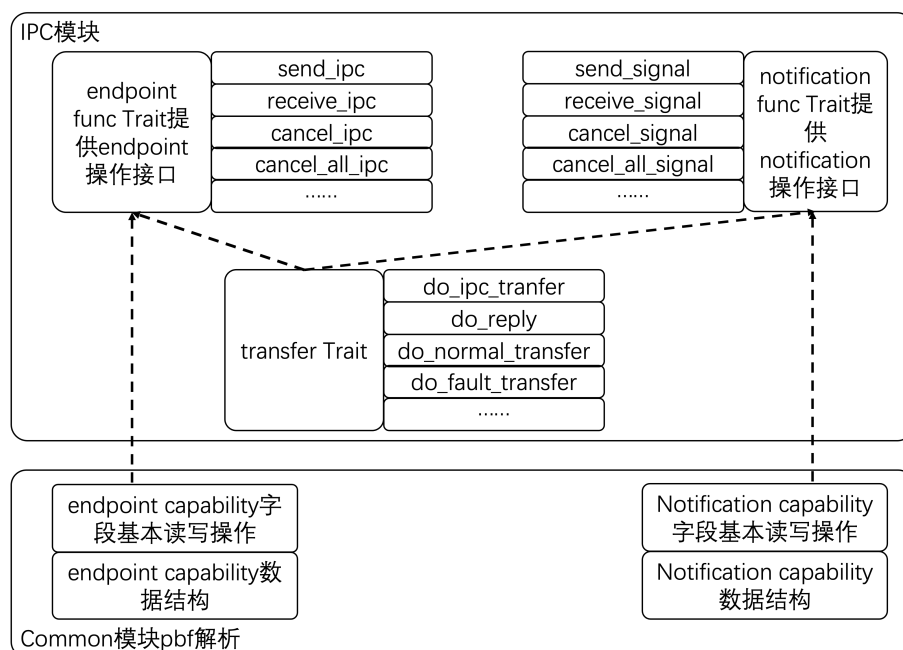


图 3.4 IPC 组件架构

3.4.1 基础定义与架构抽象

在 `common` 组件提供了 IPC 机制所需的基础数据结构和架构相关的常量定义，是所有上层组件共享的基石。其主要内容包括：

- IPC 缓冲区（IPC Buffer）结构体：定义了与 `seL4` 用户态布局兼容的消息格式。
- 消息寄存器（Message Registers）：定义了用于参数传递的寄存器集合。为兼容 `aarch64` 与 `riscv64` 架构，该组件封装了架构差异，向上提供统一的寄存器索引接口。
- 故障消息（Fault Message）：定义了在不正常情况下传递给用户态 `Handler` 的消息格式。
- IPC 相关 `Capability` 对象定义：提供了 `Endpoint`、`Notification` 等内核对象对应的 `Capability` 的 Rust 类型定义。

该组件中实现的上述内容的关键作用在于隔离架构相关性，确保 IPC 和 `kernel` 组件的实现与具体硬件平台解耦。

3.4.2 核心机制与统一传输层（IPC 组件）

IPC 组件是 IPC 机制的核心实现层。它不处理系统调用解析或策略，而是专注于提供底层的、可重用的操作原语。其核心贡献在于引入了 `Transfer trait` 作为统一的传输抽象。

- **IPC 相关的内核对象操作：**提供了对 Endpoint 和 Notification 内核对象中各字段进行操作的方法集（如检查状态、设置标签、管理等待队列等），这些方法是实现 IPC 语义的基础。
- **Transfer trait：**这是该组件的设计核心。我们抽象出了一个统一的 Transfer trait，其定义了 `do_ipc_transfer`, `do_normal_transfer`, `do_fault_transfer` 等核心方法。所有具体的 IPC 系统调用（如 `SYS_SEND`, `SYS_REPLY_RECV`），在 kernel 层被解析后，最终都会被转换为对一个或多个 Transfer 方法的调用。这种设计将复杂的 IPC 流程分解为一系列原子化的、可组合的操作，提升了代码的模块化和可测试性。

通过 IPC 组件，reL4 将 seL4 中交织在一起的 IPC 机制清晰地剥离出来，形成了一个独立的、可测试的库。

3.4.3 策略执行与兼容层（Kernel 组件）

kernel 组件是 IPC 的策略执行层和兼容性接口层。它不实现具体的传输逻辑，而是负责管理流程与策略。

- **系统调用解析与分发：**该组件解析表 2.2 中所有的 IPC 系统调用。根据调用号（如 `SYS_SEND`）和参数，它决策执行的流程，并编排调用 IPC 组件提供的 Transfer 方法来完成实际操作。
- **Fastpath 优化：**实现了 seL4 的快速路径（Fastpath）机制。在进行完整的系统调用解析前，会先进行条件判断。若满足快速路径条件（如目标 Endpoint 已就绪、消息长度符合要求等），则直接跳转到优化的处理流程，最大限度地减少开销。
- **错误与异常处理：**当 IPC 过程中发生错误或接收到异常（如 Page Fault）时，该组件负责根据内核策略，构造相应的故障消息，并通过 IPC 机制将其发送给用户态注册的 Handler。

3.4.4 reL4 的 IPC 组件化设计的价值

reL4 的 IPC 实现通过三层组件化设计，将 seL4 最复杂的子系统之一重构为一个清晰、可管理的结构。其价值不仅在于功能复现，更在于为如此复杂的系统功能提供了一种新的构建范式：

首先，Transfer trait 的引入创立了一个关键的抽象层。它将所有 IPC 系统调用的核心操作归结为一组定义明确的原子方法（如 `do_ipc_transfer`）。这使得内核中纷繁复杂的 IPC 系统调用（见表 2.2）被分解为对这些基础方法的有序组合。这种分解降低了 IPC 系统的认知负荷和测试难度，开发者可以孤立地验证每个 Transfer

方法的正确性，从而更容易保证整个 IPC 子系统的可靠性。

其次，组件化设计实现了对性能与安全顾虑的精准隔离。所有性能敏感的、需要 `unsafe` 操作的底层数据操作（如操作等待队列）被封装在 IPC 组件内。而所有策略决策（如 Fastpath 判断、调用流程编排）则被置于 kernel 层。这种隔离带来一个重要优势：开发者可以在不牺牲安全性的前提下进行深入性能优化。例如，优化 Fastpath 只需在 kernel 层调整判断逻辑，而优化 `do_ipc_transfer` 的实现只需在 IPC 组件内修改，两者互不干扰。这解决了系统编程中一个常见的权衡问题。

最后，这种架构展现了良好的可演化性。无论是扩展新的 IPC 系统调用，还是调整现有调用的语义，其主要变更通常被限制在 kernel 层的策略逻辑中，只需重新组合现有的 Transfer 方法即可。这与 seL4 单体设计中将逻辑分散在各处的模式相比，变更的局部性和可预测性都得到了提升。

综上所述，reL4 的 IPC 组件化设计表明，即便对于性能与复杂度俱佳的子系统，通过清晰的抽象（Transfer trait）和严格的层次划分，同样可以在获得 Rust 内存安全好处的同时，保持较高的性能水准，并最终得到一个更易于理解和演进的系统结构。

3.5 reL4 的任务管理与调度

与内存管理和 IPC 部分的拆分相比，reL4 的任务管理部分子系统的组件化拆分，是一个更为复杂的任务。seL4 的任务和 IPC 模块有着密切结合，这在 2.5 中的调度部分有着深入的探讨：seL4 的任务状态中就包含了通信相关的状态，而阻塞于发送或者阻塞于接收的状态，都会影响任务的调度。这种耦合意味着无法将调度策略简单地剥离到一个独立的组件中。reL4 的解决方案体现了组件化设计中的一种权衡：将通用的、稳定的机制与易变的、策略性的逻辑进行分离。

基于此，reL4 的 Task 组件被设计为调度机制提供者。它封装了线程控制块（TCB）的数据结构和基础的队列操作原语，例如 `tcb_queue_t` 调度队列以及对应的 `enqueue` 和 `dequeue` 方法。值得注意的是，为保持与 seL4 原设计在性能和行为上的一致性，该队列内部使用裸指针链表实现。在 Rust 中，此类操作被安全地封装在 `unsafe` 块内。其安全性基于一个事实：队列本身是一个链表的实现，而链表是一个简单的数据结构，因此在针对队头和队尾的操作中，通过仔细检查其是否为空指针等操作，可以简单地确认该调度队列的实现是否存在问题，只要把这些操作限制在一个小的、功能明确的模块边界内，并对其正确性进行推理和验证，就能在享受 Rust 安全生态的好处的同时，无损于性能。

而所有调度策略的决策权则被上移至 kernel 层。该层是 Task 组件接口的调用

者,负责处理由 IPC 或其他事件引发的复杂状态转换。例如,当处理一个 `SYS_SEND` 系统调用时,若目标端点未就绪, `kernel` 层会决定阻塞当前任务,并调用 `Task` 组件的接口将其状态标记为 `blocked on send` 并从就绪队列中移除。同样,在 `schedule()` 函数中, `kernel` 层依据任务的综合状态(包括其 IPC 状态)来选择下一个运行的任务。这种设计确保了所有涉及多子系统交互的复杂决策逻辑都集中在一处,使得系统的行为更加清晰和可预测。

在上下文切换的实现上, `reL4` 根据场景差异灵活地选择了不同的技术路径。内核的入口路径(如中断和异常向量)因其逻辑纯粹且与架构紧密耦合,并通过 `#[no_mangle]` 属性与 Rust 代码链接,以确保高性能。反之,内核的退出路径(如 `restore_user_context`)则需要处理更多的内核状态(如需要传入下一个任务的 TCB 指针)并可能调用其他模块(如解锁大内核锁)。因此, `reL4` 选择使用 Rust 的 `asm!` 宏以内联汇编方式实现此部分。这一选择使得底层汇编能够嵌入 Rust 的上下文,增强了复杂退出逻辑的可读性和可维护性。

综上所述, `reL4` 任务管理的设计展示了其在处理紧密耦合子系统时的组件化哲学。它并非追求机械地将某个功能全都集中在某个组件中,而是致力于建立清晰的组件间约定边界: `Task` 组件提供可靠、高效的底层机制; `kernel` 层则在此基础上,灵活地实现 `seL4` 复杂的调度语义。这种有层次的抽象将 `seL4` 的任务模型融入 Rust 的安全架构中,既未牺牲其性能优势,又为其带来了更好的模块化和可演化性。

3.6 reL4 在特定架构下的异常处理与系统优化

`seL4` 的中断处理框架(详见第 2.6 节)以其机制与策略分离的核心思想,为 `reL4` 奠定了坚实的基础,该通用框架在 `reL4` 的 `kernel` 组件中已得到实现。然而,将微内核跟具体硬件平台(如 ARM)的结合过程,往往会暴露出深层次的系统性问题。本节将不再赘述中断部分的通用实现,而是重点探讨 `reL4` 在 ARM 架构下面临的三个代表性的中断相关的实现。这些案例分别涉及硬件状态管理、安全特权指令交互和性能优化,体现了在组件化架构下中断和陷入操作设计的哲学。

3.6.1 ARM FPU 支持

高效管理浮点单元(FPU)这类大型、高开销的硬件状态,是系统内核设计的常见挑战。ARM 架构的 FPU 默认关闭,使用浮点指令会触发异常,这为实现懒惰式(Lazy)状态管理提供了硬件基础。

`reL4` 借鉴并实现了 `seL4` 的懒惰式 FPU 管理机制。其核心思想是将状态开启

的时机推迟到最后一刻，以避免不必要的开销。具体而言：系统启动时 FPU 全局关闭；当线程首次使用浮点指令触发异常时，内核才为其开启 FPU 并保存上下文。为防止此后该线程不再使用 FPU 却仍占用保存资源，reL4 引入了一个使用计数器：在多次上下文切换均未再次触发 FPU 异常后，内核会自动关闭其 FPU 权限，直至其再次使用。

不同的线程需要使用 FPU 的情况也各有不同，因此，需要维护每个线程的浮点寄存器的同时，还需要维护对应的浮点单元的控制寄存器的值。在线程切换的时候，使用对应线程的浮点单元控制寄存器的值。

另外，在具体实现中，也遇到了一些 Rust 相关的问题，在上下文保存过程中，浮点寄存器占用了 512Bytes 大小的空间，在这之后还需要再保存浮点控制寄存器，不过，Rust 的汇编语法只能支持 512Bytes 的偏移，因此如果只传入一个浮点上下文的指针地址，就无法保存浮点控制寄存器。为此我们做了额外的设计，传入两个指针分别指向浮点上下文和浮点控制寄存器保存地址。在保存完浮点寄存器之后，前一个用于指针的寄存器已经不再使用，正好用于保存通过 ARM 的 `mrs` 指令和 `msr` 指令从浮点控制寄存器中读取的值。这一解决方案不仅绕过了语言限制，更保证了较低的性能损耗，是在 Rust 安全模型中高效操作硬件状态的一个成功实践。

3.6.2 ARM 的 SMC call 的支持

ARM 的安全监视调用（SMC）是 CPU 从非安全世界（EL1）切换到安全世界（EL3）以执行最高特权操作的指令，对于 reL4 来说，这是一个额外的功能，但是对于一个完整的 ARM 环境来说，是不可或缺的，其重要作用是通过调用 SMC，来调用 ARM 的 PSCI 功能。而通过 PSCI 的通用接口，可以以通用的方法而非厂商私有的方法，来实现关机，以及多核启动等重要操作。如何安全、可控地将此类特权指令暴露给用户态，是微内核设计中的一个经典问题。

SMC 指令的执行本身会触发一个同步异常，所以实质上也是个 ARM 中断和异常体系中的重要部分。reL4 严格遵循 seL4 的机制与策略分离原则来解决这一挑战。内核不直接向用户态暴露 SMC 指令，而是提供一个专用的系统调用作为安全代理。用户线程需持有相应的 Capability 才拥有发起 SMC 调用的权限。内核在处理该系统调用时，会执行严格的参数检查，最终代表用户态在正确的时机执行 SMC 指令。

这种由内核代为执行的特权指令调用逻辑是与通过 capability 让内核进行操作的思想是一脉相承的，确保了 SMC 这一重要能力处于内核的有效控制之下，杜绝了用户态的滥用，是微内核细粒度权限控制原则的体现。它将功能的“机制”（执行 SMC）与“策略”（谁、何时、如何执行）清晰分离，既保证了安全性，又提供

了必要的功能。

3.6.3 ARM 的用户态时钟读取支持

追求高性能常常需要突破传统的抽象边界。在操作系统设计中，频繁读取高精度时钟计数器（如 ARM 的 CNTPCT_EL0）是一个常见的需求，但传统的通过系统调用读取的方式会带来较大的开销。

reL4 为此提供了一种积极的优化方案：通过配置 ARM 的性能监视寄存器（CN-TKCTL_EL1），直接允许用户态在 EL0 特权级下安全地读取物理时钟计数器（CNT-PCT_EL0）。这意味着应用程序无需陷入内核即可获取精确的时间戳，消除了系统调用带来的性能损失和延迟不确定性。同时，这样的操作也是具有相当的安全性的，时钟计数在用户态是只读的，且其读取操作不涉及任何特权状态的改变。

此优化虽然是一个小特性，但具有重要意义。它代表了 reL4 的一种设计哲学：在确保安全的前提下，尽可能利用硬件特性将性能关键路径下放到用户态。这不仅是性能的提升，更是对“什么必须在内核、什么可以放在用户态”这一微内核核心问题的深度思考与实践，是微内核最小化内核原则的一个体现，为构建高性能的用户态系统服务（如网络、存储）奠定了基础。

现有的设计也有一定的不足，目前这种特性并非使用 `capability` 进行管理，也就是说一旦开启，则用户态的所有程序都可以读取，虽然从安全性的角度来说没有什么特别的影响，不过可能不一定完全符合细粒度权限控制的设计原则，未来的工作应当更进一步设计相应的 `capability` 及其操作来对这部分功能进行管理，以保证该功能对不同的用户可以有不同的开启状态。

3.7 reL4 的混合关键系统（MCS）支持

如第 2.7 节所述，seL4 的 MCS 扩展将其安全模型从空间域延伸至时间域，是一项复杂的设计。它引入了 `scheduling_context` 和 `sched_control` 等全新内核对象与其对应的 `capability`，并深刻改变了调度器及 IPC 的语义。为 reL4 实现相关的特性，仅仅参考 seL4 实现相关功能是不够的，如 3.1 节所述，其实现不再是一个单独的独立模块，而是被设计为一个横切多个模块的通过条件编译进入内核的部分，这也对 reL4 的内核架构提出了挑战。

3.7.1 新的内核对象与内存管理

为了将时间作为一种可管理的内核资源，reL4 引入了与 seL4 对应的两个核心内核对象：`scheduling context (sc)` 和 `sched_control`。其中，`sc` 是 MCS 特性的主要

抽象，它封装了线程的执行预算、周期和优先级等重要调度参数，而 `sched_control` 则用于管理系统范围内的调度资源。除此之外，为了支持时间预算的传递，也同样实现了 `reply` 对象，在其中使用双向链表指针的结构封装了一个调用栈，这同样是一种 C 风格的，直接操作裸指针的一种 `unsafe` 方式的代码，为了兼容 seL4 的二进制接口，这也是一种合理选择。为此需要额外地说明为什么在 reL4 的 `reply` 对象中的这一双向链表实现是安全的，从直接操作来说，其所有操作方法都和数据结构定义放在一起，且代码数量并不多，其边界条件可以通过少量严格检查来确保链表操作的安全性。这些对象的实现，目前放在 `common` 组件中，以便于上层组件的使用。

reL4 在实现这些对象时，特别考虑了 seL4 的 Sporadic Server 算法对内存布局的需求。该算法需要为每个 `sc` 对象维护一个 `refill` 数组，用于记录时间预算的重填信息。为了避免额外的内存分配开销，reL4 利用了其 `untyped` 内存管理器的特性：尽管为 `sc` 对象申请的空间仅为其所需大小，但实际分配的内存块总是按照 2 的幂次向上对齐（如 2.3 所述）。这导致 `sc` 对象之后总有额外空间可以利用。reL4 利用这部分多余空间，紧邻 `sc` 对象来存放 `refill` 数组，实现了高效且紧凑的内存布局。

这一设计并非为了性能的优化，而是对 seL4 中 MCS 的原始内存管理语义的精确复现，这一实现也证明了，使用 Rust 内核编写的 reL4 同样具备管理这部分扩展内存的能力。

3.7.2 时钟系统、消息系统与调度器的改造

在实现基本的数据结构和操作之后，需要首先对时钟系统进行改造。在无需混合关键系统的情况下，只需要定时触发中断即可，但是，为了增加了混合关键系统支持，必须读取到准确的当前时钟计数值，这需要改造时钟组件的相应接口。

同时，在时钟组件的支持过程中，reL4 还考虑到了不同时钟平台的问题。以 ARM 为例，在 ARM 下有很多不同的板子，原生的 seL4 不仅支持 QEMU 模拟器，还支持了大量不同的开发板，他们可能存在共有的特性，所以在 `arch aarch64` 下都是可以使用的，但是也有一些平台特有的时钟的特性，比如如何初始化时钟需要往特定的寄存器按照特定的序列读写，这些是跟 `platform` 相关的。

因此我们设计了一个可复用的 `trait` 用于描述时钟的行为，虽然目前仅有 `aarch64` 和 `riscv64` 下的 QEMU 模拟器平台的时钟支持，但是这一 `trait` 为 reL4 移植到新的硬件平台上时提供了时钟组件的可移植性，这是 Rust 语言在内核抽象设计中的重要应用，因为它将接口（`trait`）与具体实现（`impl`）分离，从而实现了低耦合的设计。

代码 3.1 Timer 功能 trait 定义

```

1 pub trait Timer_func {
2     fn initTimer(self);
3     fn getCurrentTime(self) -> ticks_t;
4     fn setDeadline(self, deadline: ticks_t);
5     fn resetTimer(self);
6     fn ackDeadlineIRQ(self);
7 }

```

调度器也进行了相应的修改，在进入内核的多个地方，都会试图读取时间并使用 `checkbudget` 函数进行检查，更新时间戳信息并判断当前线程是否使用超过赋予给它的 CPU 带宽。在相应的 TCB 的 `enqueue` 和 `dequeue` 操作中，也需要遍历所有的 TCB 结构体查看其重填时间 `rtime` 来做决策，这相当于按照 `scheduling context` 的 `rtime` 进行排序的工作。

对应的，正如在第 2.7 节中对 `seL4` 中的 `timeout` 消息传递的描述，在某个线程使用超过赋予给他的 CPU 带宽之后，需要发送相应的 `timeout` 消息给该线程，这一套消息发送机制基本依赖于原有的消息发送机制，只是需要额外处理这一种情况即可。而这一个将处理超时对应的策略交给用户态决定的思路，再次体现了 `reL4` 内核对机制与策略分离原则的坚持。

同时，由于添加 MCS 特性后的不同 IPC 语义，也对 IPC 组件中 `transfer` 的 `trait` 中的函数接口增加了相应的参数。同时，出于兼容的目的，必须保留 `reL4` 中没有 MCS 的基础版本的代码，为此，`reL4` 选择了使用 Rust 的条件编译的解决方案，根据是否存在 MCS 这一 `feature` 来进行条件编译，从而实现了对同一个 `transfer` 的 `trait` 在不同条件下的不同接口实现进行了封装。MCS 这一特性对内核的影响实质上跨越了除了 `VSpace` 之外的所有组件，类似于 IPC 组件中 `transfer` 的 `trait` 的条件编译处理，是整个 `reL4` 内核中添加 MCS 又不破坏原有的组件间关系的重要方式。

3.8 reL4 的多处理器（SMP）并行支持

`reL4` 在 SMP 支持上的设计目标，是在 Rust 中精确复现 `seL4` 的核心 SMP 机制，同时利用 Rust 的语言特性来确保实现的安全性和可维护性。我们的工作主要集中在三个方面：每核（`percpu`）变量的管理、大内核锁的实现，以及核间通信与调度的精细化处理。

3.8.1 每核（percpu）变量的抽象与管理

在多核系统中，每个处理器核心都拥有其独享的资源，例如本地中断控制器和计时器等。seL4 通过简单的数组扩展和宏来存放并访问这些 percpu 变量。reL4 延续了这一设计思想，并利用 Rust 的强类型系统和宏机制实现了更安全、更易于维护的方案。我们定义了 `NODE_STATE` 宏，它在编译时通过条件编译 `#[cfg(feature = "enable_smp")]` 确保只在启用了 SMP 功能时才生效。该宏将对每核变量的访问封装在一个 `unsafe` 块中，并通过 `cpu_id()` 接口获取当前核心 id，从而访问一个全局数组。这种设计将底层裸指针和数组索引的直接操作限制在宏内部，为开发者提供了类型安全的封装接口，同时保留了直接访问底层硬件的灵活性。

代码 3.2 SMP 变量管理的宏定义

```

1  #[cfg(feature = "enable_smp")]
2  #[macro_export]
3  macro_rules! NODE_STATE {
4      ($field:ident) => {
5          unsafe { $crate::ksSMP[sel4_common::utils::
6                  cpu_id()].$field }
7      };
8  }

```

3.8.2 大内核锁的实现

seL4 的大内核锁通过在内核入口处获取，并在出口处释放，从而保证了同一时刻只有一个核心在内核中执行。为了在高核心数平台上减少锁竞争带来的缓存同步开销，seL4 采用了基于队列的 CLH 自旋锁。

reL4 精确地复现了这一机制，并充分利用了 Rust 语言的原子类型来构建这个复杂的同步原语。我们基于 `core` 库的 `AtomicPtr` 对象及其 `load` 和 `store` 方法，利用了 Rust 提供的内存顺序（memory ordering）保证，在不依赖外部库的情况下，构建了高效且正确的同步原语。它证明了 Rust 在系统编程中，能够实现与 C 语言同样复杂和高性能的并发机制，为 reL4 在多核平台上的正确性提供了保障。

另外，考虑在 2.8 节中讨论的内容，现有的大内核锁未必是当下最佳的选择，可以在 reL4 的未来实现中，更进一步地探索使用 Rust 的细粒度的锁结构或者无锁的结构，来探讨内核中锁结构在当下或者更新的硬件平台上的性能损失问题。

3.8.3 核间通信与细粒度调度

另外，在 reL4 中，也需要实现多核之间的调度队列的处理。在某个核心抢到锁进入内核后，设置某个 TCB 的 affinity 或者其他操作，会导致其他核心上的队列被破坏。reL4 选择发送一个 IPI（Inter-Processor Interrupt）来通知目标核心，让目标核心在进入内核之后，处理相应的操作。这种事件驱动的核间通信方式，是 seL4 乃至 reL4 确保数据一致性的重要手段，尽可能地避免了全局锁带来的阻塞问题，让内核可以快速完成关键操作而将后续调度处理交给目标核心，进而提高了系统的响应性。

3.9 reL4 构建系统与工具链支持

一个操作系统的实现不仅依赖于其内核设计的质量，也离不开高效、可靠的构建与集成工具链的支持。reL4 项目因其组件化架构以及对 seL4 二进制兼容而引入了一定的复杂性。本章节将阐述我们为管理此复杂性而开发或者适配的一系列专用工具链，这些工具是保障 reL4 高效迭代的重要基础设施，其整体的构建流程如图 3.5 所示。

整个体系通过“双模式触发”启动：既可经由开发者在本地手动执行命令，也可由代码仓库的推送事件自动触发 CI 服务（如 GitHub Actions），从而在干净环境中保证构建的一致性。此后，rust_seL4_pbf_parser 和 reL4_config 生成器等组件会根据输入命令中的 feature 配置，解析硬件架构描述文件和 config 文件，自动生成符合 seL4 二进制布局的 Rust 数据结构代码。所有这些生成的代码与 reL4 各组件的源代码最终由统一的 xtask 脚本进行中心化编排，并进一步调用 Rust cargo 构建系统，针对不同目标平台与功能特性完成编译、链接，最终产出可供测试运行的 reL4 内核二进制镜像并进行运行。

3.9.1 组件化 Rust 项目的大小仓库开发范式

严格的组件化要求每个内核组件作为独立单元进行版本控制与维护，这种设计导向了多仓库的代码管理模型。然而，多仓库在并行开发和集成测试中带来了协调挑战。一方面，我们承认必须使用组件化的思想，将组件拆分为不同的 GitHub 仓库。另一方面，在多仓集成的开发环境和代码快速迭代的开发需求下，使用单个仓库，具有较高的开发效率和迭代效率。为兼顾组件独立性与开发迭代效率，reL4 采用了一种“大小仓库”协同开发管理范式，同时维护一个一体化的主仓库（用于集成开发）和多个独立的子组件仓库（用于发布与维护），并在进行 git push 等操作时进行同步，从而实现大小仓库协同开发的方案，该方案兼顾组件化的可重用

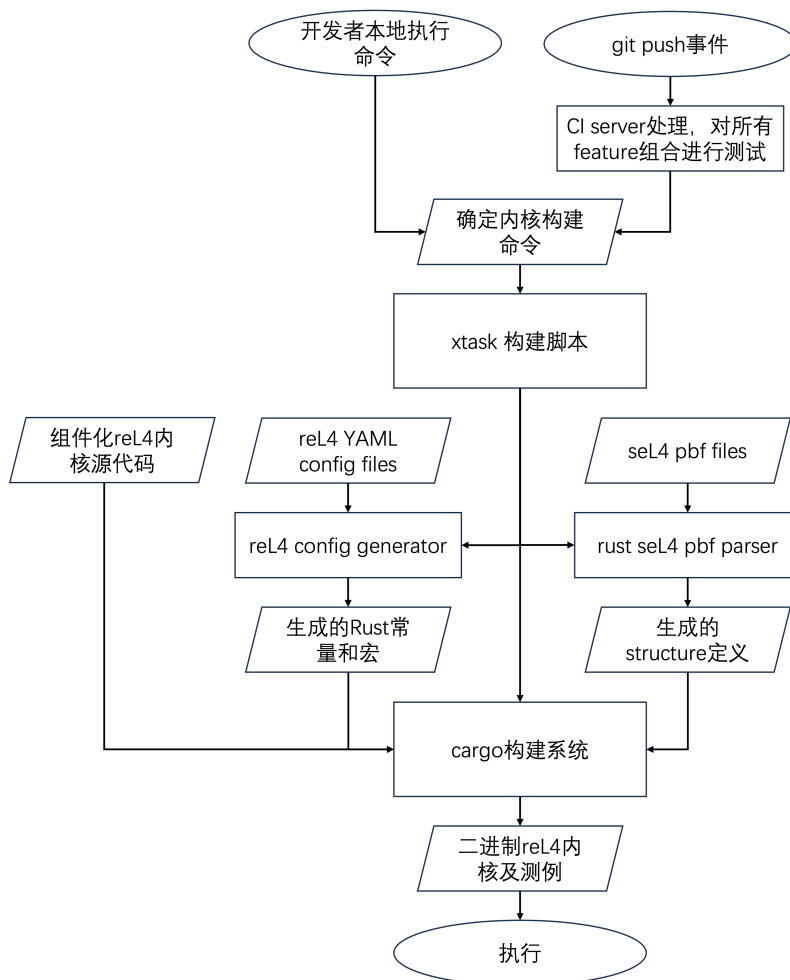


图 3.5 reL4 构建流程

性和单一仓库的快速迭代优势。

该范式的实现基于 `git-subrepo` 工具，它能够替代简单的 `git` 和 `git submodule` 工具所存在的大小仓库同步不便的问题。该工具可以单独使用类似于 ‘`git subrepo push sub-module-dir`’ 的命令，提交某个子仓库，也可以使用传统的 `git` 方式，`push` 整个大仓库。

为了自动化便捷开发，我们将该工具与 GitHub 的 CI 流程深度集成，在主仓库触发了 `push` 的 CI 操作的时候，能够自动触发 `git subrepo` 的 `push` 操作，将变更同步到多个子仓库中，同时单独触发多个子仓库各自的 CI，进行单元级别的自动化测试，从而实现大小仓库的协同工作的同时，也能进行集成化的单个组件的独立性测试。

这一大小仓库协作方式不仅仅是 reL4 内核的工程实践，而是一种组件化项目的方法论范式，为其他大型系统软件的组件化开发提供了可借鉴的工程实践框架。

3.9.2 reL4 的 xtask 构建脚本支持

reL4 的构建脚本经历多轮的变动，最初只是一个 python 脚本，但是，随着 feature 数量的增加，一个简单的 python 脚本已经无法支撑，此时，我们面临两个选择，一个是更进一步地使用更多 python 脚本，另一个方案则是使用 Rust 的构建系统，reL4 选择了采用 xtask 实现不同目标下的集中构建脚本，以更贴近 Rust 构建语义。

reL4 实现的 xtask 构建脚本最终完成了复杂的构建目标，包括：

- 组件化与集成构建：每个子组件都可以通过 cargo build 编译，同时支持整个项目的 cargo xtask 构建，体现项目的组件化功能。
- 功能配置管理：传递不同的 feature 给内核并构建，这需要处理一些 feature 之间的不兼容问题以保证鲁棒性。
- 双模式构建兼容性：reL4 最开始是作为一个库的方式跟 seL4 链接，这有利于各种调试和开发，但是 reL4 也实现了作为单独一个完整的 Rust 内核进行构建的方式，因此需要同时兼容两种方式。
- 代码生成与集成：调用后文提到的 pbf 解析脚本和集中配置脚本，在编译期根据 feature 生成一些 Rust 代码。
- seL4 测试集成与兼容性：seL4 会将内核和 test 等内容绑定放在一个文件中，reL4 xtask 需要能够兼容运行 seL4test 的方式和使用其他自定义的测例运行的方式。

3.9.3 reL4 的 pbf 文件格式解析支持

reL4 使用 Rust 语言实现了一个 rust_sel4_pbf_parser 工具，如第 3.2 小节所述，对 seL4 的 capability 的解析工具存在两个阶段，该 rust_sel4_pbf_parser 工具最初是因为 reL4 前一个阶段所使用的宏定义处理方式来解析定义 capability 不同字段存在各类问题——该宏定义方式要求开发者手动处理复杂的位偏移和地址计算，这不仅效率低下，也容易引入人为错误。

reL4 在 rust-seL4 的 PBF 解析器基础上进行了扩展和修复。rust-seL4 的 pbf 解析器忽略了 RISC-V 的 sv39 分页，而只支持 48bit 分页，同时，其主要面向用户态，而不考虑内核态高地址空间，从而导致缺少了前导的符号扩展，另外，该 pbf 解析器在解析 seL4 的 pbf 格式上，某些格式缺少解析。

reL4 的 pbf 格式解析器针对这些问题进行了修复。除此之外，针对在 capability 部分描述的 capability 类型的强制类型转换，pbf 解析器增添了一些 unsafe 的强制类型转换封装规则，从而便于 capability 的操作。

3.9.4 reL4 的集中配置脚本支持

reL4 的集中配置脚本主要针对于 seL4 在代码中各类定义过于复杂的问题，以及在原有的 reL4 中，各个常量定义不够统一存放的问题，进行了集中式的管理。在 seL4 中，使用了大量的 `#define` 语句来定义一些宏。这主要分为两类：写死的常量和根据编译时的配置选项定义的配置信息，它们被存放在各个不同的文件中。

reL4 为了解决这种定义存放的分散性，使用一个 `yaml` 文件统一存放了这些配置信息，并通过一个单独的 `reL4_config` 组件生成对应的 `config.rs` 文件和 `platform_gen.rs` 文件，前者存放写死的常量，而后者则是使用 `tera` 生成对应平台和对应该配置选项的配置信息。

这种集中式的配置管理方式，有效提升了项目的可配置性与可维护性。

3.10 本章小结

本章系统地阐述了 reL4 微内核的设计哲学、核心实现与工程实践，证明了在 Rust 语言的现代软件工程范式下，可以复现并改进像 seL4 这样复杂且经过形式化验证的微内核。

我们首先明确了 reL4 的三大设计目标：完全的 Rust 实现、严格的二进制兼容性以及完全的组件化。本章的其余部分，正是围绕着如何达成这些目标而展开的。

- 设计挑战与解决方案：我们首先从宏观上介绍了 reL4 的组件化架构，并明确了其与 seL4 单体架构的本质区别。我们随后深入探讨了如何利用 Rust 的内存安全特性来封装底层硬件操作。
- 核心子系统实现：我们详细阐述了 reL4 中最复杂的三大子系统（内存管理、IPC 和任务管理）的组件化实现。通过将机制（VSpace, IPC, Task）与策略（kernel）分离，我们不仅成功复现了 seL4 的全部功能，更通过清晰的接口和层次划分，提升了系统的模块化、可维护性与可重用性。
- SMP 与特定架构实践：本章还探讨了 reL4 在 SMP 支持以及 ARM 架构下所面临的挑战。我们精确复现了 seL4 的大内核锁和核间通信机制，并利用 Rust 原子类型实现了高效的同步原语。在 ARM 架构相关的异常处理中，我们通过对 FPU 状态管理、SMC 特权指令代理和用户态时钟读取等细节的讨论，展示了 reL4 如何在保证安全性的前提下，利用硬件特性进行性能优化。
- 工具链支持：最后，本章深入介绍了 reL4 的构建系统与工具链。我们提出并实践了大小仓库协同开发范式，解决了组件化项目在版本控制与集成上的挑战。同时，我们详细介绍了 `xtask` 构建脚本、PBF 解析器和集中配置管理等工具，这些基础设施有效地提升了项目的开发效率、可配置性和鲁棒性，是

reL4 项目可持续发展的重要保障。

总而言之，reL4 并非是对 seL4 的简单语言移植，而是一次系统性的软件工程重构。它在实践中证明了，将一个经过验证的微内核与 Rust 的现代编程范式相结合，能够产生一个兼具形式化验证的严谨性、Rust 的安全性与组件化的高效性的先进系统软件。

第 4 章 reL4-Linux-kit 用户态兼容层

4.1 reL4-Linux-kit 设计目标、原则和总体架构

4.1.1 挑战与动机

微内核架构（如 seL4）将操作系统核心功能最小化，但其完整可用性依赖于用户态的系统服务组件（service components），这些系统服务组件往往以用户态线程或者用户态进程的形式呈现。虽然 seL4 微内核对基本的内核操作提供了支持，但是，其缺少一定的应用生态，制约了其发展，而基于 seL4 微内核的系统也往往被设计实现为一个特定场景下的专用操作系统的内核，而非一个通用操作系统的内核，这就是第 1.4 节所提出的问题三（生态兼容性问题）。

在之前李龙昊^[61]、廖东海^{[62][63]}的 reL4 微内核工作中，只是通过了 seL4test 的测试，验证了内核的实现正确性，却缺少作为一个系统所需要的必要的用户态系统服务组件，更不必说对通用操作系统的兼容。因此，构建一个位于微内核之上的用户态兼容层，以透明地支持主流操作系统（如 Linux）的应用生态，成为推动组件化微内核走向通用的关键。

第 1.4 节所提出的问题二（功能可复用性问题）同样也存在于此，传统意义上的部分宏内核功能，在微内核下作为系统服务组件以用户态线程的形式呈现。幸运的是，已涌现出诸多成熟、组件化的宏内核实现（如 ArceOS）和成熟的内核组件。我们希望能够复用相应的组件化的宏内核代码和组件，快速且安全地完成微内核用户态各类系统服务组件的适配，而非“重复造轮子”式地重复开发。

在实际实现中，还存在一些更为细致的问题。

第一，通信范式的转换，在传统的宏内核下，宏内核的内核功能作为内核空间的一部分，用户态的应用和这些宏内核功能，通过系统调用进行通信，但是，在微内核的架构下，宏内核的内核功能作为用户态线程运行，用户态的宏内核应用，如何和宏内核的内核功能进行通信，成为了一个必须要回答的问题。我们基于现有的微内核线程间通信的方式，构建起一套宏内核应用和用户态的宏内核功能进行通信的机制。

第二，隔离与性能的权衡，系统内核服务的所有组件都拆解到用户态并进行频繁通信，必然带来一定的通信开销。以鸿蒙微内核的实现作为参考，为了缓解 IPC 的通信代价，我们将用户态程序设计为，可以灵活地将若干个系统服务组件组合起来，将基于 IPC 的通信方式转为最常见的函数调用通信方式，降低组件间的

成本。这种转换还要求提供便捷的 IPC 和函数调用的统一封装。

第三，面向未来的多实例隔离愿景，seL4 强大的隔离能力为更宏大的愿景提供了可能：在单一硬件平台上，同时运行多个相互隔离的“宏内核实例”。每个实例可拥有独立的服务集合与资源视图，这为云原生与 Serverless 等新兴计算范式提供了理想的安全隔离基础。用户态兼容层的设计需为这一远景预留架构上的可能性。

4.1.2 设计目标与原则

基于以上挑战和相应的动机，我们设计了一个用户态兼容框架 reL4-Linux-kit，并提出了以下 reL4-Linux-kit 的基本设计目标和原则：

- 使用 reL4 微内核作为基础内核，使用已有的成熟 rust-seL4 用户态库等模块提供的运行时支持。
- 只使用 Rust 语言从头编写若干必要的系统服务组件，而尽可能地大量复用现有的其他组件化 Rust 操作系统的组件。
- 通过 trap 操作转 IPC 技术，将通用操作系统（如 Linux）的系统调用，发送到用户态系统服务组件进行相应的处理，从而支持通用操作系统的各类 trap 到内核中的通信行为，兼容部分现有的 Linux 用户态进程，提升 reL4 微内核系统的通用性。
- 可以实现模块间灵活的组合与解耦，以达成性能和灵活性的平衡。同时对这种灵活组合情况下的通信增加统一的封装，如图 4.1 中所示，在同一个进程地址空间中的两个线程需要通信时，就使用函数调用，而在不同的进程地址空间之间的线程则通过 IPC 通信，这些都被很好地被封装在一个统一通信接口层中，而不被用户程序所感知。

据此，reL4 被设计为图 4.1 中所示架构。在 seL4 或者兼容 seL4 的 reL4 内核的基础上，root-task 提供了一些基本的系统资源管理功能（第 4.2 节），而一些 Linux service threads 通过 IPC 的方式向 Linux app 提供兼容 Linux 的各类 trap 服务，在第 4.3 节描述了 Linux 的各类 trap 服务是如何进行消息传递的，而在第 4.5 节描述了在 reL4-Linux-kit 中我们提供了什么样的用户态 Linux service。出于对 1.4 节所提出的问题二（功能可复用性问题）的回答，这些 service 除了必要的框架之外，都应当从现有的成熟 Rust 组件化操作系统的组件转换而来，并可以通过框架层灵活的完成组件间的拆分与合并（第 4.4 节）。

reL4-Linux-kit 的设计和实现也并非绝对意义上的“从头”开始，目前已有一些 seL4 环境下的 Rust 库可以复用，比如 rust-seL4 项目，初步完成了对于 seL4 原生用户态库的 Rust 环境的支持。除此之外，Rust 社区也有大量可以复用的代码库，

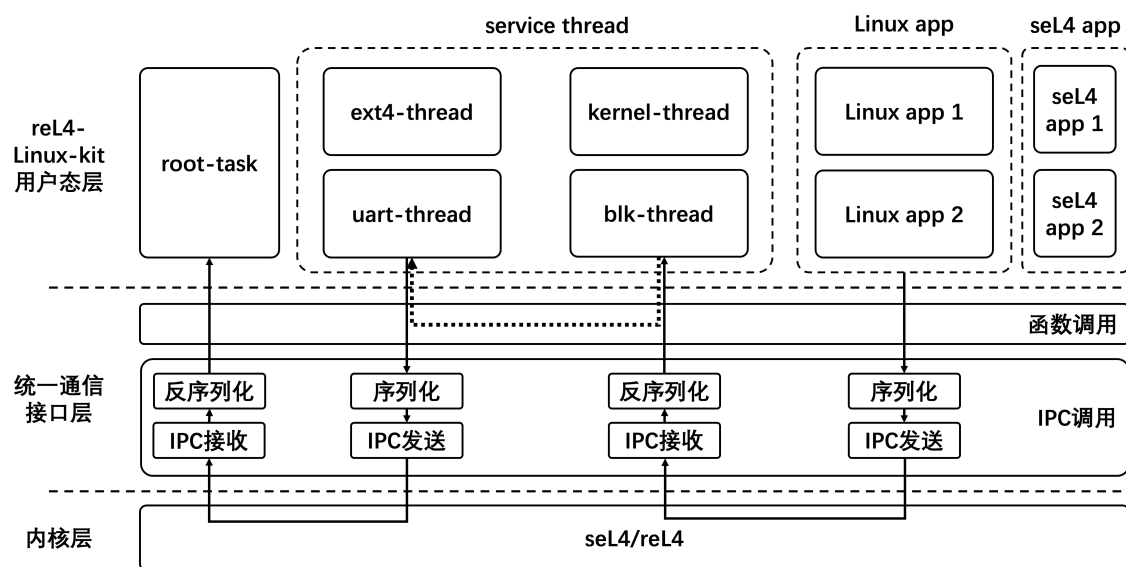


图 4.1 reL4-Linux-kit 架构图

这些在组件化内核部分详述过的组件化 Rust 工具，同样可以用于各类系统服务组件的开发。

4.2 root-task 与用户态进程管理

root-task 是内核启动后运行的第一个进程，是系统启动和资源管理上的核心程序。

首先，root-task 需要启动系统中的其他系统服务组件，但不启动所有的 Linux app，Linux app 由系统服务组件中的 kernel thread 启动。在此之前需要设计好对应的配置文件，在 root-task 启动执行的时候，解析该配置文件，并根据该配置文件给出的系统配置信息，启动所需的相关进程，系统配置文件相关信息在第 4.4 节被描述。

其次，在微内核启动之后，整个内核将具有最大权限的 capability 集合都交给了 root-task。因此 root-task 在启动其他进程的时候，需要对权限的派生等工作进行管理，以保证其他进程都获取合理的权限。

第三，root-task 需要为其他进程间的通信建立合适的端口。正如前文所述，整个系统被拆分为多个系统服务组件进程，在各个相互依赖的服务组件之间，Linux 用户态进程和服务组件之间，都需要频繁地进行通信。而在 seL4 下，进程间通信都需要通过内核获得对应的 Endpoint capability，因此，reL4-Linux-kit 要求 root-task 必须在创建各个进程时提供自己的 Endpoint。各个进程可以通过该 Endpoint 向 root-task 查询并申请需要建立通信的目标进程的 Endpoint，并基于申请到的 Endpoint 进行更进一步的通信。

第四，root-task 需要为其他进程提供基本的资源申请功能。在以 Linux 为代表的宏内核中，当一个进程需要申请一定的资源，会被内核接管并按照一定的策略进行处理，但是在微内核下，相关的资源申请只会导致内核将一个资源申请消息发送给 root-task。

4.3 Linux 用户态程序的二进制兼容机制

为了兼容 Linux 下的 app，如图 4.1 中所示，reL4-Linux-kit 在用户态维护了一系列 service threads。在 reL4-Linux-kit 中，Linux 下编译的 app 运行过程中，存在三种可能与 service threads 通信的机制：1、app 主动调用 syscall 指令，向 service threads 进行通信，2、app 在运行过程中触发一些错误行为，例如 page fault 等，需要 service threads 进行处理，3、app 在运行过程中发生了一些外部的硬件事件，这些硬件事件都需要 service threads 截获并处理。这 3 类通信机制在宏内核下都表现为 trap 入内核，可以称之为宏内核下的 trap 操作。

这些在宏内核下的 trap 操作，在 reL4 下被转换为 IPC 操作，向对应的 service threads 进行通信，再由一系列的 service threads 按照 Linux 下这三类 trap 操作的行为进行处理并返回。为了完成这一步兼容，需要首先进行一定的改造，完成 trap 操作转 IPC 的操作。

二进制兼容的目的是将各类 trap 进入内核的通信行为，转为一个向 service threads 发送消息的 IPC 操作。但该操作的实现有不同的实现方案，reL4-Linux-kit 的这一实现也经过了多轮的迭代。

4.3.1 app 主动调用 syscall 的处理

reL4-Linux-kit 最开始选择重新编译 Linux 应用程序源码的方式，具体而言，reL4-Linux-kit 设置了一个 syscall 的地址，在编译时，将 syscall 改写为一个跳转指令，用户态 service threads 在加载时，会修改其跳转位置，将它指向一个特定的 vsyscall 位置，该位置存放了发送 IPC 的代码，在 app 运行时，会跳转到对应的 vsyscall 位置并发送 IPC。

该方案存在局限，首先需要获取对应的源代码，无法直接使用现有的 Linux 二进制文件，实用性受到影响，其次还需要放入对应的 IPC 代码并修改对应的跳转地址，最后，在不同平台还存在兼容性问题，例如某些定长指令集跳转范围的问题，比如 aarch64 的 bl 指令跳转范围为前后 128Mb，而某些程序的存储地址可能超过这个范围，就难以再跳转了，在 x86-64 等变长指令集下改写二进制，也存在指令长度问题。

reL4-Linux-kit 的现有方案则充分利用了 reL4 内核的设计，如图4.2中所示。

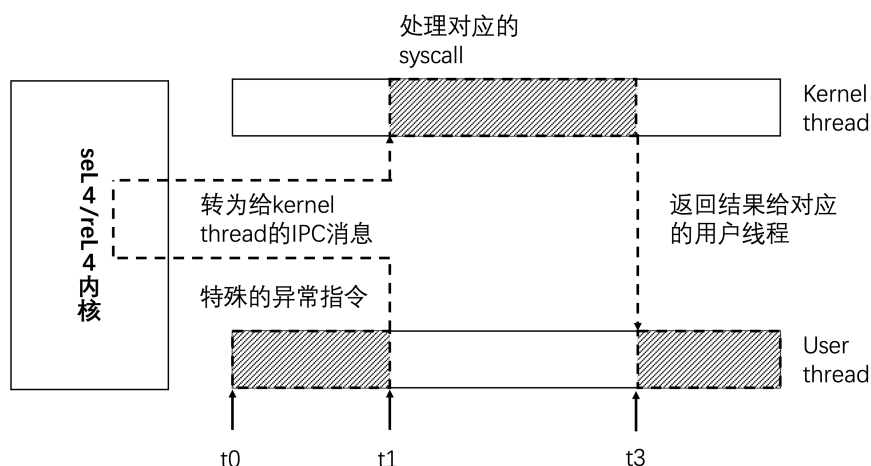


图 4.2 reL4-Linux-kit 下 syscall 转为 IPC 方案

在 reL4 中存在 fault handler，其中就包括了在读取一个异常指令时，发送对应的 fault 消息给设定的 fault handler。reL4-Linux-kit 设置异常指令执行时的 fault handler，并在运行之前，使用 python 脚本将触发 syscall 的 svc 指令替换为一个不存在对应指令的 0xdeadbeef。当用户态程序运行时，运行到该不存在的指令时，会通过内核发送一个 fault 消息给对应的 fault handler 函数，在该处判定是否是一个 syscall 转写为 0xdeadbeef 或者一条真正的异常指令，如果是一个 syscall 转写的，就处理对应的 syscall。

该方案具有很多优点。首先该方案无需重新编译原有二进制程序，也无需获取对应的源代码，同时，充分地利用了 reL4 内核提供的机制，也有一定的兼容性优势。除此之外，该方案也可以进一步地升级为，在加载时进行 syscall 指令的替换。

但该方案也存在一些问题，例如，某些程序会使用代码签名技术，在运行时进行验证确保代码未经过恶意平台进行二进制改写，该方案由于本身机制的原因，无法解决该问题。除了上述方案之外也有别的方法将 syscall 转为 IPC，比如改写用户态程序依赖的库函数等。这些方案也都各有优缺点。

4.3.2 app 在运行过程中触发错误行为

在通用的 Linux 宏内核 app 运行过程中，会因为各类的不合法行为触发错误并陷入 Linux 内核，在 reL4-Linux-kit 的模型中，这种不合法行为触发错误并陷入的操作，最终会转变为向 service threads 的通信行为，如图4.3中给出了 page fault 的处理过程。整个处理过程和前一小节中所述的 syscall 的处理没有本质上的不同，因为 syscall 也是转变为一个非法指令的错误来处理的，故在此不做详述。

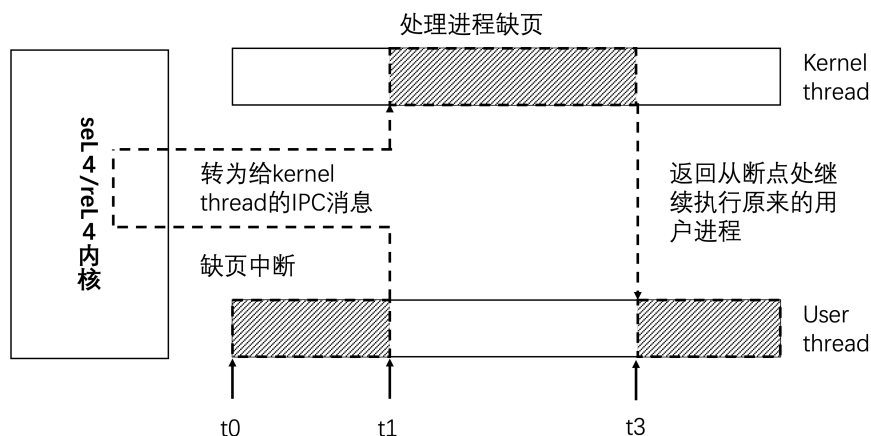


图 4.3 reL4-Linux-kit 下 page fault 转为 IPC 方案

4.3.3 app 运行过程中外部设备发送消息的行为

app 运行过程中，外部设备会产生各类中断，这些中断信号都是异步的，不可预知的。

在 reL4 微内核中，这些中断触发时，会按照微内核的处理流程进入到内核态中，并向在 service threads 中指定的 Notification 目标发送一个信号，该信号经过 service threads 处理之后，如有必要，会投递给原先被中断的 app，正如图4.4中所示。

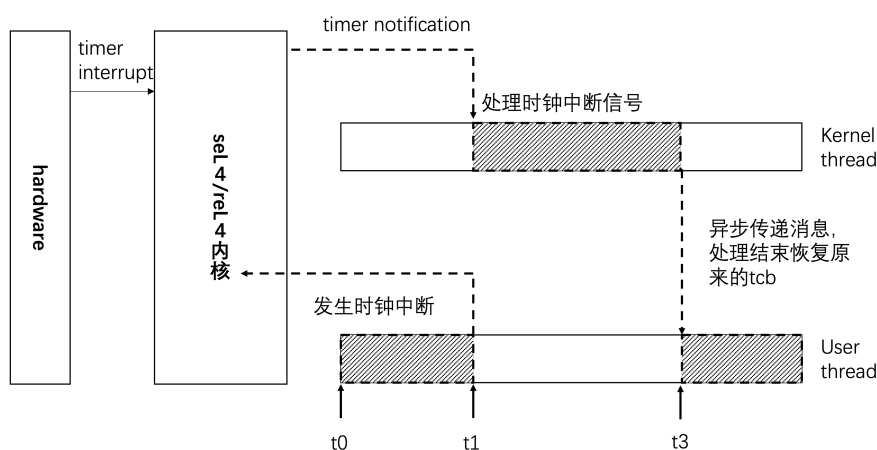


图 4.4 reL4-Linux-kit 下 timer 信号转为 IPC 方案

4.3.4 reL4-Linux-kit 和其他 Linux 兼容方案的区分

我们的原生方案和已有的其他支持 Linux 的微内核实现方案也有本质性的差别。

以 L4Linux 为代表的技术路线^[56]，是将一个完整的 Linux 内核通过一些裁剪、改写等手段，直接作为一个用户态的进程运行在用户态，该方案的优点在于，较为

完全地兼容 Linux 意味着微内核具备了良好的通用性, 但该方案的问题在于, Linux 本身由于宏内核过于复杂带来的安全性问题, 并没有因为使用了微内核作为基础而得到缓解, 同时, 把内核放在用户态反而还引入了更多的性能开销。与之相对的, reL4-Linux-kit 一方面用 Rust 语言进行了从头开始的构建而非简单的拼接, 同时实现了多个系统组件, 设计了组件间的通信机制, 而不是集中式地把他们塞到一个单独的内核进程中, 是一个更为符合微内核原生生态且兼顾安全性的方案, 这不仅提升了性能, 更从根本上将宏内核的复杂性与潜在的不安全性进行了隔离和封装。

除此之外, hypervisor 的方案, 则是利用 seL4 提供的 hypervisor 的功能, 在其之上再封装一个虚拟化层, 在该虚拟化层中运行操作系统, 或者把操作系统的多个组件分散在多个虚拟机之间, 通过虚拟机之间的通信来进行协作, 而 reL4-Linux-kit 试图在 seL4 二进制兼容的 reL4 内核上, 则直接去除了这层虚拟化层的封装, 试图提供更贴近底层的方案, 这避免了虚拟化的巨大开销, 在一定程度上对性能的提升具有正面作用。

4.4 函数调用和 IPC 通信的透明转换机制

在微内核下, 用户态的进程包括了用户程序和服务组件 (service components) 集合, 如图 4.1 中所示。但是服务组件提供功能的复杂化和细粒度化, 导致了在系统中会存在规模庞大的服务组件集合, 以及大规模的服务组件与用户程序之间、服务组件与服务组件之间的 IPC 通信, 这对微内核的性能提出了挑战。一种直观的优化策略是, 在适当场景下将多个细粒度的系统服务组件合并为单个系统服务组件, 从而将部分跨进程的 IPC 交互转换为同一地址空间内的函数调用, 降低通信开销。

另一方面, 为了利用已有的 Rust 编写的组件化宏内核的成果, 而非从头再编写一套代码, 我们将宏内核的组件改造成为独立的系统服务组件在微内核环境中运行。这就要求将原有组件间基于函数调用的通信机制, 适配为微内核下的 IPC 通信模式, 从而在最大限度保留原有实现的基础上, 将宏内核组件快速迁移至微内核环境。

基于上述动机, 我们提出了一种函数调用与 IPC 通信双向透明转换机制构想, 如图 4.5 所示。该机制提供统一的集成框架, 支持将宏内核组件中基于函数调用的通信快速转换为符合微内核规范的 IPC 通信模式; 同时, 在强调性能的场景中, 亦允许将经过这种转换改造, 或者原生采用该机制的多个通过 IPC 交互的服务组件, 合并至同一组件中执行, 并退化为函数调用以实现高效协作。尽管该机制在概念

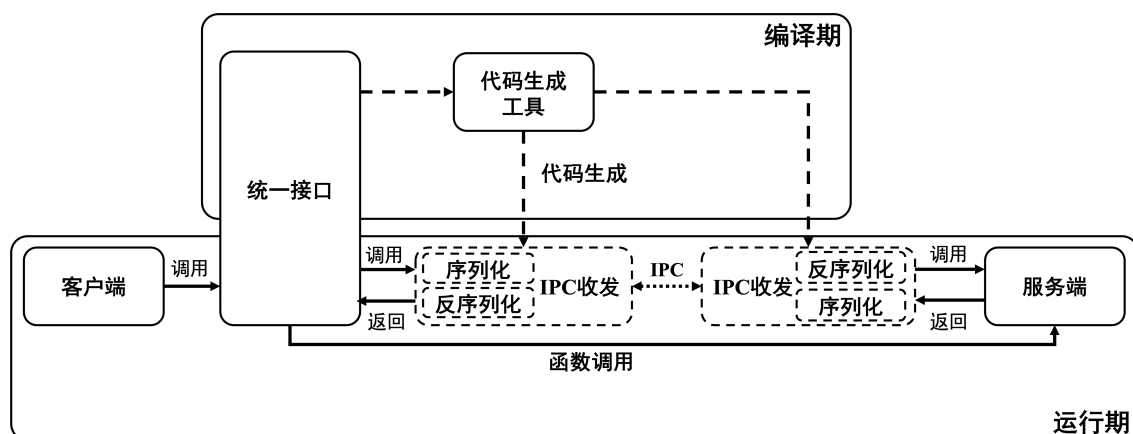


图 4.5 reL4-Linux-kit 的 IPC 和函数调用双向透明转换机制

上清晰直观，但其实现面临多方面挑战。为此，reL4-Linux-kit 提供了一整套系统化的解决方案以达成这一目标。

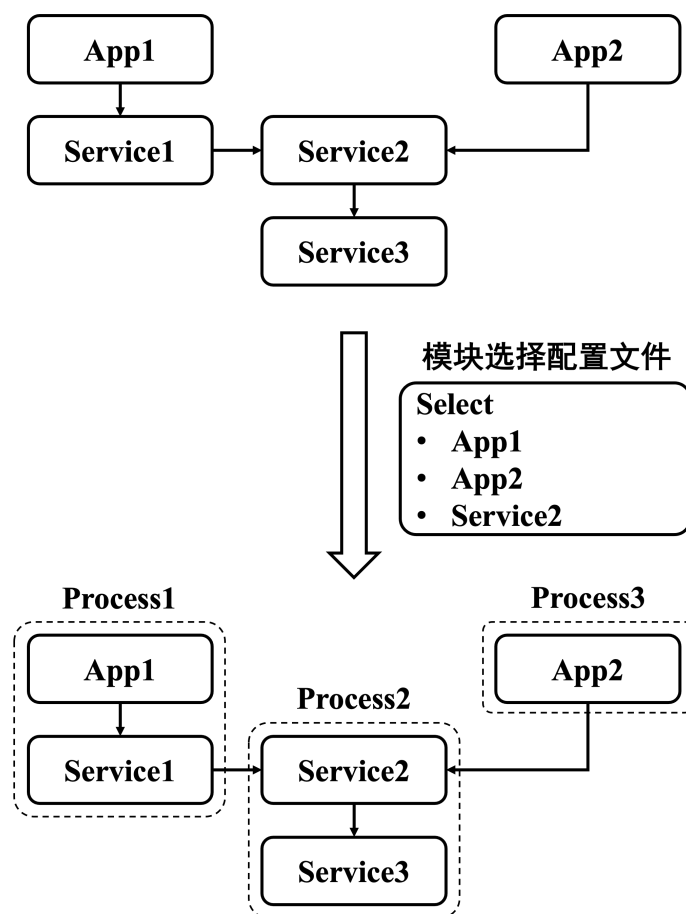


图 4.6 reL4-Linux-kit 的组件组合方案

该套方案一方面需要在编译期实现如图 4.1 中所示的 IPC 和函数调用的转换，这在 4.4.3 节描述，另一方面也需要处理好组件间的不同地址空间关系，如图 4.6 中所示，需要根据编译时的输入，决定不同组件在不同进程中的安排，同时将相应的

信息用于图 4.5 中的代码生成，详细的方案在 4.4.1 节、4.4.2 节、4.4.4 节中描述。

4.4.1 Rust 语言条件下函数调用和 IPC 调用两套代码并存的方案

为了能够同时兼容函数调用和 IPC，首先需要对每个组件做出符合 Rust 语言规范的改造。

在 Rust 中的一个项目，存在多种编译目标，一种是最终编译成为单个可执行文件，其入口地址为 `main.rs` 文件所指定的代码入口，而另一种则是编译成为一个库文件并和其他代码进行链接，通过 `lib.rs` 文件来指明其所提供的接口。在这个灵活转换的需求下，不同需求的 Rust 代码的编译存在一定的冲突：在合并为一个 `thread` 的情况下，只能存在一个顶层模块拥有 `main.rs` 文件，其他的模块都应当以库代码的形式和该顶层模块进行链接；但是，如果改为多个 `thread`，每个 `thread` 都对应一个可执行文件，则必须存在多个顶层模块，每个 `thread` 都对应一个 `main.rs` 文件，否则就会和 Rust 的模块管理系统存在冲突。

因此，我们使用了 `lib.rs` 和 `main.rs` 文件并存的方案，以便在需要时选择函数调用或者 IPC 发送。在 `lib.rs` 中提供函数调用接口，这主要通过各类 `trait` 声明的函数接口来实现；而在 `main.rs` 中提供 IPC 调用逻辑，则主要通过入口函数进入，并通过 `while` 循环持续接收消息并主动调用相关逻辑来实现。

对于顶层模块的选择，如图 4.6 中所示，当用户在编译输入中选择了 `App1`，`App2`，`Service2` 作为顶层组件时，这三个组件的 `main.rs` 会被使用，这决定了 `App1`，`App2`，`Service2` 作为该进程中的唯一顶层模块，而他们依赖且没有作为其他顶层模块的组件，则会使用库代码的方式和这几个顶层模块进行链接，相应的，他们依赖的已经作为其他进程中顶层模块的组件，如 `Service2` 组件，被 `Service1` 和 `App2` 同时依赖，但是由于已经指定其作为某个进程的顶层模块，因此必须以 IPC 的方式在 `Service1` 和 `App2` 中进行调用发送。详细地根据依赖关系和编译输入中的选择决定最终的配置的方案在 4.4.2 节详述。

对于组件化宏内核中的各类组件而言，如需支持 IPC 通信，只需要仿照类似形式：在调整好对应的 `Cargo.toml` 的同时，增加 `main.rs`，并在其中使用 `while` 循环不断接收消息，将其转换为对组件接口的调用逻辑，即可完成封装。需要注意的是，通过 `while` 循环接收的消息需要涵盖该组件对外提供的所有接口。而使用函数调用方式合并多个组件的代码则相对简单，与在 Rust 中正常使用模块的逻辑基本一致。

4.4.2 灵活可配置的组件间函数调用或 IPC 调用选择方案

其次，该系统采用了一种可配置的构建方案，需要明确定义各个服务组件（service components）间的配置与依赖关系。具体而言，reL4-Linux-kit 通过一个文件来描述每个组件的资源持有与依赖关系，并利用一个图算法脚本，根据用户输入的参数动态决定哪些组件应被合并。

最终，完整的系统配置信息被拆分为不同的部分，分别用于构建系统和运行时系统：一部分提供给 reL4-Linux-kit 的构建系统，其中包含组件的资源信息，以便在构建时通过函数调用的代码路径将多个组件合并编译；另一部分则提供给根任务（root-task）运行时系统，其中仅包含组件的查找信息，用于在启动时了解并启动所有独立的服务组件。

仍然以图 4.6 中的案例和 4.4.1 节中的分析为例，在输入了配置选择 App1, App2, Service2 作为顶层模块后，在 App1 和 Service1 之间，Service2 和 Service3 之间需要通过函数调用进行调用，而 Service1 和 Service2 之间，App2 和 Service2 之间，通过 IPC 进行调用。这些最终的配置信息以及其他各个组件拥有的资源信息等内容需要传递给编译期和运行期的代码，以便编译出图中的 Process1-3 并在运行期启动这些进程。

对于配置信息文件而言，由于 Rust 的 Cargo 包管理器使用 TOML 文件来定义项目的元数据和依赖关系，因此我们也选择使用 TOML 文件来进行系统配置信息的描述。

在 TOML 文件中，想要声明一个组件，在开头是一个 [[task]] 标识符，随后可能跟着若干个字段来描述系统信息：

表 4.1 TOML 中可配置信息列表

属性	描述
name	指定组件的名称，在单独存在的时候会用来让 root-task 标识
file	指定组件的可执行文件，当组件单独存在的时候，root-task 引入这个文件。这个文件需要在 target 目录下
mem	组件需要的设备内存，当组件独立存在的时候会被映射到当前地址空间，组件被合并的时候会归入最后的顶层模块中，格式为：[被映射到的地址，设备地址，大小]
dma	组件需要的内存，同 mem，可以被继承，格式为：[被映射的地址，大小]，dma 地址可以为任意可用内存，只需要保证连续即可

续表 4.1 TOML 中可配置信息列表

属性	描述
deps	组件的依赖关系，描述当前组件所依赖的其他组件，当组件没有选择独立的时候可以继承相应的资源
cfg	组件被选中作为独立组件的时候，会为 rust 构建脚本添加一个 <code>cfg</code> ，并且可以在源代码中使用 <code>#[cfg(xxx)]</code> 的形式进行条件判断，类似于 <code>feature</code> 的形式

通过 TOML 文件中的 `deps` 等属性，即可构成一个组件与组件间的依赖关系，而用户在构建时的输入则会根据需要灵活地指明需要哪些独立的组件，此时则需要根据图算法脚本来最终确定到底哪些程序需要作为一个独立的组件进程来运行，而哪些程序需要合并。一方面，需要能够检测出来给定的系统描述文件中没有环的存在。另一方面，用户在构建系统时指定的参数可能未必代表了最终需要独立的组件，这需要根据系统的配置信息和用户输入综合分析。

解析程序的核心功能是一个简单的图解析算法，首先使用基本的图遍历算法来确保 TOML 描述文件中没有环，用户所选择的组件和该组件依赖到的组件，入度会增加一，如果在对一个组件的依赖组件进行入度增加一处理的时候发现其已经独立了，那么这个增加一的操作不会再对这个已经独立的组件的依赖组件进行处理。最后会统计入度大于一的组件的数量，并将其作为单独的进程运行，并将这些依赖的资源转移到该组件上。所有入度大于等于一的组件就是在系统中使用的独立组件。

下面用一个简单的案例来描述组件的独立和资源的继承：现有三个组件 A, B, C，且 A 和 B 都依赖 C。A, B 互不依赖。当用户仅选择 A 的时候，A, C 就会绑定在一起编译，A 中继承了 C 的资源和信息；当同时选择 A, C 的时候，就认为 A, C 独立运行，它们之间通过 IPC 进行通信；如果选择 A, B 那么由于 A, B 都依赖 C，且 C 的资源无法复制，那么就会产生三个独立的进程 A, B, C。

目前，该图算法脚本使用一个 python 脚本来编写，但这仅是一个临时性的解决方案，未来可能进一步地使用 Rust 脚本重构。一方面，TOML 文件是 Rust 生态中最广泛使用的脚本文件，其格式解析目前已有成熟且被广泛应用的库，使用 python 而非 Rust 来完成这一步骤，对整个项目的编程语言一致性存在影响，另一方面，目前使用 Python 编写 TOML 文件解析需要使用 `tomllib` 库，而该库依赖 3.11 或者更高版本的 Python，但部分测试环境中使用的 Ubuntu 22.04 默认的最高 Python 支持版本为 3.10，这意味着在这些测试环境下还需要额外升级 Python，这带来了更多的兼容性负担。

4.4.3 函数调用和 IPC 调用的消息传递一致性解决方案

在操作系统架构中，函数调用通常借助栈来传递参数，并通过寄存器返回结果；而在微内核环境中，进程间通信（IPC）则依赖于共享内存或消息寄存器（如 seL4 的 IPCBuffer）进行数据传递。为实现函数调用与 IPC 之间的透明转换，必须将函数参数与返回值进行序列化与反序列化，使其能够通过消息进行传递。本文借助 Rust 生态中的 zerocopy 库，实现了参数与返回值的零拷贝序列化，从而在保证类型安全的前提下，高效地将栈基参数传递转换为可通过 IPC 传输的字节序列。

系统通过过程宏（proc-macro）自动分析函数签名，识别参数与返回值的类型，并据此生成相应的序列化与反序列化代码。具体而言，对于基本数值类型（如整数类型），系统直接将其存入消息寄存器，利用 as_bytes() 方法实现零拷贝转换；对于复杂类型（如结构体或字符串），则将其序列化为字节流，并通过共享内存区域进行传输。

如图 4.5 中所示，如果选择 IPC 的消息传递方式，在编译阶段，会根据各个模块的源代码和过程宏实现的统一接口，生成对应的序列化和反序列化代码，并嵌入到对应的 IPC 消息传递代码中。在运行时，会执行自动生成的序列化和反序列化代码并进行消息传递过程。

代码 4.1 IPC 的参数传递过程

```

1 pub fn parse_arg(pat: &PatType) -> proc_macro2::
    TokenStream {
2     let name = pat.pat.clone();
3     let pat_ty = get_type(&pat.ty.clone());
4     match pat_ty {
5         ReturnTypeEnum::Number => quote! {
6             ib.msg_regs_mut()[reg_len] = #name as u64;
7             reg_len += 1;
8         },
9         _ => quote! {
10             let bytes = #name.as_bytes();
11             ib.msg_regs_mut()[reg_len] = bytes.len()
                as u64;
12             let offset = (reg_len + 1) * size_of::(<
                sel4::Word>());
13             ib.msg_bytes_mut()[offset..offset + bytes.

```

```

len()).copy_from_slice(bytes);
14     reg_len += bytes.len().div_ceil(size_of::<
        sel4::Word>()) + 1;
15 },
16 }
17 }
18
19 pub fn parse_return(ret_ty: &ReturnType) ->
    proc_macro2::TokenStream {
20     match ret_ty {
21         syn::ReturnType::Default => quote! {},
22         syn::ReturnType::Type(_, ty) => {
23             let ty = get_type(ty);
24             match ty {
25                 ReturnTypeEnum::Number => quote! {
26                     sel4::with_ipc_buffer_mut(|ib| ib.
                        msg_regs()[0] as _)
27                 },
28                 ReturnTypeEnum::MessageInfo => quote!
                    {ret},
29                 ReturnTypeEnum::Others => quote! {
                    todo!("Not_support_non-
                        numeric_return_type") },
30             }
31         }
32     }
33 }

```

如上述代码所示，系统根据参数类型分别生成不同的处理逻辑：数值类型直接存入寄存器，而非数值类型则通过计算偏移量写入共享内存缓冲区，同时在寄存器中记录其长度信息。返回值的处理机制类似，根据类型信息自动选择从消息寄存器或字节缓冲区中反序列化结果。

4.4.4 reL4 下的 IPC 调用适配方案

除了上述的消息透明转换封装方案之外，还需要考虑 seL4 下的通信目的地发现。在使用函数调用方式进行通信时，跳转到对应的目标函数是通过链接器定位对应的函数偏移，并进行跳转，但使用 seL4 下的 IPC 通信方式时，需要传入对应的 Endpoint 信息。reL4-Linux-kit 选择在创建进程的时候就给出 root-task 的 Endpoint，并由 root-task 记录每个被其启动的 service components 的 Endpoint，在某个 service components 需要知道其他某个 service components 的 Endpoint 时，发送请求消息给 root-task 并由 root-task 发送对应的 Endpoint 给请求者。最终，根据 Endpoint 信息，调用如下的代码发送消息。

代码 4.2 灵活选择 IPC 方式

```
1 let call_stat = if is_impl {
2     quote!(self.ep.call(msg))
3 } else {
4     quote!(call_ep!(msg))
5 };
```

4.5 用户态核心服务组件

reL4-Linux-kit 的用户态服务组件是其兼容 Linux 生态的核心。这些组件被设计为独立的、权限最小的进程，通过高效的 IPC 机制协同工作，也可以通过上述的透明转换机制，和其他组件进行合并。所有模块均使用 Rust 实现，不仅确保了自身的内存安全，也通过严格的类型系统保障了进程间消息的合法性，从整体上构建了一个高可信的用户态环境。

4.5.1 kernel thread

kernel thread 是 reL4-Linux-kit 在运行时的核心程序，其承担了多项复杂功能，超越了简单的 Linux syscall 分发者的角色，承担了构建完整的 Linux 应用运行时环境的职责。

- Linux 应用生命周期管理:kernel thread 是所有 Linux 用户态应用的创建者与管理者。与由 root-task 启动的系统服务组件不同，它需要为每个应用维护一套符合 Linux 语义的私有状态，例如，在 service components 中，并没有 fd table 的概念，但是 Linux app 中，则存在相关的 fd table 概念。这要求在 kernel thread 中，除了向下使用 reL4 的进程相关的接口之外，还需要维护额外的

fd table 数据结构用于指明 Linux app 已经使用的文件的 fd 等信息。这些与 Linux 相关的数据结构正是由于其向上对接 Linux app，向下对接 reL4 接口的设计目标所决定的，在 syscall 处理相关的流程中，是无可避免的，因此，必须放在 kernel thread 而不是其它系统服务组件中。这一设计决策至关重要：它将状态密集的、Linux 特有的逻辑集中于此，避免了将其复杂性渗透到下层更通用的服务组件（如文件系统）中，从而保持了底层服务的简洁性和纯洁性。

- 系统调用的拦截与分发：kernel thread 是整个 syscall 的核心，具体而言，在 kernel thread 进程中，需要查看产生错误的 fault 的原因是否为 0xdeadbeef 指令，来判断是否是来自于 Linux app 的一个系统调用，如果是，则调用 handle_syscall 进行进一步的系统调用的分发，并更进一步地进行例如向 fs thread 发送消息等操作，在此过程中，它扮演了系统调用调度中枢的角色。
- 轻量级并发模型：对于传统的宏内核操作系统来说，一个 Linux app 必然在内核态存在一个对应的内核线程，但是在微内核的语境下，这样的内核线程意味着更为巨大的开销，因此，reL4-Linux-kit 利用 Rust 语言的无栈协程机制和 rust-seL4 库中提供的 seL4 下的 single-threaded-executor 等工具，用每个协程来对应 Linux 下的每个 app，从而在极低的开销下实现了大量应用的并发运行，这是对微内核进程模型开销问题的一个优雅解决方案。
- 错误处理：kernel thread 还处理其子进程（Linux apps）产生的其他异常（如缺页错误），在 kernel thread 错误处理的流程中，如果不为 0xdeadbeef（即 syscall），就需要在 kernel thread 中具体查看错误的原因并进行处理。目前 reL4-Linux-kit 出于原型验证和必要性的考虑，主要处理虚拟机管理错误（VMFault），其余错误均被视为非法。这也体现了微内核的另一个优势：错误处理策略可在用户态灵活定义和更新。

综上所述，kernel thread 的精心设计在满足 Linux 兼容性必要需求的同时，最大限度地实现了系统服务的组合与解耦，并通过协程异步的并发模型提供了高效的支持。

4.5.2 其他系统服务组件

在文件系统方面，reL4-Linux-kit 目前已经实现了基本的 ext4 文件系统的支持。得益于 Rust 生态众多的库，reL4-Linux-kit 目前对 ext4 的支持，依赖于现有的 lwext4 库，同时使用了 Rust 的 trait 特性，将 fs 的操作抽象为一套 FSIface 的 trait 操作，提供了基本的 open, read, write, mkdir, close 等操作。在运行时，使用一个 loop 循环，不断地查看发送给 fs thread 的消息，并做相应的分发，调用 lwext4

提供的接口来完成需要的文件系统的操作。

在设备驱动程序方面，reL4-Linux-kit 已经实现了 blk thread 和 uart thread 两个驱动相关的 service components。与 fs thread 相同，Rust 为此提供了基本的库支持，blk thread 直接使用 QEMU 模拟器环境下的 virtio drivers 驱动库，而 uart thread 则依赖于 ARM 下的 pl011 串口驱动库，并将这两个驱动 thread 抽象为包含 readblock 和 writeblock 等基本块设备操作的 BlockIface 以及包含 putchar 和 getchar 等基本流设备操作的 UartIface。其在运行时也同样不停地循环查看发送给这些组件的消息并分发，并调用相应已经实现的驱动的接口。

相比于承担复杂状态管理职责的 kernel thread，文件系统与驱动等服务组件的设计遵循了单一职责原则，其内部逻辑保持简洁和专注。这种设计不仅使得架构清晰明了，也因其功能的内聚性而更容易保证正确性，并为通过第 4.4 节机制进行灵活合并提供了便利，从而在整体上提升了系统性能与可维护性。

4.6 reL4-Linux-kit 的运行样例分析：以文件系统服务为例

为了能够更详细地描述在 reL4-Linux-kit 的工作原理，在本节我们给出了一个文件系统操作的样例，如图 4.7 所示。

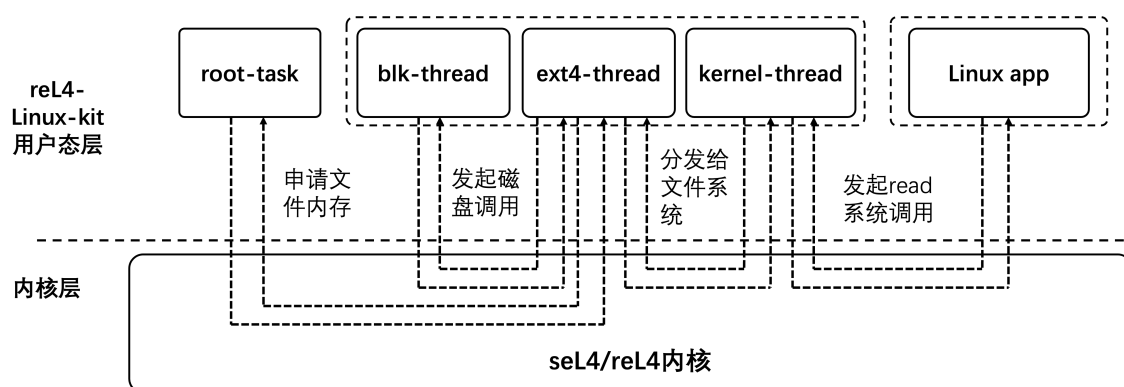


图 4.7 reL4-Linux-kit 示例

正如前文所述，为了完成宏内核的功能，在用户态维护了若干服务线程，以完成传统的 Linux 内核的功能。其中与 Linux 文件操作相关的组件，包括了 kernel 组件，文件系统组件，磁盘设备驱动组件。

依照 4.4 节中的内容，这些组件在用户态，可以按照 IPC 的方式来进行组件间调用，也可以使用 function call 的方式来进行组件间调用。前者需要将 kernel 组件/文件系统组件/磁盘设备驱动组件拆分到三个不同的 threads 中，并将这些 threads 放在同一个地址空间里。后者则将文件系统组件和磁盘设备驱动组件合并入 kernel 组件所在的 kernel thread 中。也可以按照前文所述，选择其中两个组件组合，合并

入一个 thread，而另一个组件单独在另一个 thread 中的方式，并以此选择各个组件间的调用方式为 IPC call 或者 function call。图中表示了拆分到三个不同的 threads 的方式。

常见的 Linux 文件系统相关 syscall 操作包括 open/read/write 等，以下以 read 操作为例。当一个 Linux 进程发起系统调用的时候，根据 4.3 节所述的内容，这会触发一个特殊的异常指令，seL4 内核会根据 kernel 组件预先设定好的 fault handler，向 kernel 组件传递消息。

kernel 组件充当了一个 syscall 处理的中枢，在收到这个表示 syscall 的特殊 fault 消息之后，经过一定的处理后，会通过 IPC 或者 function call 的方式调用对应的文件系统组件接口函数，传入对应的参数。

相应的文件系统组件会完成对这个 read 操作的具体实现。在这个过程中，一方面，需要进一步地与磁盘操作组件进行通信，完成将磁盘上的某几个块读入的操作，另一方面，读入的块需要额外的物理内存进行存放，这也会涉及到申请更多物理内存的操作，因此需要和 root-task 通信，并进一步地向 seL4/reL4 内核发起系统调用，以映射相应的物理内存。

在这些步骤完成后，文件系统组件会将消息返回给 kernel 组件，并由 kernel 组件改写原有的 Linux 进程的 TCB 中的相应寄存器，以将结果传递回该进程。最后切换回原来的 Linux 进程，从而完成了整个发起 read 系统调用的操作。

4.7 本章小结

本章详细阐述了 reL4-Linux-kit 用户态兼容层的设计与实现。我们首先在第 4.1 节提出了以用户态内存安全和生态兼容为核心的设计目标与总体架构。以此为指导，我们系统性地解决了在形式化验证的微内核上构建实用系统所面临的关键技术挑战，并构建了可验证的原型系统。

首先，我们设计了基于 Fault Handler 的 syscall 拦截机制（第 4.3 节），通过巧妙的指令替换实现了对 Linux 二进制程序的零修改兼容，无需源码即可运行现有应用。

其次，我们提出了一个创新的、可配置的函数调用与 IPC 透明转换框架（第 4.4 节）。该框架包含配置描述语言、图合并算法及基于过程宏的零拷贝序列化机制，使得系统能够在性能与隔离性之间进行灵活的权衡，并有效地便利了已有组件化宏内核代码的迁移与复用。

第三，我们基于 Rust 语言实现了一系列核心用户态服务组件（第 4.5 节），包括 kernel thread、文件系统和驱动服务。这些模块通过清晰的 trait 接口进行抽象，

通过 IPC 进行协作，共同构建了一个内存安全、故障隔离的 Linux 兼容运行时环境。

最后，我们给出了一个在 reL4-Linux-kit 框架下，实现文件系统功能的样例分析，用更为具体的描述描写了，在 reL4-Linux-kit 下一个具体的系统服务是如何运行的。

综上所述，reL4-Linux-kit 不仅仅是一个简单的兼容层，它提供了一个完整的、安全的、可用于构建基于微内核的通用操作系统的用户态框架。我们的工作表明，通过巧妙地结合 seL4/reL4 微内核的强大特性与 Rust 语言的内存安全优势，能够有效地应对用户态系统的安全性及兼容性挑战。

我们未来的工作将围绕以下几个方面展开：优化 IPC 性能、扩展对更多 Linux syscall 和驱动类型的支持、复用更多的宏内核组件以及对该用户态框架本身进行形式化建模与验证等，以期构建一个从内核到应用全栈可信的高安全操作系统。

第 5 章 实现与评估

本章旨在描述 reL4 微内核和 reL4-Linux-kit 兼容层的实现情况，并进行系统性的评估。该评估工作主要围绕两部分展开：一是对系统功能完整性的验证，二是对系统综合性能的评测。本节将完成以下测试：

- reL4 内核与 seL4 内核的二进制兼容情况分析
- reL4-Linux-kit 与部分关键 Linux syscall 的兼容性情况分析
- reL4-Linux-kit 对调用了相关 Linux syscall 的测例集合的运行情况分析
- 基于 seL4 或者 reL4 的 reL4-Linux-kit 的运行情况的综合性能评估

通过功能与性能两个维度的评估，系统检验本研究是否达成既定目标，并为后续的优化与深入研究提供依据。

reL4 内核目前支持 riscv64 和 aarch64 两个架构。考虑到廖东海^{[62][63]}等人实现的 reL4 早期版本已经实现了对 riscv64 架构的基本支持，而本文的工作主要是在其基础上进行组件化调整，并进一步支持 aarch64 架构与诸多特性，同时实现用户态的 Linux 兼容，因此，对 riscv64 架构的 reL4 内核进行过度测试并无必要。基于此，本文对测评环节作出一定调整。

具体来说，对系统功能完整性的验证部分，本文使用 seL4 的 seL4test 测试集合测试 riscv64 和 aarch64 架构的 reL4 内核，以验证其对之前工作的完整继承性，以及对 seL4 内核功能的关键支持。

在后续的性能测试阶段，由于已经基本验证了其实现和早期版本 reL4 的兼容，相关性能测试内容在李龙昊^[61]和廖东海^{[62][63]}等人的工作中已给出较多对比，且组件化的调整工作未引入显著的性能差异，因此本文的性能测试工作不再对 riscv64 架构下的 reL4 内核进行过多的测试，而主要着重于展现并分析 aarch64 架构的 reL4 实现的性能测评。考虑到这一因素，对于 Linux 兼容层的功能完整性验证和综合性能测试，也同样基于 aarch64 架构来展现，这样，既可以较为完整地展现整个系统实现的性能，又避免过度冗余。

5.1 系统实现情况

本节详细描述了 reL4 组件化微内核和 reL4-Linux-kit 兼容层的实现情况。该项目的代码均可以在 github 组织中找到：<https://github.com/orgs/reL4team2/repositories>，总计包含 36 个仓库，该项目的构建和运行的方法，详见链接：https://reL4team.github.io/zh/docs/quick_start/。

reL4 组件化微内核由于前文所述的组件化以及各类构建工具等工作, 因而目前包含了多个仓库, 其内核主仓库链接为: <https://github.com/reL4team2/reL4-integral>, 总计包含约 2.2 万行 Rust 内核代码。作为对比, 廖东海^[62]^[63]等人实现的早期 reL4 版本中, 代码量约 8900 行的 Rust 代码, 本次组件化重构与功能扩充共产生了 300 余次提交。

reL4-Linux-kit 的仓库链接为<https://github.com/reL4team2/reL4-linux-kit>, 总计代码量约为 7000 行 Rust 代码, 支持 60 多个 Linux 系统调用。

5.2 功能实现评估

5.2.1 reL4 对 seL4 二进制接口的功能实现评估的测试环境

如本章开头所述, 对于 reL4 内核的功能实现测试, 需要同时基于 riscv64 和 aarch64 架构。本节的测评环境基于 QEMU 模拟器, 即便如此, 仍然需要指定相关的 QEMU 参数。相关参数如代码 5.1 所示。

代码 5.1 seL4test 测例的不同架构和参数下的基本 QEMU 模拟器启动命令

```
1 # riscv64 架构下的单核 QEMU 启动命令
2
3 qemu-system-riscv64 \
4     -machine spike \
5     -cpu rv64 \
6     -nographic \
7     -serial mon:stdio \
8     -m size=4095M \
9     -bios none \
10    -kernel /path/to/reL4 kernel \
11    -smp 1
12
13 # aarch64 架构下的单核 QEMU 启动命令
14 qemu-system-aarch64 \
15     -machine virt,virtualization=on \
16     -cpu cortex-a53 \
17     -nographic \
18     -m size=2G \
```

```

19     -kernel /path/to/reL4 kernel \
20     -smp 1

```

上述的 QEMU 命令中，给定了模拟的 CPU 架构，使用的内存大小等模拟环境的基本信息，因此无需再使用文字详细描述其模拟平台的设置。

除此之外，还需要指定相应的测试代码，我们使用的测试代码链接为 <https://github.com/reL4team2/sel4test>，commit 版本为 64e094003bd8629c0f61c7f9daba3034c3dbdb4a，该版本的 sel4test 相比于原生的 sel4test 做了一些针对 reL4 的适配性改动，并跳过了一些难以进行的测试代码，但未进行任何重大改动。

5.2.2 reL4 对 sel4 二进制接口的功能实现评估

reL4 的基本目标为实现一个兼容 sel4 接口的 Rust 内核，因此需要检验其功能的正确性。主要的检测方法为测试通过 sel4 原生测例的个数。

sel4test 是 sel4 官方实现的原生测例，总共有 241 个测例，这些测例根据表和表中的分类，针对 sel4 内核中不同的 syscall 接口进行测试，用于测试内核是否被正确地实现。我们以这些测例作为标准，测试 reL4 的接口是否和 sel4 兼容。

原生的 sel4 的 test 依据不同的 feature，在测试中也会给出不同的测试样例。reL4 中并没有完全实现所有的 feature，例如 reL4 只实现了 aarch64 和 riscv64 两个架构的支持，那么，需要 x86 这一 feature 的测例就无法进行评测。另外在 reL4 内核添加相关 feature 之后，这些 feature 之间具有不同的组合，存在有的 feature 不能兼容，而有的 feature 需要同时出现等情况。经过统计，reL4 现有实现的 feature 下能够支持的测例分类及其通过情况如表 A.1 中所示。

在表 A.1 中列举的测例分类中，部分测例虽然存在但无需测试，相关的原因也在表格中列举，除此之外，reL4 已经支持的 feature 所需的测例均可以正常通过。

在表 5.1 中列举了 reL4 中无需测试的测例，例如 EPT 和 IO Port 特性相关的测例主要只能在 x86 下支持，因此在目前仅支持 aarch64 和 riscv64 的 reL4 中无需通过。其他测例亦类似，多数是由于目前 reL4 项目所涉及的硬件特性不足而无需支持。

表 5.1 reL4 未测试的测例分类及其数量

测例分类	测例个数
Smp Irq	1
SMMU	1

续表 5.1 reL4 未测试的测例分类及其数量

测例分类	测例个数
Stack Alignment	1
EPT	13
Benchmark	1
Breakpoint	7
Break Request	1
Single Step	1
Cache Flush	4
Unknown Syscall	1
IO Ports	1
IOPT	11
vcpu	1
total	44

5.2.3 reL4-Linux-kit 的运行环境

我们采用如代码 5.2 所示的环境配置运行 reL4-Linux-kit。

代码 5.2 reL4-Linux-kit 的 QEMU 模拟器启动命令

```

1 qemu-system-aarch64 \
2     -drive file=mount.img,\
3         if=none,format=raw,id=x0 \
4     -device virtio-blk-device,drive=x0 \
5     -device virtio-net-device,netdev=net0 \
6     -netdev user,id=net0,\
7         hostfwd=tcp::5555-:5555,\
8         hostfwd=udp::5555-:5555 \
9     -machine virt,virtualization=on \
10    -cpu cortex-a57 \
11    -m size=1G \
12    -serial mon:stdio \
13    -nographic \
14    -kernel target/image.elf

```

同样的，该 QEMU 模拟器的启动命令也很好地说明了我们所使用的平台和配置环境，无需再用文字进行说明。

5.2.4 reL4-Linux-kit 对 Linux syscall 的实现评估

reL4-Linux-kit 的设计目标为，兼容现有的部分 Linux 用户态程序。

要兼容 Linux 用户态程序，一方面需要内核支持程序所调用的 Linux syscall，另一方面，一个完整的 Linux 系统生态还应该包括 procfs/sysfs 等虚拟文件系统。这些虚拟文件系统对系统运行的管理，和内核具有同等的重要性。但由于工作量限制，我们目前只考虑对于 Linux syscall 的兼容。

为此，我们选择了若干测例集合作为 Linux 兼容层的性能测试基准。这些测例集合基本只依赖 Linux syscall，而不依赖 procfs/sysfs 等除 syscall 之外的系统环境内容；同时，它们涵盖了一个兼容 Linux 的操作系统内核所需的基础 syscall 功能。我们以这些测例的支持与否，来表明是否完成了微内核下对 Linux 兼容的初步支持。因此，我们对 reL4-Linux-kit 的功能评估主要集中于：为运行这些测例集合而需要兼容的 Linux syscall 数量。

在表 A.2 中列举了 reL4-Linux-kit 在测试中所能支持的 Linux syscall 的数量，这些 syscall 构成了一个兼容 Linux 的系统必要的基础 syscall 集合。根据其功能的不同，可以分为进程管理、内存管理、权限管理、设备控制、时间管理、事件管理、文件系统、系统信息、信号处理、资源管理等 10 类共 69 个具体 syscall。其中部分 syscall 使用了 fake 实现，即仅仅返回 Ok(0)，而不做任何实际的处理，相关信息也在最后一列中列举。

我们选择的相关测例集合，主要包括 basic、run-static、busybox-testcase、lua、iozone、libc、lmbench 等测试集合。在表 A.3 中列举了上述测例集合所使用的 syscall，这些测例集合基本能够涵盖我们实现的 syscall 集合，只有两个已实现的 syscall 并未被这些测例集合使用，这两个测例在表 5.1 中列举。

表 5.2 已实现但未被 Linux 测例使用的系统调用

系统调用名称	类别	是否为 fake 实现
getgid	权限管理	是
clock_gettime	时间管理	否

最后需要指出，在我们的测例集合中，还有部分 syscall 被调用到，但并未在

reL4-Linux-kit 中实现，这些 syscall 及其含义在表 5.3 列举。它们主要集中在网络管理模块。由于 reL4-Linux-kit 并未实现相应网络模块，因此这些 syscall 尚未支持；同时，run-static 测例集合中的 socket 测例也会相应报错。我们忽略这部分报错，其余少数未实现的系统调用因不影响测试核心结果的正确性，故在当前阶段予以保留。

表 5.3 Linux 测例需要但未实现的系统调用

系统调用名称	类别	简要描述	使用的测例组合
exit-group	进程管理	终止进程组中的所有线程	run-static, busybox-testcase, lua, iotest, libc, lmbench
madvise	内存管理	提供内存使用建议	iotest, libc
sysinfo	系统信息	获取系统信息	lua
set-robust-list	进程管理	设置健壮互斥锁列表	run-static
umask	文件系统	设置文件模式创建掩码	lmbench
accept	网络管理	接受套接字上的连接	run-static
bind	网络管理	将名称绑定到套接字	run-static
connect	网络管理	在套接字上发起连接	run-static
getsockname	网络管理	获取套接字名称	run-static
listen	网络管理	监听套接字上的连接	run-static
recvfrom	网络管理	从套接字接收消息	run-static
sendto	网络管理	向套接字发送消息	run-static
setsockopt	网络管理	设置套接字选项	run-static
socket	网络管理	创建通信端点	run-static

基于 seL4 内核与基于 reL4 内核的 reL4-Linux-kit 目前均能够正确完成上述 Linux 测例。该结果进一步表明 reL4 内核在实现上的正确性，即 reL4 内核能够二进制兼容 seL4 内核；同时也表明 reL4-Linux-kit 的现有实现能够正确支持 Linux 程序运行。

5.3 reL4 性能评估

对于 reL4 相关的性能评测，目前 seL4 内核项目已经实现了 seL4bench 的测例支持，用于测试 seL4 的多种性能。为了支持这些测试，必须在 reL4 项目中增加 seL4bench 相关 syscall 和一些硬件环境的支持。

为此，在 3.6 节介绍的用户态时钟的功能之外，还需要进一步地设置 ARM 下的 PMUSERENR_EL0 寄存器的使能位。该寄存器的使能位控制着硬件性能寄存器是否能够在用户态进行读取，这对于 seL4bench 的精确性能评测有着重要意义。

除此之外，seL4bench 项目的部分测试结果，在 QEMU 模拟器的测试环境下无法很好地模拟，例如 L1-cache 等 QEMU 模拟器没有模拟的硬件的性能评估，以及部分性能评测在 seL4bench 中会检验测试的稳定情况，而在 QEMU 模拟器下存在一定程度的时间漂移问题导致难以通过该 benchmark。这些测例相关的处理会在后续小节更进一步地进行说明。

5.3.1 reL4 内核性能评估的测试环境

我们的测试环境仍然如代码 5.1 中所示的 QEMU 模拟器环境和相应的配置。

但是由于涉及到性能测试，因此需要额外说明运行 QEMU 模拟器的硬件性能。我们评测的硬件平台配置如下：

- CPU:AMD Ryzen 9 9950X3D (32) @ 5.756GHz
- OS:Ubuntu 24.04.3 LTS x86-64
- Kernel:Linux 6.14.0-28-generic
- Memory:96GB

5.3.2 One Way IPC 测试

这里的 IPC 相关测试主要指的是单向的 IPC 的性能。seL4bench 的相关测试并没有支持 MCS 特性，因此如图 2.2 中列举的 MCS 相关 syscall 的测试并没有支持。

seL4bench 中单向 IPC 性能测试涉及的 syscall 包括：SYS_CALL、SYS_SEND、SYS_REPLY_RECV 这三个 syscall。在该测试中的相关测试还对比了 IPC 的两个线程是否在同一个地址空间下，如图 5.1 和图 5.2 中的性能测试对比所示。该测试套件同样对比了不同 IPC 消息长度下的差别，如图 5.2 和 5.3 中的性能测试对比所示。

就基本的性能结论而言，这三个测试的表现中，seL4 和 reL4 的性能相近。部分测例 reL4 性能超过 seL4，而部分测例相反。

这种差异一方面可能来源于调用路径上的代码优化情况，即 reL4 在某些调用路径上的代码优化情况不如 seL4，如之前提及的在某些地方的冗余重复检查。另

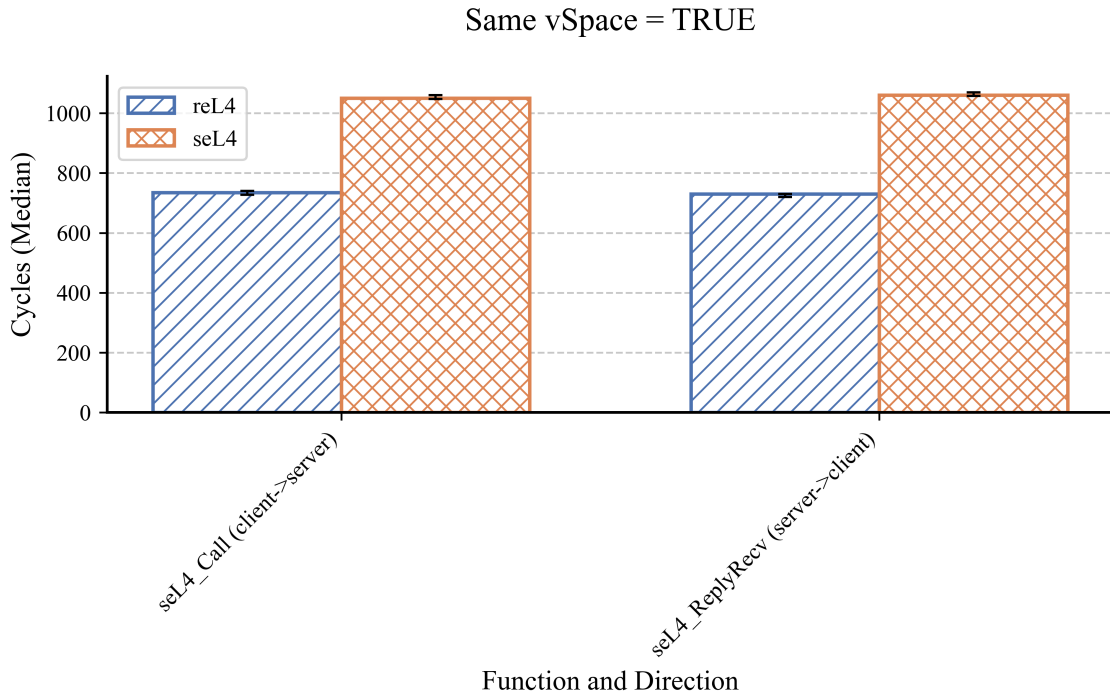


图 5.1 同一地址空间下的 one way ipc 性能对比

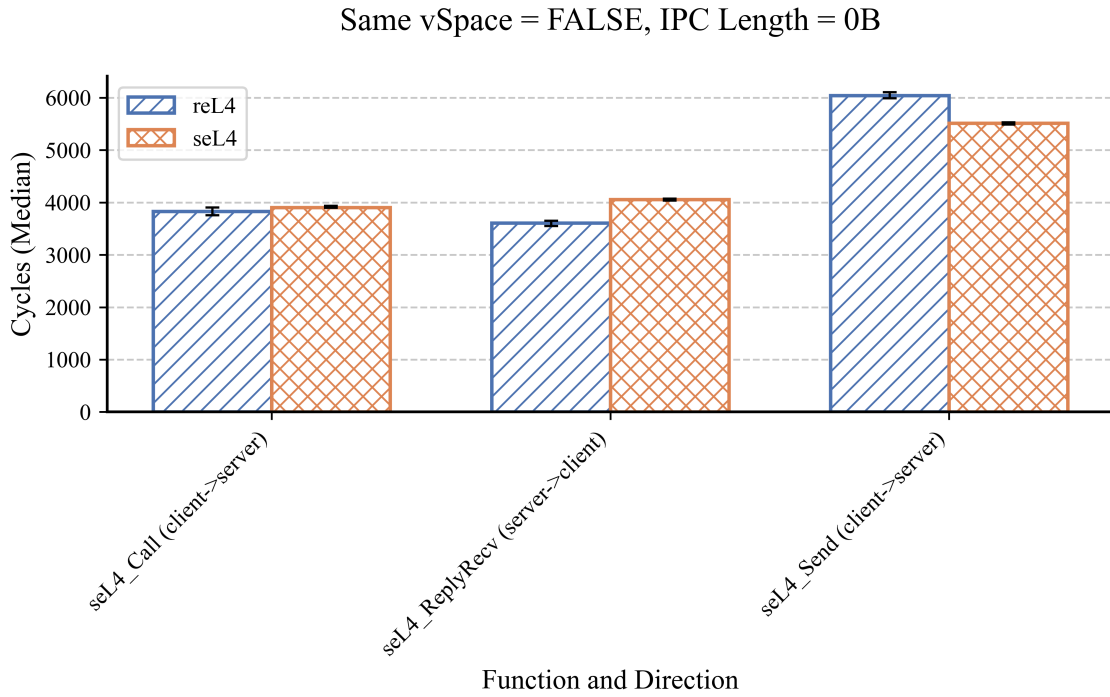


图 5.2 不同地址空间下长度为 0B 的 one way ipc 性能对比

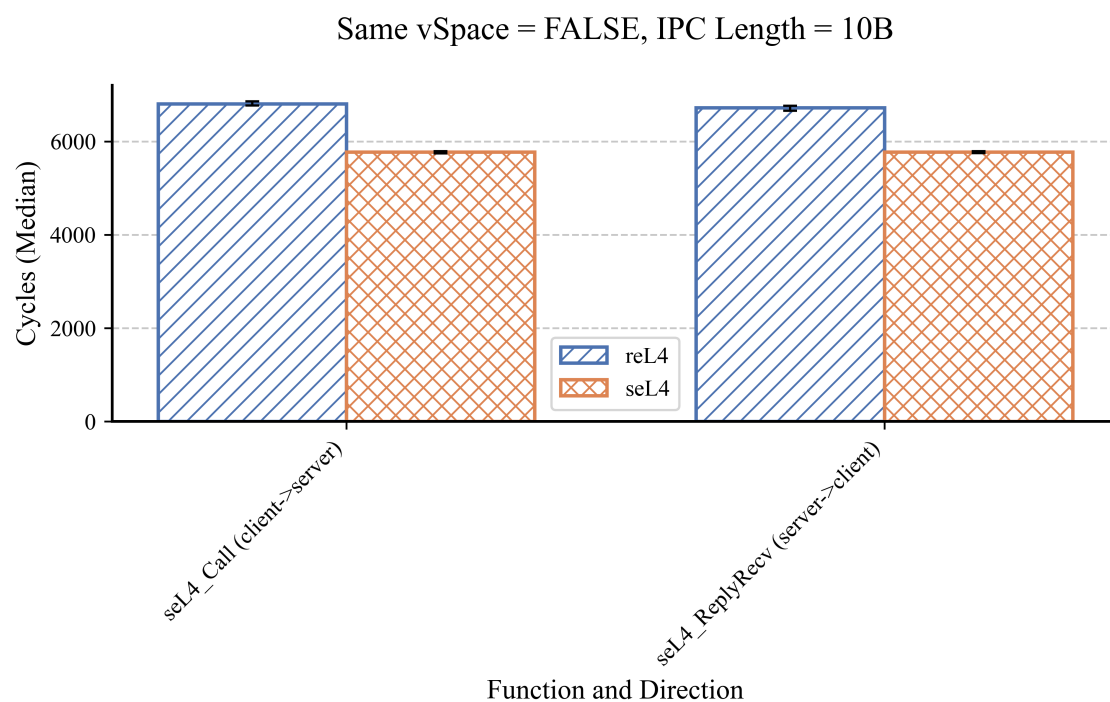


图 5.3 不同地址空间下长度为 10B 的 one way ipc 性能对比

一方面可能来源于 Rust 语言和 C 语言及其编译器所带来的差异。

5.3.3 中断相关测试

对于 reL4 和 seL4 的中断相关性能的对比较测试结果，如图5.4中所示。

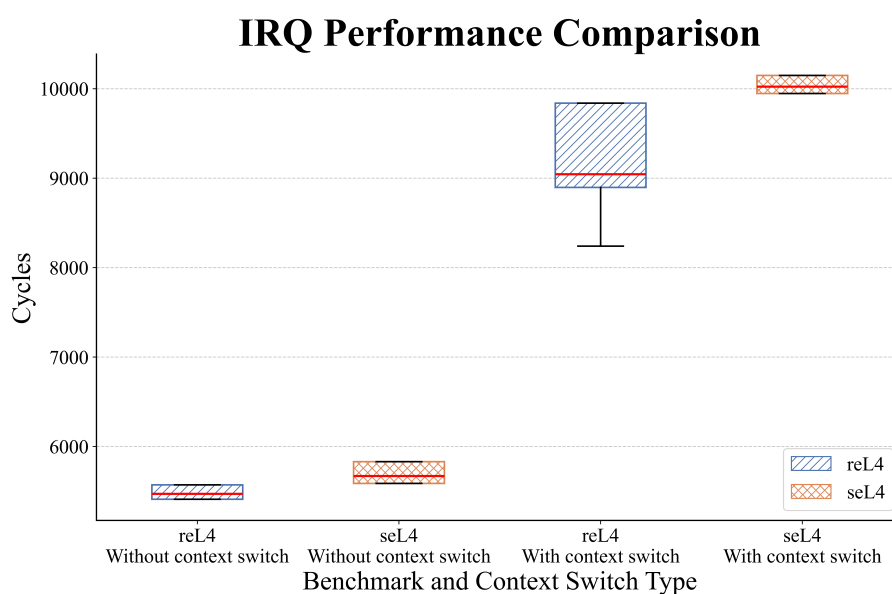


图 5.4 irq 性能对比

该箱形图展示了是否存在上下文切换情况下的 irq 性能，在该图中，seL4 的 irq 测试，无论是否存在 context switch，其波动范围都较小，具有很好的稳定性。

reL4 的 irq，虽然整体的性能相对要更好，但是稳定性则不如 seL4，尤其是在包含了 context switch 的情况下，其波动范围更大，极值发生概率更高。

对于没有 context switch 的情况下，reL4 的 irq 测试同样表现出较好的稳定性，并且性能开销低于 seL4 的性能开销，其基本的 cycle 数量和 ipc 测试中的 cycle 数量较为接近，可以认为，基本没有 context switch 情形下的 irq 实现较好。

对于 context switch 情形下的性能测试，需要关注，箱型图的上四分位数和最大值几乎重复，平均值和下四分之一分位数也较为接近，除此之外，最大值和 seL4 测试情形的差距，基本和没有 context switch 情形下 reL4 和 seL4 的差距接近。从这个测试结果可以认为，在存在 context switch 情形下的性能数值，按照离散程度可以分为三部分，一部分是和最大值及四分之三分位数聚集在一起，基本可以认为和 seL4 的执行具有相似路径，另一部分则主要是图中平均数附近，而少数则在最小值附近。

对应到代码层面，意味着，在 context switch 的情况下，reL4 不仅具有和 seL4 相同的执行路径（并且性能更优），同时还至少有两个更优化的执行路径，对应着性能数值的另外两部分聚类情况。

5.3.4 信号相关测试

信号机制同样是 seL4 下重要的消息发送机制，对于信号相关测试的结果在图 5.5 和图 5.6 中表明。

在图 5.5 中展现向更高优先级的进程发送 signal 的 reL4 和 seL4 的性能对比，在图 5.6 中则为向更低优先级的进程发送 signal 的性能对比。

在这两个测试结果中，呈现相反的结果，具体来说，向高优先级进程发送消息，在 reL4 下具有更少的响应时间，相反，向低优先级进程发送消息，在 reL4 下具有更多的响应时间。

从逻辑上来说，这是合理的，高优先级响应更快带来了低优先级响应更慢的问题。而从设计角度出发，reL4/seL4 本身就具有实时性的设计需求，尽管限于各种原因，没有能够进行一些实时性测试，但是高优先级的更快响应，对于一个具有实时性需求的内核而言，是具有优势的。

5.3.5 同步相关测试

同步相关测试目的是测试通过 seL4/reL4 的通信机制实现消息同步所带来的开销，相比于单纯的 One Way IPC 性能，该测试提供了更全面的系统消息传递开销评估。

在图 5.7 中显示了在不同数量消息等待者情况下，消息等待者的平均等待时间

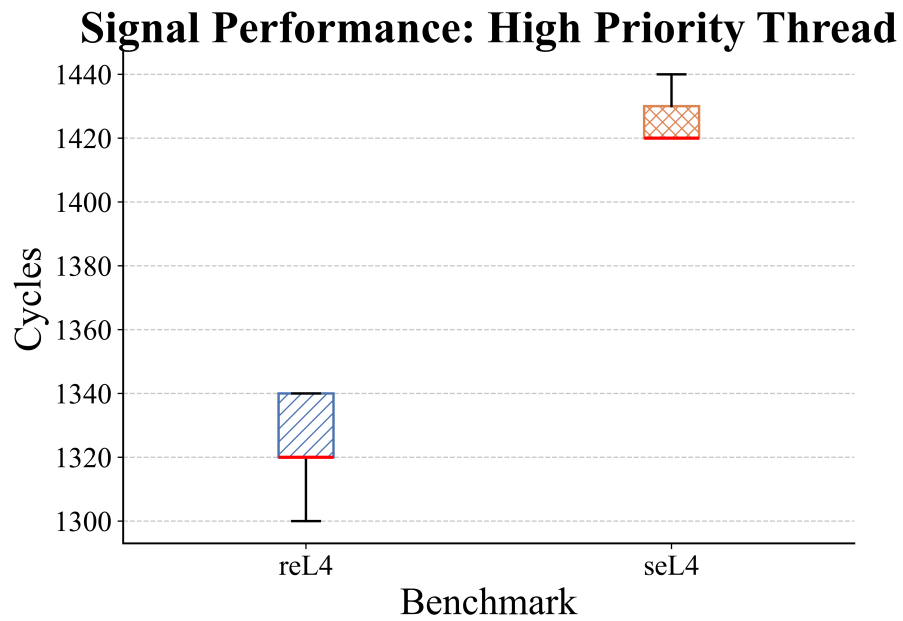


图 5.5 signal 性能对比（向高优先级进程发送）

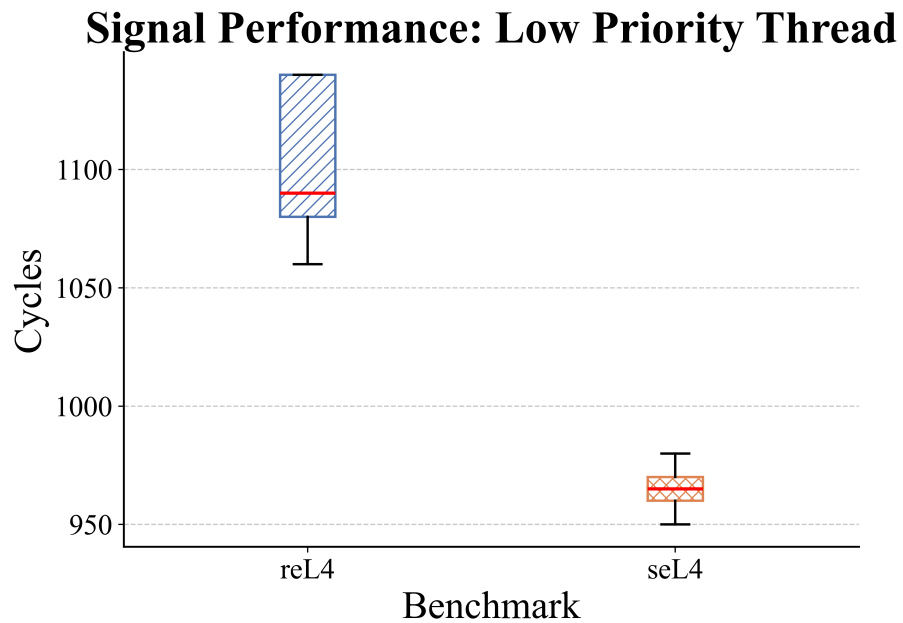


图 5.6 signal 性能对比（向低优先级进程发送）

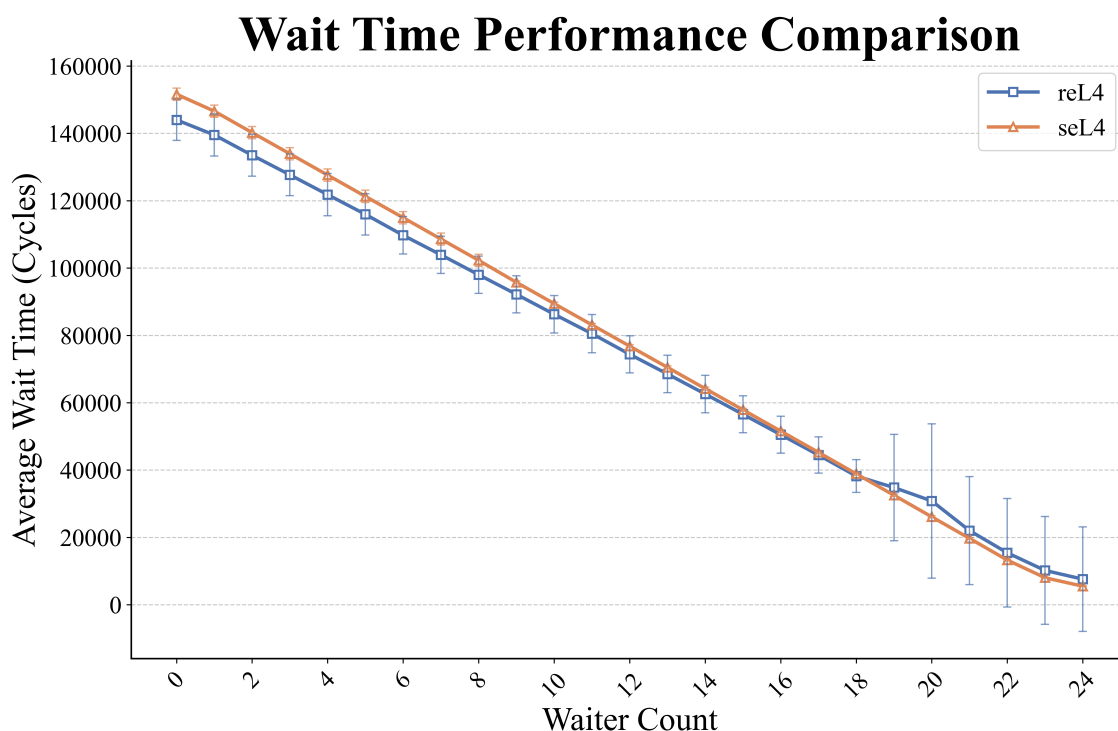


图 5.7 不同线程数量下等待时间对比

随线程数量的增加的变化。

从测试结果来看，随着消息等待者的数量增长，在 reL4 和 seL4 下的平均等待时间均呈现下降趋势。随着消息等待者的数量上升，整体的消息发送和接收的时间通常是增长的，因为更多的消息需要发送和接收。

而每个消息接收者存在从发送消息到被调度出来的时延，因此在消息接收者数量少的时候，消息接收者必须等待其他进程被调度出来并执行，这增加了其时延。当消息接收者数量多的时候，一个消息接收者接收到消息之后，下一次调度的进程依然是一个消息接收者，因此其平均接收时间通常会逐步下降。

在图5.7中，在消息等待者数量少的时候，seL4 略有一点优势，在消息等待者数量增长时，seL4 性能更优一些，而且具有相当的平稳性，在 waiter count 数量达到 20 左右的时候，reL4 出现了明显的波动。总体而言，reL4 和 seL4 的差距不大。

5.3.6 映射相关测试

映射相关测试主要针对系统的内存映射性能的评估，其结果如图 5.8所示。

在图 5.8中，总共包括了五个测试项目：

- Prepare Page Tables
- Allocate Pages Mapping
- Mapping

- Protect Pages as Read Only
- Unprotect Pages

这几个测试项目的具体内容，基本可以从上述测试项目的名称中直接得出，主要是对各级页表的性能测试，以及修改页表的保护属性等内容。

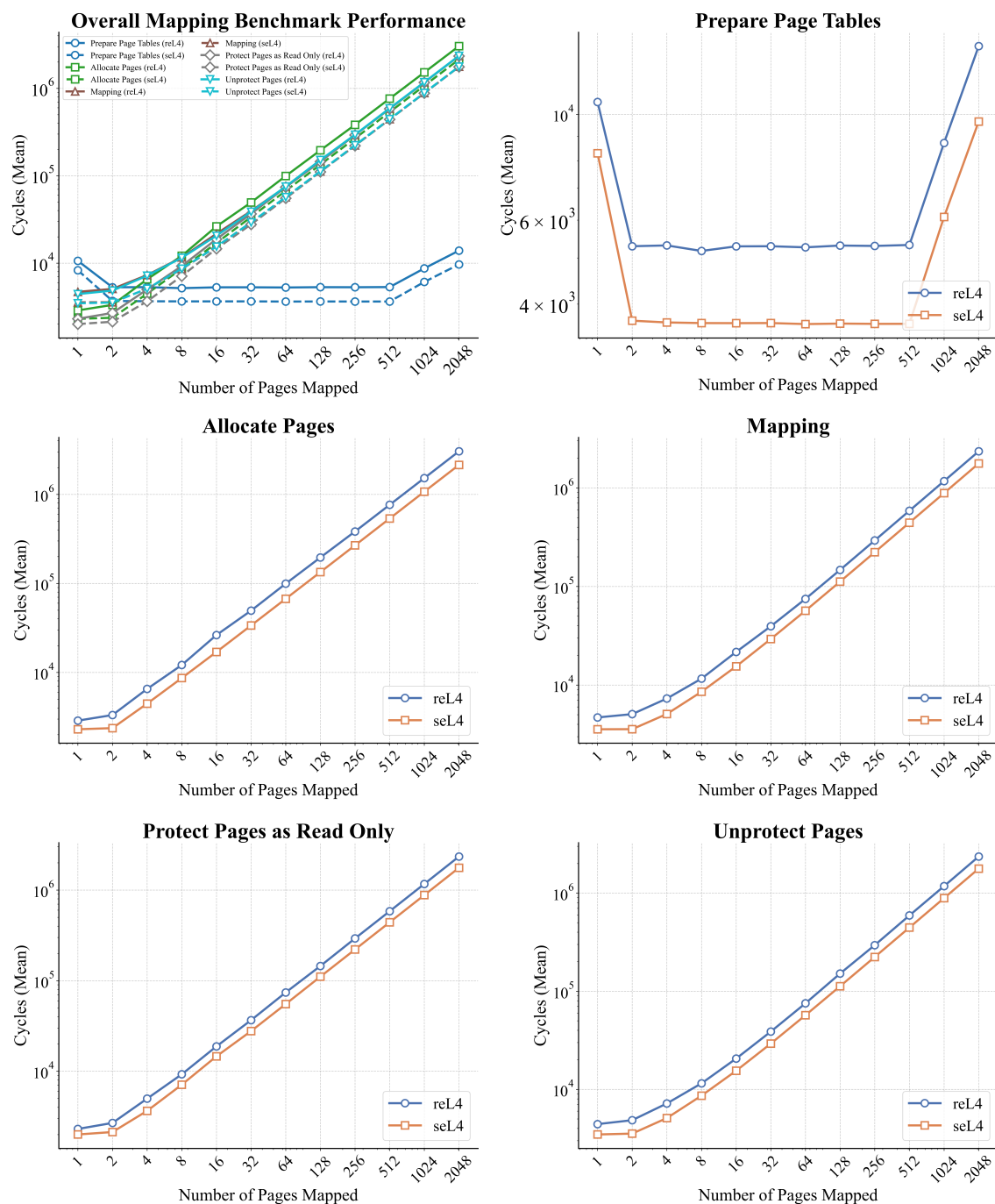


图 5.8 内存映射的性能对比

从测试结果来看，reL4 和 seL4 之间表现出来的性能行为基本一致，基本可以说明，reL4 在功能上可以对标 seL4 内核，但是存在一个性能差距（请注意坐标轴

为对数坐标轴)，这在几乎所有的内存映射测试中都是存在的。

5.4 reL4-Linux-kit 综合性能评估的测试环境

reL4-Linux-kit 综合性能评估的测试环境和代码 5.2 基本一致，而其宿主机的配置则和 5.3.1 中的宿主机配置一致。

5.5 reL4-Linux-kit 综合性能评估

对于 reL4-Linux-kit 的测试，由于其可以基于 seL4 或者 reL4 内核运行，因此，可以对比其在 reL4 和 seL4 上的执行效率来评测 reL4-Linux-kit。同时，由于其支持 Linux syscall，也可以将 reL4-Linux-kit 在 reL4 上的性能，对比原生 Linux 的执行性能。

对于 reL4-Linux-kit 的测试内容，主要包括 libc, busybox, lua, iotest, lmbench 等测试。

除此之外，函数调用转 IPC 的设计，会导致不同模块合并成一个模块的行为会影响其性能，对此也需要做评估。我们只评测所有可拆分的模块均进行拆分，以及所有模块均进行合并的结果，而不是穷举所有模块的组合可能。

对于评测的指标，主要采用运行时间作为指标，为了时间测量的准确性，我们首先运行若干次进行预热，之后再运行若干次，去掉最高值和最低值以排除极端数值的影响，再做进一步处理。预热的次数取 2 次，正式运行的次数选取 5 次，由于总共只有五次，去掉最高值和最低值，只有 3 次，进一步处理再做中位数等没有实际统计意义，因此选取剩下三次的平均值作为结果的值。

对于计时的方式，reL4-Linux-kit 虽然支持了部分时钟相关的 syscall 接口，但是使用 time 命令运行仍然存在一些问题，因此采用如下方法运行：首先，在 host 环境中统计测例在 QEMU 模拟器中运行的时间，同时，由于在测例运行之外还运行了启动等内容，因此需要将总的运行时间减去不带任何测例运行的时间，即可得到在 QEMU 模拟器中测例的运行时间。由于每一次运行时间存在波动，因此，只能将上述去除最高值最低值之后的平均值进行相减，以尽量得到一个精确的值。对于 Linux 环境来说，虽然 QEMU 模拟器环境中存在一定的时钟漂移问题，但其误差相对于整个测试时长来说通常较小，因此也可以直接在 busybox 中运行 time 命令，从而相对精确地测算出时间。

另外，由于 reL4-Linux-kit 的运行都在 QEMU 模拟器环境下，所以对于 Linux 的测试，也必须在 QEMU 模拟器环境下运行并测试，以尽可能排除虚拟化环境的

影响。因此我们选用 Linux 内核 6.16.3 版本，并采用和 reL4-Linux-kit 运行态相同的 aarch64 架构，作为 QEMU 模拟器中测试的 Linux 版本。

测试结果如表 5.4 所示。

表 5.4 Linux 测例在不同操作系统配置下的性能测试结果（单位：秒）

测例类型	Linux 内核 测例运行时间	seL4 内核 reL4-Linux-kit		reL4 内核 reL4-Linux-kit	
		合并模块运行时间	拆分模块运行时间	合并模块运行时间	拆分模块运行时间
basic	1.133	1.555	1.572	1.442	1.449
run-static	6.210	4.252	4.337	3.572	3.695
busybox-testcase	1.270	2.995	3.038	2.616	2.706
lua	0.093	0.760	0.783	0.622	0.656
iozone	84.563	149.439	157.940	141.838	157.006
libc	4.283	13.719	13.862	12.128	12.574
lmbench	99.260	60.494	60.102	59.346	59.386

对于表 5.4 的测评结果，可以观察到基于 seL4/reL4 内核的 reL4-Linux-kit 和 Linux 内核之间具有一定的性能差距。微内核的几个测例中，以 reL4 作为内核，并将所有服务组件拼合成为单个服务组件的方式运行时，性能与 Linux 差距最小。由于取消了系统服务组件间的 IPC，因此该测例下与 Linux 的性能差距主要来源于两个地方，一个是 reL4 截获用户态 Linux app 的 syscall 并转发给用户态系统服务组件，与直接进行 syscall 进入宏内核的性能差距，这是使用我们所设计的 syscall 转 IPC 方法存在的开销；另一个是用户态系统服务组件本身和 Linux 执行的性能差距，Linux 作为一个经典的宏内核实现，其具有良好的优化，而 reL4-Linux-kit 仅作为验证兼容系统可行性的一个尝试。因此，reL4 下的 reL4-Linux-kit 和原生 Linux 下存在一定的性能差距，符合预期。

其次，对于我们设计的函数调用和 IPC 调用之间的透明转换机制，分别在 reL4 内核和 seL4 内核的基础上进行了评测。这两者之间的性能差距纯粹是来自于函数调用和 IPC 调用方式之间的差距，在同一个内核下，两者差距始终不大。总体而言，这两种通信方式之间差距相对不大，但也跟不同测例集合需要的组件间通信数量有一定关系，在如 iozone 这样的组件间通信数量较多的测例集中，就会出现

函数调用通信转为 IPC 通信带来的开销更大的现象。总体而言，根据以上测试结果，可以认为我们设计的函数调用和 IPC 调用透明转换的机制具有较好的性能优势。

第三，对于 reL4 和 seL4 之间的实现性能对比。这是兼容同一个二进制接口的内核的两个不同实现，从测试结果可见，reL4 实现的性能均强于 seL4 实现的性能，总体综合性能差距在 10% 左右。由于这些测例本身大量调用了 seL4/reL4 的各类系统调用而非个别系统调用路径，无法简单地认为是其中的某个系统调用路径性能过差而产生这样的差距。该性能差距应当是系统级别的差距，而不是在之前的 benchmark 测试中的部分系统调用路径的差距，同时，也有一部分的性能差异可能来自于 Rust 和 C 两门语言的性能差异。由此可以认为，reL4 的整体性能实现更优。

最后，需要看到若干特殊情况。在 run-static 和 lmbench 两个测例中，Linux 所花费的时间更多，这并非意味着 Linux 的性能更差，而是由于其中个别测例的问题。例如，在前面分析功能实现的时候，run-static 测例集合中的 socket 测例集合并未实现，因此会存在 fake 实现和报错的问题，而 Linux 对网络 socket 的超时报错有着自己的机制，并非直接简单的 fake 实现，因此可能会导致测试完成的时间有所不同。这些差异实际来自于 syscall 实现的语义兼容性和 reL4-Linux-kit 只完成了部分 Linux syscall 导致的，而并非来自内核的实现性能。

5.6 本章小结

本章对 reL4-Linux-kit 原型系统进行了系统评估，主要包括功能实现评估和性能评估两个维度。

第一，在功能实现评估方面（见第 5.2 节），通过系统性的测试验证了 reL4 内核与 seL4 内核的二进制接口兼容性，证实了基于组件化重构的 reL4 内核能够完整保持与 seL4 的二进制兼容特性。同时，详细评估了 reL4-Linux-kit 对 Linux 系统调用的兼容性支持范围，并测试了多组 Linux 应用程序在该环境下的运行情况和系统调用情况。

功能实现评估的实验结果表明，基于组件化架构重构的 reL4 微内核及其配套的 reL4-Linux-kit 用户态兼容层，作为一个完整的研究原型系统，实现了既定的功能性目标。这为构建全栈内存安全的操作系统架构提供了一个可行且具有实践价值的技术方案。

第二，在性能评估方面（见第 5.5 节），我们对比分析了同一测试套件在原生 Linux 环境、基于 seL4 内核的 reL4-Linux-kit 环境以及基于 reL4 内核的 reL4-Linux-kit 环境下的性能表现。测试还包含了系统组件合并与解耦两种模式下的性能差异

比较。对于观察到的性能差异，我们从微内核架构特性、IPC 开销、Rust 开发语言等角度进行了初步分析。

性能评估结果显示，尽管 reL4 和 reL4-Linux-kit 的实现与原生 Linux 性能存在预期内的差距，但作为一个研究原型系统，其性能表现仍在可接受范围内，且基本优于现有的 seL4 实现，这证明了该架构的实用性和进一步发展潜力。这些性能数据也为后续优化工作提供了有价值的参考依据。

第6章 总结与展望

6.1 全文总结

本文从第1章开头提出的组件化微内核系统演进目标出发，聚焦于微内核落地过程中面临的三类关键挑战：内核实现中的耦合性与可扩展性（问题一）、功能可复用性（问题二）以及面向宏内核生态的兼容性（问题三）（见第1.4节的问题一至问题三）。围绕上述问题，本文以经过形式化验证的 seL4 为基础，采用内存安全的 Rust 进行深度重构，并扩展用户态生态兼容能力，最终形成了“reL4 内核 + reL4-Linux-kit 兼容层”的完整研究体系。

第一，面向问题一（耦合性与可扩展性），本文设计并实现了与 seL4 二进制兼容的 reL4 微内核。通过在保持 seL4 用户态接口二进制布局一致的前提下引入 Rust 的安全抽象与约束，将 unsafe 操作限制在必要的底层封装中，从而在工程上提升了内核实现的可维护性，并为后续特性集成留出了演进空间。对应地，第2章回顾了 seL4 的关键机制与原则，第3章将这些机制在 reL4 中重构为可拆分组件，为“可扩展的演进”提供结构基础。

第二，面向问题二（功能可复用性），本文提出并实现了深度组件化的微内核架构。第3章将内核划分为 CSpace、VSpace、IPC、Task 等边界清晰的组件，并在组件间定义了稳定的交互约定：例如以 VSpace 分离地址空间机制，以 Transfer trait 等抽象化 IPC 传输原语，以 kernel 层承载策略编排。组件化结果不仅提升了代码的模块化程度，还把“功能复用”转化为可操作的工程约束：组件可独立维护、可在不同内核演进路径中复用，并通过合理的接口边界降低耦合扩散风险。

第三，面向问题三（生态兼容性），本文构建了用户态兼容层框架 reL4-Linux-kit。第4章通过基于 Fault Handler 的系统调用二进制兼容机制，将 Linux 二进制程序中的 trap 行为转换为向用户态服务组件发送 IPC 消息，从而实现无需修改源码的兼容路径；同时，本文进一步提出函数调用与 IPC 通信的双向透明转换机制，使得在同一进程地址空间内可退化为函数调用以降低开销，而跨进程/跨地址空间的协作则采用 IPC 以维持隔离性。该兼容层进一步提供了关键的用户态服务组件（如 kernel thread、文件系统与驱动服务），并以 Rust trait 接口抽象实现细粒度的服务协作，从而把“生态兼容”落在可运行的系统上。

第四，通过第5章的功能与性能评估，本文完成了对研究方案的验证闭环。功能评估方面，reL4 内核在测试中保持了与 seL4 的高度二进制接口兼容性；reL4-Linux-kit 实现了与部分关键 Linux syscall 的兼容，并能够运行多组 Linux 应用/测

例，从而验证了生态兼容路径的可行性。性能评估方面，本文观察到 reL4-Linux-kit 相较原生 Linux 存在预期性能差距，主要来源于 syscall 拦截并转发到用户态服务组件的开销以及用户态服务执行的差异；同时实验表明在“组件合并”的策略下性能可进一步逼近 Linux 基线，并且在同一二进制接口条件下 reL4 相对 seL4 的综合性能差距控制在可接受范围内，为后续优化给出了明确方向。

综上，本文不仅实现了从 C 到 Rust 的语言级迁移，更把形式化验证基础上的内核能力通过组件化工程方式落实为可演进内核结构；同时通过用户态兼容层与透明转换机制，把内核安全优势扩展到全栈系统可用性层面，并最终形成了面向宏内核生态的可运行解决方案。

6.2 未来工作展望

尽管本文取得了上述研究成果，但受限于时间与篇幅，仍有诸多方向值得在未来进行更深入的探索：

1. 形式化验证的延伸：本文实现的 reL4 微内核继承了 seL4 微内核的设计规范，但其 Rust 实现本身仍尚未完成形式化验证。未来工作可进一步结合 Rust 生态的形式化验证/形式化测试工具链，对关键内核组件与 unsafe 封装边界开展验证，以期获得“内存安全 + 逻辑正确”的更强保证。
2. 性能的深度优化与评测：本文已验证系统的功能正确性与基本性能。未来可开展更深入的性能剖析，尤其针对 IPC 透明转换机制、多核同步（大内核锁）开销以及 Linux 兼容层性能损耗进行精细化 profiling。在此基础上，可探索更高效的 IPC 调度策略、面向多核的细粒度/无锁同步方案，以及更贴合硬件的优化（例如用户态中断路径等），以提升系统整体性能与可预测性。
3. 硬件架构与驱动的扩展：目前 reL4 的支持主要集中在 aarch64 与 riscv64 等架构以及 QEMU 模拟平台。未来工作可将其移植到更多硬件架构（如 x86-64），并扩展相应平台的时钟、中断与设备驱动能力，从而拓宽可用场景。
4. 用户态兼容生态的完善：在 reL4-Linux-kit 的基础上，未来可进一步扩展 Linux 系统调用兼容范围，并逐步引入与真实应用密切相关的特性（如信号、进程间通信机制等），提升兼容生态的完整度与可迁移性。

本文的工作为微内核的发展打开了新的可能性。我们相信，沿着上述方向持续探索与迭代，最终能够构建出兼具安全性、高性能与通用性的实用操作系统。

参考文献

- [1] Klein G, Elphinstone K, Heiser G, et al. sel4: Formal verification of an os kernel[C]//Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles. 2009: 207-220.
- [2] Accetta M, Baron R, Bolosky W, et al. Mach: A new kernel foundation for unix development [Z]. 1986.
- [3] Herder J N, Bos H, Gras B, et al. Minix 3: A highly reliable, self-repairing operating system [J]. ACM SIGOPS Operating Systems Review, 2006, 40(3): 80-89.
- [4] Singh A. Mac os x internals a systems approach[J]. pearson schweiz ag, 2006.
- [5] Just for fun: The story of an accidental revolutionary[M]. 2001.
- [6] Tanenbaum A S, Woodhull A S. Operating systems - design and implementation (3. ed.)[M]. Operating systems - design and implementation (3. ed.), 2006.
- [7] Leather A. Intel management engine runs minix 3 os[EB/OL]. Hexus.net, 2017[2025-09-05]. <https://m.hexus.net/tech/news/software/111857-intel-management-engine-runs-minix-3-os/>.
- [8] Liedtke J. On micro-kernel construction[J]. ACM SIGOPS Operating Systems Review, 1995, 29(5): 237-250.
- [9] Liedtke J. Improving ipc by kernel design[C]//Proceedings of the fourteenth ACM symposium on Operating systems principles. 1993: 175-188.
- [10] QNX Software Systems. System architecture: The microkernel[M/OL]. QNX Software Systems Limited, 2004[2025-09-07]. https://www.qnx.com/developers/docs/6.3.0SP3/neutrino/syss_arch/kernel.html.
- [11] Hohmuth D I M, Hohmuth M. Pragmatic nonblocking synchronization for real-time systems [J]. USENIX Association, 2002.
- [12] Technische Universität Dresden. The fiasco.oc microkernel[EB/OL]. Faculty of Computer Science, TU Dresden, 2025[2025-09-07]. <http://os.inf.tu-dresden.de/fiasco/>.
- [13] Heiser G, Leslie B. The okl4 microvisor: convergence point of microkernels and hypervisors [J]. DBLP, 2010.
- [14] OKL. L4 microkernel family[Z]. 2013.
- [15] Operating Systems Group, Technische Universität Dresden. Bibliography of publications on l4ka and fiasco.oc microkernels[EB/OL]. 2025[2025-09-07]. <https://os.inf.tu-dresden.de/L4/bib.html>.
- [16] BlackBerry Limited. Blackberry qnx hypervisor awarded world' s first automotive safety integrity level d certification[EB/OL]. 2019[2025-09-07]. <https://www.blackberry.com/us/en/company/newsroom/press-releases/2019/blackberry-qnx-hypervisor-awarded-worlds-first-automotive-safety-integrity-level-d-certification>.
- [17] Fuchsia 中文社区. Fuchsia China: Fuchsia OS 中文社区[EB/OL]. 2025[2025-09-07]. <https://fuchsia-china.com>.

-
- [18] Xia Y, Du D, Hua Z, et al. Boosting inter-process communication with architectural support[J]. ACM Transactions on Computer Systems (TOCS), 2022, 39(1-4): 1-35.
 - [19] Mi Z, Li D, Yang Z, et al. Skybridge: Fast and secure inter-process communication for micro-kernels[C]//Proceedings of the Fourteenth EuroSys Conference 2019. 2019: 1-15.
 - [20] Watson R N, Woodruff J, Neumann P G, et al. Cheri: A hybrid capability-system architecture for scalable software compartmentalization[C]//2015 IEEE Symposium on Security and Privacy. IEEE, 2015: 20-37.
 - [21] Narayanan V, Baranowski M S, Ryzhyk L, et al. Redleaf: Towards an operating system for safe and verified firmware[C]//Proceedings of the Workshop on Hot Topics in Operating Systems. 2019: 37-44.
 - [22] Chen H, Miao X, Jia N, et al. Microkernel goes general: Performance and compatibility in the {HongMeng} production microkernel[C]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 2024: 465-485.
 - [23] Vestal S. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance[C]//28th IEEE international real-time systems symposium (RTSS 2007). IEEE, 2007: 239-243.
 - [24] Audsley N C. On priority assignment in fixed priority scheduling[J]. Information Processing Letters, 2001, 79(1): 39-44.
 - [25] Baruah S K, Burns A, Davis R I. Response-time analysis for mixed criticality systems[C]//2011 IEEE 32nd Real-Time Systems Symposium. IEEE, 2011: 34-43.
 - [26] Baruah S, Bonifaci V, D'angelo G, et al. Preemptive uniprocessor scheduling of mixed-criticality sporadic task systems[J]. Journal of the ACM (JACM), 2015, 62(2): 1-33.
 - [27] Baruah S K, Bonifaci V, d' Angelo G, et al. Mixed-criticality scheduling of sporadic task systems[C]//European symposium on algorithms. Springer, 2011: 555-566.
 - [28] Mollison M S, Erickson J P, Anderson J H, et al. Mixed-criticality real-time scheduling for multicore systems[C]//2010 10th IEEE international conference on computer and information technology. IEEE, 2010: 1864-1871.
 - [29] Chisholm M, Ward B C, Kim N, et al. Cache sharing and isolation tradeoffs in multicore mixed-criticality systems[C]//2015 IEEE Real-Time Systems Symposium. IEEE, 2015: 305-316.
 - [30] Baruah S, Chattopadhyay B, Li H, et al. Mixed-criticality scheduling on multiprocessors[J]. Real-Time Systems, 2014, 50(1): 142-177.
 - [31] Gratia R, Robert T, Pautet L. Adaptation of run to mixed-criticality systems[J]. JRWRTC, 2014: 25.
 - [32] Gratia R, Robert T, Pautet L. Generalized mixed-criticality scheduling based on run[C]//Proceedings of the 23rd International Conference on Real Time and Networks Systems. 2015: 267-276.
 - [33] Al-bayati Z, Zhao Q, Youssef A, et al. Enhanced partitioned scheduling of mixed-criticality systems on multicore platforms[C]//The 20th Asia and South Pacific Design Automation Conference. IEEE, 2015: 630-635.

- [34] Xu H, Burns A. Semi-partitioned model for dual-core mixed criticality system[C]//Proceedings of the 23rd International Conference on Real Time and Networks Systems. 2015: 257-266.
- [35] Naghavi A, Safari S, Hessabi S. Tolerating permanent faults with low-energy overhead in multi-core mixed-criticality systems[J]. IEEE Transactions on Emerging Topics in Computing, 2021, 10(2): 985-996.
- [36] Ali A, Kim K H. Cluster-based multicore real-time mixed-criticality scheduling[J]. Journal of Systems Architecture, 2017, 79: 45-58.
- [37] Kim H, Broman D, Lee E A, et al. A predictable and command-level priority-based dram controller for mixed-criticality systems[C]//21st IEEE Real-Time and Embedded Technology and Applications Symposium. IEEE, 2015: 317-326.
- [38] Kritikakou A, Baldellon O, Pagetti C, et al. Monitoring on-line timing information to support mixed-critical workloads[C]//IEEE Real-Time Systems Symposium 2013. 2013: 9-10.
- [39] Kritikakou A, Rochange C, Faugère M, et al. Distributed run-time wcet controller for concurrent critical tasks in mixed-critical systems[C]//Proceedings of the 22nd International Conference on Real-Time Networks and Systems. 2014: 139-148.
- [40] Yun H, Yao G, Pellizzoni R, et al. Memory access control in multiprocessor for real-time systems with mixed criticality[C]//2012 24th Euromicro conference on real-time systems. IEEE, 2012: 299-308.
- [41] Kotaba O, Nowotsch J, Paulitsch M, et al. Multicore in real-time systems—temporal isolation challenges due to shared resources[C]//16th Design, Automation & Test in Europe Conference and Exhibition. 2013.
- [42] Freitag J, Uhrig S, Ungerer T. Virtual timing isolation for mixed-criticality systems[C]//30th Euromicro Conference on Real-Time Systems (ECRTS 2018). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018: 13-1.
- [43] Fleming T. Extending mixed criticality scheduling[D]. University of York, 2013.
- [44] Ekberg P, Stigge M, Guan N, et al. State-based mode switching with applications to mixed criticality systems[J]. Proc. WMC, RTSS, 2013: 61-66.
- [45] Ekberg P, Yi W. Bounding and shaping the demand of generalized mixed-criticality sporadic task systems[J]. Real-time systems, 2014, 50(1): 48-86.
- [46] Chen G, Guan N, Liu D, et al. Utilization-based scheduling of flexible mixed-criticality real-time tasks[J]. IEEE Transactions on Computers, 2017, 67(4): 543-558.
- [47] Ranjbar B, Hoseinghorban A, Sahoo S S, et al. Improving the timing behaviour of mixed-criticality systems using chebyshev's theorem[C]//2021 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2021: 264-269.
- [48] Ekberg P, Yi W. Schedulability analysis of a graph-based task model for mixed-criticality systems[J]. Real-time systems, 2016, 52(1): 1-37.
- [49] Yang J, Xu G, Chen G, et al. Efficient runtime slack management for edf-vd-based mixed-criticality scheduling[J]. Journal of Systems Architecture, 2021, 117: 102119.

-
- [50] Chen G, Guan N, Hu B, et al. Edf-vd scheduling of flexible mixed-criticality system with multiple-shot transitions[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(11): 2393-2403.
- [51] Baruah S, Guo Z. Mixed-criticality scheduling upon varying-speed processors[C]//2013 IEEE 34th Real-Time Systems Symposium. IEEE, 2013: 68-77.
- [52] Huang P, Kumar P, Giannopoulou G, et al. Energy efficient dvfs scheduling for mixed-criticality systems[C]//Proceedings of the 14th International Conference on Embedded Software. 2014: 1-10.
- [53] Lyons A, McLeod K, Almatary H, et al. Scheduling-context capabilities: A principled, light-weight operating-system mechanism for managing time[C]//Proceedings of the Thirteenth EuroSys Conference. 2018: 1-16.
- [54] Sprunt B, Sha L, Lehoczky J. Aperiodic task scheduling for hard-real-time systems[J]. Real-Time Systems, 1989, 1(1): 27-60.
- [55] Stanovich M, Baker T P, Wang A I, et al. Defects of the posix sporadic server and how to correct them[C]//2010 16th IEEE Real-Time and Embedded Technology and Applications Symposium. IEEE, 2010: 35-45.
- [56] Härtig H, Hohmuth M, Liedtke J, et al. The performance of μ -kernel-based systems[J]. ACM SIGOPS Operating Systems Review, 1997, 31(5): 66-77.
- [57] The MklLinux Development Community. Mklinux: Linux for powerpc macintosh computers [EB/OL]. 1998[2025-09-07]. <https://mklinux.org>.
- [58] Li H, Guo L, Yang Y, et al. An empirical study of {Rust-for-Linux}: The success, dissatisfaction, and compromise[C]//2024 USENIX Annual Technical Conference (USENIX ATC 24). 2024: 425-443.
- [59] rCore OS Development Team. rcore os: The rust implementation of tcos (teaching os)[CP/OL]. GitHub, 2025[2025-09-07]. <https://github.com/rcore-os/rCore>.
- [60] Redox OS Developers. Redox os - your next OS[EB/OL]. 2024[2025-09-07]. <https://redox-os.org>.
- [61] 李龙昊. reL4 微内核的设计与实现[R]. 北京: 清华大学计算机科学与技术系, 2023.
- [62] 廖东海. ReL4: 高性能异步微内核设计与实现[D/OL]. 北京: 北京理工大学, 2025. https://github.com/CtrlZ233/graduation_thesis/blob/main/ReL4.pdf.
- [63] 廖东海, 陆慧梅, 陈伟豪, 等. ReL4: 高性能异步微内核设计与实现[J]. 小型微型计算机系统, 2025.
- [64] Team A D. ArceOS: A modular, multilingual library operating system[EB/OL]. 2023[2023-10-15]. <https://github.com/arceos-org>.
- [65] Peters S, Danis A, Elphinstone K, et al. For a microkernel, a big lock is fine[C]//Proceedings of the 6th Asia-Pacific Workshop on Systems. 2015: 1-7.

附录 A 其他附表

A.1 reL4 不同 feature 总计需要的有效测例分类及数量

表 A.1 reL4 不同 feature 总计需要的有效测例分类及数量

测例分类	测例内容简要描述	测例个数	测例通过个数
Sys Call	测试 seL4 的基本 syscall 的执行	16	16
Serserv Parent	测试服务器 (Server) 服务中的父线程功能	10	10
Timer	测试内核的定时器 (Timer) 中断功能	2	2
Bind	测试将通知 (Notification) 对象绑定到线程控制块 (TCB) 的操作。	6	6
Cnode OP	测试 CNode (Capability Node) 的操作, 如复制、派生、删除能力项等, 验证能力空间的操纵。	9	9
Cspace	测试能力空间 (Capability Space) 的整体管理	1	1
Domains	测试 seL4 的域 (Domain) 调度机制, 确保不同域之间的隔离与调度。	5	5
Cancel Badge Send	测试在 IPC 发送过程中, 带有标识 (Badge) 的通知在取消发送时的行为。	2	2
Page Fault	测试处理缺页异常 (Page Fault) 的机制, 包括页错误处理程序的正确调用。其中最后一个测例无论如何都不需要运行	10	9
Time Out	测试 MCS 的 TimeOut 机制下的行为是否符合预期。	3	3
FPU	测试浮点运算单元 (FPU) 的状态在线程切换时的正确保存与恢复。	3	3

续表 A.1 reL4 不同 feature 总计需要的有效测例分类及数量

测例分类	测例内容简要描述	测例个数	测例通过个数
Frame Exports	测试帧 (Frame) 能力是否能够正确地跨地址空间映射。	1	1
Frame XN	测试将帧 (Frame) 映射为不可执行 (eXecute Never) 页的功能, 验证安全特性。	2	2
Framed IPC	测试使用专门的 IPC 帧 (Frame) 进行通信的机制。	3	3
Retype	测试将未初始化的内存对象重定型 (Retype) 为特定内核对象 (如线程、CNode 等) 的过程。	3	3
Interrupt	测试内核的中断处理架构, 包括中断的接收、处理和确认。	5	5
IPCRIGHTS	测试在 IPC 消息中传递能力 (Rights) 的机制, 验证权限的正确传递和剥离。	5	5
IPC	测试基本的线程间通信 (IPC) 机制, IPC0028 测例无论如何都不会运行到	27	26
Multi Core	测试内核在多核 (SMP) 环境下的正确性, 包括核间通信、同步和调度。	5	5
NB Wait	测试非阻塞 (Non-Blocking) 的等待操作	1	1
PT	测试页表 (Page Table) 的操作, 如映射、取消映射和属性设置。PT0002 要求 Aarch32 架构, reL4 不支持	2	1
Preempt Revoke	测试在撤销 (Revoke) 大量能力时, 内核的抢占 (Preemption) 行为是否正确。	1	1
Regression		3	3
Sched Context	测试调度上下文 (Sched Context) 对象的分配、配置和控制, 用于控制线程的执行时间。	13	13

续表 A.1 reL4 不同 feature 总计需要的有效测例分类及数量

测例分类	测例内容简要描述	测例个数	测例通过个数
Sched	测试内核的调度器 (Scheduler) 算法和行为, 如优先级调度、时间片轮转等。	21	21
Serserv Client	测试客户端与服务器 (Server) 之间的交互逻辑。	5	5
Serserv Cli Proc	测试作为客户端角色的线程与服务器之间的交互。	5	5
SMC	测试安全监控调用 (Secure Monitor Call) 的支持	8	8
SYNC	测试内核中通知机制实现的同步原语	4	4
Threads	测试线程 (Thread) 的创建、配置、启动、挂起和删除等生命周期管理。	2	2
tls	测试线程本地存储 (Thread-Local Storage) 功能	3	3
trivial	用于验证系统最基本的功能是否启动并运行。	3	3
vspace	测试虚拟地址空间 (VSpace) 的管理, 包括地址空间的创建、切换和销毁。VSpace0010 测例需要 IA32 架构, reL4 不支持	8	7
total		197	193

A.2 reL4-Linux-kit 的 Linux syscall 实现情况

表 A.2 reL4-Linux-kit 的 Linux syscall 实现情况

syscall 分类	Syscall 名称	简要含义	是否为 fake 实现
进程管理	clone	创建新进程	否
	execve	执行程序	否
	exit	终止当前进程	否
	getpid	获取进程 ID	否
	getppid	获取父进程 ID	否

续表 A.2 reL4-Linux-kit 的 Linux syscall 实现情况

syscall 分类	Syscall 名称	简要含义	是否为 fake 实现
	gettid	获取线程 ID	否
	kill	向进程发送信号	否
	tkill	向线程发送信号	否
	sched-yield	让出处理器	否
	set-tid-address	设置线程 ID 地址	否
	wait4	等待进程终止并获取资源使用情况	否
	futex	快速用户空间互斥锁	否
	shmget	获取共享内存段	否
	shmat	附加共享内存段	否
	shmctl	控制共享内存段	否
	get-robust-list	获取稳健互斥锁列表	是
内存管理	brk	改变数据段大小	否
	mmap	映射文件或设备到内存	否
	munmap	解除内存映射	否
	mprotect	设置内存保护权限	是
权限管理	getuid	获取用户 ID	是
	getgid	获取组 ID	是
	geteuid	获取有效用户 ID	是
	getegid	获取有效组 ID	是
设备控制	ioctl	设备控制操作	否
时间管理	clock_gettime	获取时钟时间	否
	gettimeofday	获取当前时间	否
	nanosleep	高精度睡眠	否
	setitimer	设置间隔定时器	否
事件管理	ppoll	等待文件描述符上的事件（带超时）	否
	pselect6	等待文件描述符上的事件（带超时和信号掩码）	否
文件系统	chdir	改变当前工作目录	否
	close	关闭文件描述符	否

续表 A.2 reL4-Linux-kit 的 Linux syscall 实现情况

syscall 分类	Syscall 名称	简要含义	是否为 fake 实现
	dup	复制文件描述符	否
	dup3	复制文件描述符并指定新描述符的 flags	否
	faccessat	检查文件访问权限 (at 版本)	否
	fcntl	操作文件描述符	否
	fstat	获取文件状态	否
	fstatat	获取文件状态 (at 版本)	否
	ftruncate	截断文件到指定长度	否
	statfs	获取文件系统统计信息	否
	getcwd	获取当前工作目录	否
	getdents64	获取目录条目	否
	lseek	移动文件读写指针	否
	mkdirat	创建目录 (at 版本)	否
	mount	挂载文件系统	否
	openat	打开文件 (at 版本)	否
	pipe2	创建管道 (带有 flags)	否
	read	从文件读取数据	否
	readv	从文件读取数据到多个缓冲区	否
	pread64	从文件指定偏移处读取数据	否
	write	向文件写入数据	否
	writev	从多个缓冲区向文件写入数据	否
	pwrite64	向文件指定偏移处写入数据	否
	renameat	重命名文件 (at 版本, 这里调用 renameat2)	否
	sendfile	在文件描述符之间传输数据	否
	umount2	卸载文件系统	否
	unlinkat	删除文件 (at 版本)	否

续表 A.2 reL4-Linux-kit 的 Linux syscall 实现情况

syscall 分类	Syscall 名称	简要含义	是否为 fake 实现
	utimensat	设置文件时间戳（at 版本）	否
	msync	同步内存映射文件到磁盘	是
	sync	同步所有缓存到磁盘	是
	fsync	同步文件缓存到磁盘	是
系统信息	getrusage	获取资源使用情况	否
	uname	获取系统信息	否
信号处理	rtsigaction	设置信号处理函数	否
	rtsigprocmask	设置信号掩码	否
	rtsigreturn	从信号处理返回	否
	rtsigtimedwait	等待信号（带超时）	否
资源管理	prlimit64	设置或获取资源限制	否

A.3 Linux 测例 syscall 调用统计表 (aarch64)

表 A.3 Linux 测例 syscall 调用统计表 (aarch64)

Syscall Type	basic	run-static	busybox-testcase	lua	iozone	libc	lmbench
文件管理	getcwd, dup, dup3, mkdirat, unlinkat, umount2, mount, chdir, openat, close, get-dents64, read, write, fstat	getcwd, dup, dup3, fcntl, ioctl, unlinkat, statfs, chdir, openat, close, lseek, read, write, readv, writev, pread64, fstatat, fstat, utimensat	getcwd, dup3, fcntl, ioctl, mkdirat, unlinkat, openat, close, get-dents64, lseek, read, write, readv, writev, fstatat, utimensat	getcwd, fcntl, ioctl, unlinkat, re-nameat, faccessat, openat, close, read, write, readv, sendfile, ppoll	getcwd, fcntl, ioctl, unlinkat, openat, close, lseek, read, write, readv, writev, pread64, pwrite64, pselect6, fstatat, sync, fsync	unlinkat, openat, close, lseek, read, writev, ftruncate	getcwd, dup, fcntl, ioctl, unlinkat, openat, close, pipe2, lseek, read, write, writev, pselect6, fstatat, fstat, fsync, utimensat, umask
进程管理	exit, getpid, getppid, gettid, clone, execve, wait4, sched-yield	exit, exit-group, set-tid-address, getpid, getppid, getuid, geteuid, getegid, gettid, clone, execve, wait4, prlimit64, futex, set-robust-list, get-robust-list	exit, exit-group, set-tid-address, getpid, getppid, getuid, gettid, clone, execve, wait4	exit, exit-group, set-tid-address, getpid, getppid, getuid, gettid, clone, execve, wait4	exit, exit-group, set-tid-address, getpid, getppid, getuid, gettid, clone, execve, wait4	exit, exit-group, set-tid-address, gettid, clone, execve, wait4	exit, exit-group, set-tid-address, getrusage, getpid, getppid, getuid, gettid, clone, execve, wait4, prlimit64

续表 A.3 Linux 测例 syscall 调用统计表 (aarch64)

Syscall Type	basic	run-static	busybox-testcase	lua	iozone	libc	lmbench
内存管理	brk, munmap, mmap	brk, munmap, mmap, mprotect	brk, munmap, mmap	brk, munmap, mmap	brk, munmap, mmap, madvise	brk, munmap, mmap, madvise	brk, munmap, mmap, mprotect, msync
信号处理	rtsigaction, rtsig-proc-mask	kill, rtsigaction, rtsig-proc-mask, rtsig-timed-wait, rtsigreturn	rtsigaction, rtsig-proc-mask, rtsigreturn	rtsigaction, rtsig-proc-mask	rtsigaction, rtsig-proc-mask	rtsigaction, rtsig-proc-mask	kill, rtsigaction, rtsig-proc-mask, rtsigreturn
网络管理		socket, bind, listen, accept, connect, getsockname, sendto, recvfrom, setsockopt					
时间管理	nanosleep, gettimeofday	nanosleep	nanosleep		nanosleep		setitimer
系统信息	uname	uname	uname	uname, sysinfo	uname		uname
进程间通信	pipe2	pipe2	pipe2			shmget, shmctl, shmat	pipe2

致 谢

衷心感谢导师的精心指导。学生朽木。

声 明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果，不包含涉及国家秘密的内容。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____日 期：_____

个人简历、在学期间完成的相关学术成果

个人简历

197×年××月××日出生于四川××县。

1992年9月考入××大学化学系××化学专业，1996年7月本科毕业并获得理学学士学位。

1996年9月免试进入清华大学化学系攻读××化学博士至今。

在学期间完成的相关学术成果

学术论文：

- [1] Yang Y, Ren T L, Zhang L T, et al. Miniature microphone with silicon-based ferroelectric thin films[J]. Integrated Ferroelectrics, 2003, 52:229-235.
- [2] 杨轶, 张宁欣, 任天令, 等. 硅基铁电微声学器件中薄膜残余应力的研究 [J]. 中国机械工程, 2005, 16(14):1289-1291.
- [3] 杨轶, 张宁欣, 任天令, 等. 集成铁电器件中的关键工艺研究 [J]. 仪器仪表学报, 2003, 24(S4):192-193.
- [4] Yang Y, Ren T L, Zhu Y P, et al. PMUTs for handwriting recognition. In press[J]. (已被 Integrated Ferroelectrics 录用)

专利：

- [5] 任天令, 杨轶, 朱一平, 等. 硅基铁电微声学传感器畴极化区域控制和电极连接的方法: 中国, CN1602118A[P]. 2005-03-30.
- [6] Ren T L, Yang Y, Zhu Y P, et al. Piezoelectric micro acoustic sensor based on ferroelectric materials: USA, No.11/215, 102[P]. (美国发明专利申请号.)

指导教师评语

论文提出了……

答辩委员会决议书

论文提出了……

论文取得的主要创新性成果包括：

1. ……
2. ……
3. ……

论文工作表明作者在 ××××× 具有 ××××× 知识，具有 ×××× 能力，论文 ××××，
答辩 ××××。

答辩委员会表决，（× 票/一致）同意通过论文答辩，并建议授予 ×××（姓名）
×××（门类）学博士/硕士学位。